<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN"> <html><head>
<title>404 Not Found</title> </head><body> <h1>Not Found</h1>
The requested URL was not found on this server. <hr> <address>Apache/2.4.58
(Ubuntu) Server at ctan.math.utah.edu Port 443</address> </body></html>
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN"> <html><head>
<title>404 Not Found</title> </head><body> <h1>Not Found</h1>
The requested URL was not found on this server. </body></html>

xv6: a simple, Unix-like teaching operating system

Russ Cox Frans Kaashoek Robert Morris

28 марта 2026 г.

Оглавление

1 Интерфейсы операционной системы 9
- 1.1 Процессы и память . . . 11
- 1.2 Ввод-вывод и файловые дескрипторы . . . 13
- 1.3 Каналы . . . 17
- 1.4 Файловая система . . . 18
- 1.5 Реальный мир . . . 20
- 1.6 Упражнения . . . 21

2 Организация операционной системы 23
- 2.1 Абстрагирование физических ресурсов . . . 24
- 2.2 Пользовательский режим, режим супервизора и системные вызовы . . . 25
- 2.3 Организация ядра . . . 26
- 2.4 Код: организация xv6 . . . 28
- 2.5 Обзор процессов . . . 29
- 2.6 Код: запуск xv6, первый процесс и системный вызов . . . 31
- 2.7 Модель безопасности . . . 32
- 2.8 Реальный мир . . . 33
- 2.9 Упражнения . . . 33

3 Таблицы страниц 35
- 3.1 Аппаратная поддержка страничной адресации . . . 35
- 3.2 Адресное пространство ядра . . . 38
- 3.3 Код: создание адресного пространства . . . 40
- 3.4 Выделение физической памяти . . . 41
- 3.5 Код: аллокатор физической памяти . . . 41
- 3.6 Адресное пространство процесса . . . 42
- 3.7 Код: sbrk . . . 44
- 3.8 Код: exec . . . 44
- 3.9 Реальный мир . . . 46
- 3.10 Упражнения . . . 47

4 Ловушки и системные вызовы 51
4.1 Механизм ловушек RISC-V . . . 52
4.2 Ловушки из пространства пользователя . . . 53
4.3 Код: вызов системных вызовов . . . 55
4.4 Код: аргументы системных вызовов . . . 56
4.5 Ловушки из пространства ядра . . . 57
4.6 Исключения при обращении к страницам . . . 58
4.7 Реальный мир . . . 61
4.8 Упражнения . . . 61

5 Прерывания и драйверы устройств 63
5.1 Код: ввод с консоли . . . 63
5.2 Код: вывод на консоль . . . 65
5.3 Конкурентность в драйверах . . . 65
5.4 Прерывания таймера . . . 66
5.5 Реальный мир . . . 67
5.6 Упражнения . . . 68

6 Блокировки 69
6.1 Гонки . . . 70
6.2 Код: блокировки . . . 73
6.3 Код: использование блокировок . . . 74
6.4 Взаимная блокировка и порядок захвата блокировок . . . 76
6.5 Реентерабельные блокировки . . . 77
6.6 Блокировки и обработчики прерываний . . . 78
6.7 Упорядочивание инструкций и памяти . . . 79
6.8 Блокировки сна . . . 80
6.9 Реальный мир . . . 81
6.10 Упражнения . . . 82

7 Планирование 83
7.1 Мультиплексирование . . . 83
7.2 Код: переключение контекста . . . 84
7.3 Код: планирование . . . 85
7.4 Код: mycpu и myproc . . . 87
7.5 Sleep и wakeup . . . 88
7.6 Код: Sleep и wakeup . . . 91
7.7 Код: Каналы . . . 92
7.8 Код: Wait, exit и kill . . . 93
7.9 Блокировка процесса . . . 95
7.10 Реальный мир . . . 96
7.11 Упражнения . . . 98

8 Файловая система 101
8.1 Обзор . . . 102
8.2 Уровень кэша буферов . . . 103
8.3 Код: кэш буферов . . . 104
8.4 Уровень журналирования . . . 105
8.5 Проектирование журнала . . . 106
8.6 Код: журналирование . . . 107
8.7 Код: аллокатор блоков . . . 108
8.8 Уровень inode . . . 109
8.9 Код: inode . . . 110
8.10 Код: содержимое inode . . . 112
8.11 Код: уровень каталогов . . . 114
8.12 Код: пути . . . 114
8.13 Уровень файловых дескрипторов . . . 115
8.14 Код: системные вызовы . . . 116
8.15 Реальный мир . . . 118
8.16 Упражнения . . . 119

9 Параллелизм: дополнительные аспекты 121
9.1 Паттерны блокировок . . . 121
9.2 Паттерны, похожие на блокировки . . . 122
9.3 Полное отсутствие блокировок . . . 123
9.4 Параллелизм . . . 124
9.5 Упражнения . . . 125

10 Заключение 127

Предисловие и благодарности

Это черновой учебный текст, предназначенный для курса по операционным системам. Он объясняет основные концепции операционных систем на примере учебного ядра xv6. Xv6 создан по образцу Unix Version 6 (v6) [17] Дениса Ритчи и Кена Томпсона. Xv6 следует структуре и стилю v6 в общих чертах, но реализован на ANSI C [7] для многоядерной платформы RISC-V [15].

Этот текст следует читать вместе с исходным кодом xv6 — такой подход вдохновлён «Комментарием к UNIX 6-го издания» Джона Лайонса [11]. Ссылки на онлайн-ресурсы по v6 и xv6, включая ряд лабораторных заданий на базе xv6, см. на https://pdos.csail.mit.edu/6.1810.

Мы использовали этот текст в курсах 6.828 и 6.1810 по операционным системам в MIT. Мы благодарим преподавателей, ассистентов и студентов этих курсов, которые прямо или косвенно внесли вклад в xv6. В особенности мы хотели бы поблагодарить Адама Белая (Adam Belay), Остина Клементса (Austin Clements) и Николая Зельдовича (Nickolai Zeldovich). Также мы хотели бы поблагодарить тех, кто присылал нам сообщения об ошибках в тексте или предложения по его улучшению: Abutalib Aghayev, Sebastian Boehm, brandb97, Anton Burtsev, Raphael Carvalho, Tej Chajed, Rasit Eskicioglu, Color Fuzzy, Wojciech Gac, Giuseppe, Tao Guo, Haibo Hao, Naoki Hayama, Chris Henderson, Robert Hilderman, Eden Hochbaum, Wolfgang Keller, Henry Laih, Jin Li, Austin Liew, Pavan Maddamsetti, Jacek Masiulaniec, Michael McConville, m3hm00d, miguelgvieira, Mark Morrissey, Muhammed Mourad, Harry Pan, Harry Porter, Siyuan Qian, Askar Safin, Salman Shah, Huang Sha, Vikram Shenoy, Adeodato Simó, Ruslan Savchenko, Pawel Szczurko, Warren Toomey, tyfkda, tzerbib, Vanush Vaswani, Xi Wang, и Zou Chang Wei, Sam Whitlock, LucyShawYang, и Meng Zhou.

Если вы обнаружите ошибки или у вас есть предложения по улучшению, пожалуйста, напишите Франсу Касхуку и Роберту Моррису (kaashoek,rtm@csail.mit.edu).

Глава 1

Интерфейсы операционной системы

Задача операционной системы — разделить компьютер между несколькими программами и предоставить более удобный набор сервисов, чем тот, что поддерживается аппаратурой напрямую. Операционная система управляет низкоуровневым оборудованием и абстрагирует его так, что, например, текстовому редактору не нужно знать, какой конкретно тип дискового оборудования используется. Операционная система делит аппаратные ресурсы между несколькими программами, чтобы они работали (или казалось, что работают) одновременно. Наконец, операционные системы предоставляют программам контролируемые способы взаимодействия, позволяя им обмениваться данными или совместно выполнять задачи.

Операционная система предоставляет сервисы пользовательским программам через интерфейс. Разработать хороший интерфейс — непростая задача. С одной стороны, хотелось бы, чтобы интерфейс был простым и компактным — это облегчает корректную реализацию. С другой стороны, может возникнуть искушение предложить приложениям множество сложных функциональных возможностей. Решение этого противоречия — проектировать интерфейсы, опирающиеся на несколько механизмов, которые можно комбинировать для обеспечения широкой функциональности.

Эта книга использует одну операционную систему в качестве конкретного примера для иллюстрации концепций операционных систем. Эта операционная система, xv6, реализует базовые интерфейсы, введённые операционной системой Unix Кена Томпсона и Денниса Ритчи [17], а также воспроизводит внутреннюю архитектуру Unix. Unix предоставляет компактный интерфейс, механизмы которого хорошо сочетаются друг с другом, обеспечивая удивительную степень универсальности. Этот интерфейс оказался настолько удачным, что современные операционные системы — BSD, Linux, macOS, Solaris, и даже, в меньшей степени, Microsoft Windows — имеют Unix-подобные интерфейсы. Понимание xv6 — хорошая отправная точка для понимания любой из этих систем и многих других.

Как показано на рисунке 1.1, xv6 имеет традиционную форму ядра — специальной программы, предоставляющей сервисы выполняющимся программам. Каждая выполняющаяся программа, называемая процессом, имеет память, содержащую инструкции, данные и стек. Инструкции реализуют вычисления программы. Данные — это пере-

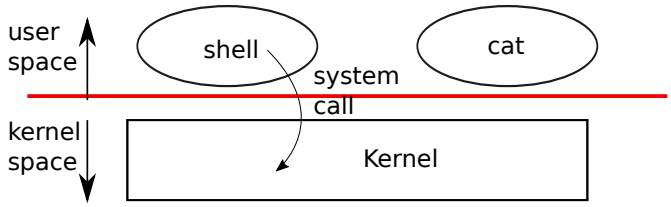

Рис. 1.1: Ядро и два пользовательских процесса.

менные, над которыми производятся вычисления. Стек организует вызовы процедур программы. На данном компьютере обычно работает много процессов, но только одно ядро.

Когда процессу нужно воспользоваться сервисом ядра, он вызывает системный вызов — один из вызовов в интерфейсе операционной системы. Системный вызов передаёт управление ядру; ядро выполняет запрошенный сервис и возвращает управление. Таким образом, процесс попеременно выполняется в пространстве пользователя и в пространстве ядра.

Как подробно описано в последующих главах, ядро использует механизмы аппаратной защиты, предоставляемые процессором[1], чтобы гарантировать, что каждый процесс, выполняющийся в пространстве пользователя, может обращаться только к собственной памяти. Ядро выполняется с аппаратными привилегиями, необходимыми для реализации этих защитных механизмов; пользовательские программы выполняются без этих привилегий. Когда пользовательская программа вызывает системный вызов, аппаратура повышает уровень привилегий и начинает выполнять заранее определённую функцию в ядре.

Набор системных вызовов, предоставляемых ядром, и есть тот интерфейс, который видят пользовательские программы. Ядро xv6 предоставляет подмножество сервисов и системных вызовов, традиционно предлагаемых ядрами Unix. На рисунке 1.2 перечислены все системные вызовы xv6.

Остальная часть этой главы описывает сервисы xv6 — процессы, память, файловые дескрипторы, каналы и файловую систему — и иллюстрирует их фрагментами кода и обсуждением того, как оболочка (shell), интерфейс командной строки Unix, использует их. Использование системных вызовов в оболочке демонстрирует, насколько тщательно они спроектированы.

Оболочка — это обычная программа, которая читает команды от пользователя и выполняет их. То, что оболочка является пользовательской программой, а не частью ядра, демонстрирует мощь интерфейса системных вызовов: в оболочке нет ничего особенного. Это также означает, что оболочку легко заменить; в результате современные Unix-системы предлагают разнообразные оболочки на выбор, каждая со своим пользовательским интерфейсом и возможностями сценариев. Оболочка xv6 — это простая

[1]В этом тексте аппаратный элемент, выполняющий вычисления, как правило, обозначается термином CPU — аббревиатурой от центрального процессора (central processing unit). В другой документации (например, в спецификации RISC-V) вместо CPU также используются слова processor, core и hart.

Системный вызов	Описание
int fork()	Создать процесс, вернуть PID дочернего процесса.
int exit(int status)	Завершить текущий процесс; статус передаётся в wait(). Возврат
int wait(int *status)	Ждать завершения дочернего процесса; статус выхода в *status;
int kill(int pid)	Завершить процесс с PID. Возвращает 0 или -1 при ошибке.
int getpid()	Вернуть PID текущего процесса.
int sleep(int n)	Приостановиться на n тактов таймера.
int exec(char *file, char *argv[])	Загрузить файл и выполнить его с аргументами; возвращает упр
char *sbrk(int n)	Увеличить память процесса на n байт. Возвращает начало новой
int open(char *file, int flags)	Открыть файл; флаги задают режим чтения/записи; возвращает
int write(int fd, char *buf, int n)	Записать n байт из buf в файловый дескриптор fd; возвращает n
int read(int fd, char *buf, int n)	Прочитать n байт в buf; возвращает число прочитанных байт ил
int close(int fd)	Освободить открытый файловый дескриптор fd.
int dup(int fd)	Вернуть новый файловый дескриптор, ссылающийся на тот же ф
int pipe(int p[])	Создать канал, поместить файловые дескрипторы чтения/записи
int chdir(char *dir)	Сменить текущий каталог.
int mkdir(char *dir)	Создать новый каталог.
int mknod(char *file, int, int)	Создать файл устройства.
int fstat(int fd, struct stat *st)	Поместить информацию об открытом файле в *st.
int stat(char *file, struct stat *st)	Поместить информацию об именованном файле в *st.
int link(char *file1, char *file2)	Создать ещё одно имя (file2) для файла file1.
int unlink(char *file)	Удалить файл.

Рис. 1.2: Системные вызовы xv6. Если не указано иное, эти вызовы возвращают 0 при отсутствии ошибки и -1 при наличии ошибки.

реализация сути Unix Bourne shell. Её реализация находится по ссылке (user/sh.c:1).

1.1 Процессы и память

Процесс xv6 состоит из пользовательской памяти (инструкции, данные и стек) и состояния, приватного для ядра и относящегося к данному процессу. Xv6 разделяет время между процессами: он прозрачно переключает доступные CPU между множеством процессов, ожидающих выполнения. Когда процесс не выполняется, xv6 сохраняет регистры CPU процесса и восстанавливает их при следующем запуске. Ядро связывает с каждым процессом идентификатор процесса (process identifier), или PID.

Процесс может создать новый процесс с помощью системного вызова fork. fork создаёт новому процессу точную копию памяти вызывающего процесса —и инструкции, и данные. fork возвращает управление как в исходном, так и в новом процессах. В исходном процессе fork возвращает PID нового процесса. В новом процессе fork возвращает ноль. Исходный и новый процессы обычно называются родительским и дочерним.

Например, рассмотрим следующий фрагмент программы на языке программирова-

ния C [7]:

```
int pid = fork();
if(pid > 0){
  printf("parent: child=%d\n", pid);
  pid = wait((int *) 0);
  printf("child %d is done\n", pid);
} else if(pid == 0){
  printf("child: exiting\n");
  exit(0);
} else {
  printf("fork error\n");
}
```

Системный вызов exit заставляет вызывающий процесс прекратить выполнение и освободить ресурсы, такие как память и открытые файлы. Exit принимает целочисленный аргумент статуса, по соглашению 0 означает успех, а 1 — неудачу. Системный вызов wait возвращает PID завершившегося (или убитого) дочернего процесса текущего процесса и копирует статус завершения дочернего процесса по адресу, переданному в wait; если ни один из дочерних процессов вызывающего ещё не завершился, wait ожидает завершения одного из них. Если у вызывающего нет дочерних процессов, wait немедленно возвращает -1. Если родителю не важен статус завершения дочернего процесса, он может передать нулевой адрес в wait.

В данном примере строки вывода

```
parent: child=1234
child: exiting
```

могут появиться в любом порядке (или даже вперемешку), в зависимости от того, родительский или дочерний процесс первым выполнит вызов printf. После завершения дочернего процесса вызов wait родителя возвращает управление, и родитель выводит

```
parent: child 1234 is done
```

Хотя изначально дочерний процесс имеет то же содержимое памяти, что и родительский, они выполняются с отдельной памятью и отдельными регистрами: изменение переменной в одном не влияет на другой. Например, когда возвращаемое значение wait сохраняется в pid в родительском процессе, это не изменяет переменную pid в дочернем. Значение pid в дочернем процессе по-прежнему будет равно нулю.

Системный вызов exec заменяет память вызывающего процесса новым образом памяти, загруженным из файла, хранящегося в файловой системе. Файл должен иметь определённый формат, который указывает, какая часть файла содержит инструкции, какая часть — данные, с какой инструкции начинать выполнение и т.д. Xv6 использует формат ELF, который подробнее рассматривается в главе 3. Обычно файл является результатом компиляции исходного кода программы. Когда exec завершается успешно, он не возвращает управление вызывающей программе; вместо этого инструкции, загруженные из файла, начинают выполняться с точки входа, объявленной в заголовке ELF.

exec принимает два аргумента: имя файла, содержащего исполняемый код, и массив строковых аргументов. Например:

```
char *argv[3];

argv[0] = "echo";
argv[1] = "hello";
argv[2] = 0;
exec("/bin/echo", argv);
printf("exec error\n");
```

Этот фрагмент заменяет вызывающую программу экземпляром программы /bin/echo, запущенной со списком аргументов echo hello. Большинство программ игнорируют первый элемент массива аргументов, которым по соглашению является имя самой программы.

Оболочка xv6 использует описанные вызовы для запуска программ от имени пользователей. Основная структура оболочки проста; см. main (user/sh.c:146). Основной цикл читает строку ввода от пользователя с помощью getcmd. Затем он вызывает fork, который создаёт копию процесса оболочки. Родительский процесс вызывает wait, а дочерний выполняет команду. Например, если пользователь ввёл “echo hello” в оболочке, runcmd будет вызван с аргументом “echo hello”. runcmd (user/sh.c:55) выполняет саму команду. Для “echo hello” он вызовет exec (user/sh.c:79). Если exec завершится успешно, дочерний процесс будет выполнять инструкции из echo вместо runcmd. В какой-то момент echo вызовет exit, что заставит родительский процесс вернуться из wait в main (user/sh.c:146).

Может возникнуть вопрос, почему fork и exec не объединены в один вызов; позже мы увидим, что оболочка использует это разделение в своей реализации перенаправления ввода-вывода. Чтобы избежать расточительности, связанной с созданием дубликата процесса и его немедленной заменой (с помощью exec), ядра операционных систем оптимизируют реализацию fork для этого случая, используя техники виртуальной памяти, такие как копирование при записи (copy-on-write) (см. раздел 4.6).

Xv6 выделяет большую часть пользовательской памяти неявно: fork выделяет память, необходимую для копии памяти родителя в дочернем процессе, а exec выделяет достаточно памяти для размещения исполняемого файла. Процесс, которому во время выполнения требуется больше памяти (например, для malloc), может вызвать sbrk(n), чтобы увеличить свою область данных на n байт; sbrk возвращает адрес начала новой памяти.

1.2 Ввод-вывод и файловые дескрипторы

Файловый дескриптор —это небольшое целое число, представляющее управляемый ядром объект, из которого процесс может читать или в который может писать. Процесс может получить файловый дескриптор, открыв файл, каталог или устройство, создав канал или продублировав существующий дескриптор. Для простоты мы часто будем называть объект, на который ссылается файловый дескриптор, «файлом»; интерфейс

файловых дескрипторов абстрагирует различия между файлами, каналами и устройствами, делая их все похожими на потоки байтов. Ввод и вывод мы будем называть вводом-выводом (I/O).

Внутри ядро xv6 использует файловый дескриптор как индекс в таблице, относящейся к конкретному процессу, так что каждый процесс имеет собственное пространство файловых дескрипторов, начинающееся с нуля. По соглашению процесс читает из файлового дескриптора 0 (стандартный ввод), записывает вывод в файловый дескриптор 1 (стандартный вывод) и записывает сообщения об ошибках в файловый дескриптор 2 (стандартный вывод ошибок). Как мы увидим, оболочка использует это соглашение для реализации перенаправления ввода-вывода и конвейеров. Оболочка гарантирует, что у неё всегда открыты три файловых дескриптора (user/sh.c:152), которые по умолчанию являются дескрипторами для консоли.

Системные вызовы read и write читают байты из и записывают байты в открытые файлы, обозначенные файловыми дескрипторами. Вызов read(fd, buf, n) читает не более n байт из файлового дескриптора fd, копирует их в buf и возвращает количество прочитанных байт. Каждый файловый дескриптор, ссылающийся на файл, имеет связанное с ним смещение. read читает данные с текущего смещения в файле, а затем продвигает это смещение на количество прочитанных байт: следующий вызов read вернёт байты, следующие за теми, что были возвращены первым read. Когда больше нет байт для чтения, read возвращает ноль, сигнализируя о конце файла.

Вызов write(fd, buf, n) записывает n байт из buf в файловый дескриптор fd и возвращает количество записанных байт. Меньше n байт записывается только при возникновении ошибки. Как и read, write записывает данные по текущему смещению в файле, а затем продвигает это смещение на количество записанных байт: каждый вызов write продолжает с того места, где остановился предыдущий.

Следующий фрагмент программы (составляющий суть программы cat) копирует данные со стандартного ввода на стандартный вывод. При возникновении ошибки он записывает сообщение на стандартный вывод ошибок.

```
char buf[512];
int n;

for(;;){
  n = read(0, buf, sizeof buf);
  if(n == 0)
    break;
  if(n < 0){
    fprintf(2, "read error\n");
    exit(1);
  }
  if(write(1, buf, n) != n){
    fprintf(2, "write error\n");
    exit(1);
  }
```

```
}
```

Важно отметить, что в этом фрагменте кода cat не знает, читает ли он из файла, консоли или канала. Аналогично cat не знает, выводит ли он данные на консоль, в файл или куда-либо ещё. Использование файловых дескрипторов и соглашения о том, что файловый дескриптор 0 является вводом, а файловый дескриптор 1 — выводом, позволяет реализовать cat простым способом.

Системный вызов close освобождает файловый дескриптор, делая его доступным для повторного использования будущими вызовами open, pipe или dup (см. ниже). Вновь выделенный файловый дескриптор всегда является наименьшим незанятым дескриптором текущего процесса.

Файловые дескрипторы и fork взаимодействуют, упрощая реализацию перенаправления ввода-вывода. fork копирует таблицу файловых дескрипторов родителя вместе с его памятью, так что дочерний процесс начинает работу с теми же открытыми файлами, что и родительский. Системный вызов exec заменяет память вызывающего процесса, но сохраняет его таблицу файлов. Это поведение позволяет оболочке реализовать перенаправление ввода-вывода путём порождения дочернего процесса,

повторного открытия выбранных файловых дескрипторов в нём и последующего вызова exec для запуска новой программы. Вот упрощённая версия кода, который оболочка выполняет для команды cat < input.txt:

```
char *argv[2];

argv[0] = "cat";
argv[1] = 0;
if(fork() == 0) {
  close(0);
  open("input.txt", O_RDONLY);
  exec("cat", argv);
}
```

После того как дочерний процесс закрывает файловый дескриптор 0, open гарантированно использует именно этот дескриптор для вновь открытого input.txt: 0 будет наименьшим доступным файловым дескриптором. cat затем выполняется, причём файловый дескриптор 0 (стандартный ввод) ссылается на input.txt. Файловые дескрипторы родительского процесса не изменяются этой последовательностью действий, поскольку она модифицирует только дескрипторы дочернего.

Код перенаправления ввода-вывода в оболочке xv6 работает именно таким образом (user/sh.c:83). Напомним, что в данном месте кода оболочка уже породила дочерний процесс и что runcmd вызовет exec для загрузки новой программы.

Второй аргумент open представляет собой набор флагов, выраженных в виде битов, которые управляют поведением open. Возможные значения определены в заголовочном файле управления файлами (fcntl) (kernel/fcntl.h:1-5): O_RDONLY, O_WRONLY, O_RDWR, O_CREATE и O_TRUNC, которые указывают open открыть файл для чтения, только для записи, для чтения и записи, создать файл, если он не существует, и

усечь файл до нулевой длины соответственно.

Теперь должно быть понятно, почему полезно то, что fork и exec являются отдельными вызовами: между ними у оболочки есть возможность перенаправить ввод-вывод дочернего процесса, не нарушая настройки ввода-вывода основной оболочки. Можно было бы представить гипотетический комбинированный системный вызов forkexec, но варианты перенаправления ввода-вывода с таким вызовом выглядели бы громоздко. Оболочка могла бы изменить собственные настройки ввода-вывода перед вызовом forkexec (и затем отменить эти изменения); или forkexec мог бы принимать инструкции по перенаправлению ввода-вывода в качестве аргументов; или (наименее привлекательный вариант) каждая программа вроде cat должна была бы самостоятельно выполнять перенаправление ввода-вывода.

Хотя fork копирует таблицу файловых дескрипторов, каждое базовое смещение файла разделяется между родительским и дочерним процессами. Рассмотрим следующий пример:

```
if(fork() == 0) {
  write(1, "hello ", 6);
  exit(0);
} else {
  wait(0);
  write(1, "world\n", 6);
}
```

В конце этого фрагмента файл, прикреплённый к файловому дескриптору 1, будет содержать данные hello world. Вызов write в родительском процессе (который, благодаря wait, выполняется только после завершения дочернего) продолжает с того места, где остановился write дочернего процесса. Это поведение помогает создавать последовательный вывод из последовательностей команд оболочки, например (echo hello; echo world) >output.txt.

Системный вызов dup дублирует существующий файловый дескриптор, возвращая новый, ссылающийся на тот же базовый объект ввода-вывода. Оба файловых дескриптора совместно используют смещение, как и файловые дескрипторы, продублированные с помощью fork. Вот ещё один способ записать hello world в файл:

```
fd = dup(1);
write(1, "hello ", 6);
write(fd, "world\n", 6);
```

Два файловых дескриптора совместно используют смещение, если они были получены из одного исходного файлового дескриптора посредством последовательности вызовов fork и dup. В противном случае файловые дескрипторы не разделяют смещения, даже если они были получены в результате вызовов open для одного и того же файла. dup позволяет оболочкам реализовывать такие команды: ls existing- file non-existing- file > tmp1 2>&1. Запись 2>&1 сообщает оболочке, что команде нужно передать файловый дескриптор 2, являющийся дубликатом дескриптора 1. И имя существующего файла, и сообщение об ошибке для несуществующего файла появятся в файле tmp1. Оболочка

xv6 не поддерживает перенаправление ввода-вывода для файлового дескриптора ошибок, но теперь вы знаете, как это реализовать.

Файловые дескрипторы являются мощной абстракцией, поскольку скрывают детали того, с чем они связаны: процесс, записывающий в файловый дескриптор 1, может записывать в файл, устройство вроде консоли или канал.

1.3 Каналы

Канал (pipe) — это небольшой буфер ядра, предоставляемый процессам в виде пары файловых дескрипторов: один для чтения, другой для записи. Запись данных на один конец канала делает эти данные доступными для чтения с другого конца. Каналы предоставляют процессам способ обмениваться данными.

Следующий пример кода запускает программу wc со стандартным вводом, подключённым к читающему концу канала.

```
int p[2];
char *argv[2];

argv[0] = "wc";
argv[1] = 0;

pipe(p);
if(fork() == 0) {
  close(0);
  dup(p[0]);
  close(p[0]);
  close(p[1]);
  exec("/bin/wc", argv);
} else {
  close(p[0]);
  write(p[1], "hello world\n", 12);
  close(p[1]);
}
```

Программа вызывает pipe, который создаёт новый канал и записывает файловые дескрипторы чтения и записи в массив p. После fork и у родительского, и у дочернего процесса есть файловые дескрипторы, ссылающиеся на канал. Дочерний процесс вызывает close и dup, чтобы файловый дескриптор ноль ссылался на читающий конец канала, закрывает файловые дескрипторы в p и вызывает exec для запуска wc. Когда wc читает со своего стандартного ввода, он читает из канала. Родительский процесс закрывает читающую сторону канала, записывает данные в канал и затем закрывает пишущую сторону.

Если данных нет, read на канале ждёт, пока не будут записаны данные или пока все файловые дескрипторы, ссылающиеся на пишущий конец, не будут закрыты; в последнем случае read вернёт 0, как если бы был достигнут конец файла данных. То, что

read блокируется до тех пор, пока не станет невозможным поступление новых данных, —одна из причин, почему важно, чтобы дочерний процесс закрыл пишущий конец канала перед запуском wc в примере выше: если бы один из файловых дескрипторов wc ссылался на пишущий конец канала, wc никогда не увидел бы конца файла.

Оболочка xv6 реализует конвейеры вроде grep fork sh.c | wc -l способом, аналогичным приведённому выше коду (user/sh.c:101). Дочерний процесс создаёт канал для соединения левого конца конвейера с правым. Затем он вызывает fork и runcmd для левого конца конвейера и fork и runcmd для правого конца, и ожидает завершения обоих. Правый конец конвейера может быть командой, которая сама включает канал (например, a | b | c), которая в свою очередь порождает два новых дочерних процесса (один для b и один для c). Таким образом, оболочка может создать дерево процессов. Листья этого дерева являются командами, а внутренние узлы —процессами, ожидающими завершения левого и правого дочерних процессов.

На первый взгляд каналы могут казаться не более мощными, чем временные файлы. Конвейер

```
echo hello world | wc
```

можно было бы реализовать без каналов следующим образом:

```
echo hello world >/tmp/xyz; wc </tmp/xyz
```

Каналы имеют как минимум три преимущества перед временными файлами в данной ситуации. Во-первых, каналы автоматически очищают себя; при перенаправлении через файл оболочке пришлось бы позаботиться об удалении /tmp/xyz по завершении работы. Во-вторых, каналы могут передавать произвольно длинные потоки данных, тогда как перенаправление через файл требует достаточно свободного места на диске для хранения всех данных. В-третьих, каналы обеспечивают параллельное выполнение этапов конвейера, тогда как файловый подход требует завершения первой программы перед запуском второй.

1.4 Файловая система

Файловая система xv6 предоставляет файлы данных, содержащие неинтерпретируемые массивы байтов, и каталоги, содержащие именованные ссылки на файлы данных и другие каталоги. Каталоги образуют дерево, начинающееся с специального каталога, называемого корневым. Путь вроде /a/b/c ссылается на файл или каталог с именем c внутри каталога с именем b внутри каталога с именем a в корневом каталоге /. Пути, не начинающиеся с /, вычисляются относительно текущего каталога вызывающего процесса, который можно изменить с помощью системного вызова chdir. Оба следующих фрагмента кода открывают один и тот же файл (при условии, что все задействованные каталоги существуют):

```
chdir("/a");
chdir("b");
```

```
open("c", O_RDONLY);

open("/a/b/c", O_RDONLY);
```

Первый фрагмент изменяет текущий каталог процесса на /a/b; второй не ссылается на текущий каталог процесса и не изменяет его.

Существуют системные вызовы для создания новых файлов и каталогов: mkdir создаёт новый каталог, open с флагом O_CREATE создаёт новый файл данных, а mknod создаёт новый файл устройства. Следующий пример иллюстрирует все три:

```
mkdir("/dir");
fd = open("/dir/file", O_CREATE|O_WRONLY);
close(fd);
mknod("/console", 1, 1);
```

mknod создаёт специальный файл, ссылающийся на устройство. С файлом устройства связаны старший и младший номера устройства (два аргумента mknod), которые однозначно идентифицируют устройство ядра. Когда процесс впоследствии открывает файл устройства, ядро перенаправляет системные вызовы read и write на реализацию этого устройства в ядре вместо передачи их файловой системе.

Имя файла отличается от самого файла; один и тот же базовый файл, называемый иноде (inode), может иметь несколько имён, называемых ссылками (links). Каждая ссылка представляет собой запись в каталоге; запись содержит имя файла и ссылку на инод. Инод хранит метаданные о файле, включая его тип (файл, каталог или устройство), его длину, расположение содержимого файла на диске и количество ссылок на файл.

Системный вызов fstat извлекает информацию из инода, на который ссылается файловый дескриптор. Он заполняет структуру struct stat, определённую в stat.h (kernel/stat.h) следующим образом:

```
#define T_DIR     1   // Directory
#define T_FILE    2   // File
#define T_DEVICE  3   // Device

struct stat {
  int dev;     // File system's disk device
  uint ino;    // Inode number
  short type;  // Type of file
  short nlink; // Number of links to file
  uint64 size; // Size of file in bytes
};
```

Системный вызов link создаёт ещё одно имя в файловой системе, ссылающееся на тот же инод, что и существующий файл. Следующий фрагмент создаёт новый файл с именами a и b.

```
open("a", O_CREATE|O_WRONLY);
link("a", "b");
```

Чтение из a или запись в него равнозначны чтению из b или записи в него. Каждый инод идентифицируется уникальным номером инода. После приведённой выше последовательности кода можно убедиться, что a и b ссылаются на одно и то же содержимое, изучив результат fstat: оба вернут одинаковый номер инода (ino), а счётчик nlink будет равен 2.

Системный вызов unlink удаляет имя из файловой системы. Инод файла и занятое его содержимым дисковое пространство освобождаются только тогда, когда счётчик ссылок на файл равен нулю и ни один файловый дескриптор не ссылается на него. Таким образом, добавление

```
unlink("a");
```

к последней последовательности кода оставит инод и содержимое файла доступными через имя b. Кроме того,

```
fd = open("/tmp/xyz", O_CREATE|O_RDWR);
unlink("/tmp/xyz");
```

является идиоматическим способом создания временного инода без имени, который будет очищен, когда процесс закроет fd или завершится.

Unix предоставляет файловые утилиты, вызываемые из оболочки как пользовательские программы, например mkdir, ln и rm. Такой дизайн позволяет любому расширить интерфейс командной строки путём добавления новых пользовательских программ. Оглядываясь назад, этот подход кажется очевидным, но другие системы, разработанные в то же время, что и Unix, зачастую встраивали подобные команды прямо в оболочку (а оболочку — в ядро).

Исключением является cd, который встроен в оболочку cd должен изменить текущий рабочий каталог самой оболочки. Если бы cd выполнялась как обычная команда, оболочка породила бы дочерний процесс, дочерний процесс выполнил бы cd, и cd изменил бы рабочий каталог дочернего процесса. Рабочий каталог родителя (то есть самой оболочки) не изменился бы.

1.5 Реальный мир

Сочетание в Unix «стандартных» файловых дескрипторов, каналов и удобного синтаксиса оболочки для операций над ними стало серьёзным прорывом в написании универсальных повторно используемых программ. Эта идея породила культуру «программных инструментов», которая во многом обусловила силу и популярность Unix, а оболочка стала первым так называемым «языком сценариев». Интерфейс системных вызовов Unix сохраняется по сей день в таких системах, как BSD, Linux и macOS.

Интерфейс системных вызовов Unix был стандартизирован посредством стандарта Portable Operating System Interface (POSIX). Xv6 не соответствует POSIX: в нём отсутствуют многие системные вызовы (в том числе базовые, такие как lseek), а многие из имеющихся системных вызовов отличаются от стандарта. Наши основные цели для xv6 — простота и ясность при обеспечении простого Unix-подобного интерфейса

системных вызовов. Ряд людей расширил xv6 несколькими дополнительными системными вызовами и простой библиотекой C для запуска базовых Unix-программ. Однако современные ядра предоставляют гораздо больше системных вызовов и гораздо больше видов сервисов, чем xv6. Например, они поддерживают сетевое взаимодействие, оконные системы, потоки пользовательского уровня, драйверы для множества устройств и многое другое. Современные ядра постоянно и стремительно развиваются и предлагают множество возможностей, выходящих за рамки POSIX.

Unix унифицировал доступ к нескольким типам ресурсов (файлы, каталоги и устройства) через единый набор интерфейсов имён файлов и файловых дескрипторов. Эта идея может быть распространена на большее число видов ресурсов; хороший пример —Plan 9 [16], в котором концепция «ресурсы суть файлы» была применена к сетям, графике и другим областям. Тем не менее большинство Unix-производных операционных систем не последовали этим путём.

Файловая система и файловые дескрипторы оказались мощными абстракциями. Тем не менее существуют и другие модели интерфейсов операционных систем. Multics, предшественник Unix, абстрагировал хранилище файлов таким образом, что оно выглядело как память, порождая совершенно иной стиль интерфейса. Сложность дизайна Multics оказала непосредственное влияние на разработчиков Unix, которые стремились создать нечто более простое.

Xv6 не поддерживает понятие пользователей и защиту одного пользователя от другого; говоря языком Unix, все процессы xv6 выполняются с правами root.

В этой книге рассматривается то, как xv6 реализует свой Unix-подобный интерфейс, однако описанные идеи и концепции применимы не только к Unix. Любая операционная система должна мультиплексировать процессы на базовом оборудовании, изолировать процессы друг от друга и предоставлять механизмы для контролируемого межпроцессного взаимодействия. Изучив xv6, вы сможете взглянуть на другие, более сложные операционные системы и увидеть в них концепции, лежащие в основе xv6.

1.6 Упражнения

1. Напишите программу, которая использует системные вызовы UNIX для «пинг-понга» байта между двумя процессами через пару каналов —по одному в каждом направлении. Измерьте производительность программы в количестве обменов в секунду.

Глава 2

Организация операционной системы

Ключевое требование к операционной системе — поддержка нескольких задач одновременно. Например, используя интерфейс системных вызовов, описанный в Chapter 1, процесс может порождать новые процессы с помощью fork. Операционная система должна разделять по времени ресурсы компьютера между этими процессами. Например, даже если процессов больше, чем аппаратных процессоров, операционная система обязана гарантировать, что каждый процесс получит возможность выполниться. Кроме того, операционная система должна обеспечивать изоляцию между процессами. То есть если один процесс содержит ошибку и работает некорректно, это не должно влиять на процессы, не зависящие от неисправного. Однако полная изоляция была бы слишком строгим ограничением, поскольку процессы должны иметь возможность намеренно взаимодействовать; примером служат конвейеры. Таким образом, операционная система должна удовлетворять трём требованиям: мультиплексирование, изоляция и взаимодействие.

В этой главе даётся общее представление о том, как операционные системы организованы для выполнения этих трёх требований. Существует множество способов достичь этого, однако данный учебник сосредоточен на широко распространённых архитектурах, построенных вокруг монолитного ядра, применяемого во многих Unix-системах. В главе также описывается процесс xv6 как единица изоляции и создание первого процесса при запуске xv6.

Xv6 работает на многоядерном[1] микропроцессоре RISC-V, и значительная часть его низкоуровневой функциональности (например, реализация процессов) специфична для RISC-V. RISC-V — это 64-битный процессор, а xv6 написан на «LP64»-диалекте C, то есть тип long (L) и указатели (P) в языке C занимают 64 бита, а int — 32 бита. В книге предполагается, что читатель имеет опыт программирования на уровне машинных инструкций для какой-либо архитектуры; особенности RISC-V будут вводиться по мере необходимости. Полезным справочником по RISC-V служит «The RISC-V Reader: An

[1] Под «многоядерным» в данном тексте подразумевается несколько процессоров, совместно использующих память, но выполняющих инструкции параллельно, каждый со своим набором регистров. Иногда в тексте используется термин мультипроцессор как синоним многоядерного, хотя мультипроцессор может также обозначать компьютер с несколькими отдельными процессорными чипами.

Open Architecture Atlas» [15]. Пользовательский ISA [2] и привилегированная архитектура [3] являются официальными спецификациями.

Процессор в полноценном компьютере окружён вспомогательным оборудованием, большая часть которого представляет собой интерфейсы ввода-вывода. Xv6 написан для аппаратного обеспечения, эмулируемого параметром qemu «-machine virt». Это включает ОЗУ, ПЗУ с загрузочным кодом, последовательный интерфейс для клавиатуры и экрана пользователя, а также диск для хранения данных.

2.1 Абстрагирование физических ресурсов

Первый вопрос, который может возникнуть при знакомстве с операционной системой: зачем она вообще нужна? Системные вызовы из Figure 1.2 можно было бы реализовать в виде библиотеки, с которой линкуются приложения. При таком подходе каждое приложение могло бы иметь собственную библиотеку, адаптированную под его нужды. Приложения могли бы напрямую взаимодействовать с аппаратными ресурсами и использовать их наиболее подходящим способом (например, для достижения высокой или предсказуемой производительности). Некоторые операционные системы для встраиваемых устройств или систем реального времени организованы именно так.

Недостаток библиотечного подхода заключается в том, что при наличии более одного запущенного приложения все приложения должны вести себя корректно. Например, каждое приложение должно периодически уступать процессор, чтобы другие приложения могли выполняться. Такая кооперативная схема разделения времени может быть приемлема, если все приложения доверяют друг другу и не содержат ошибок. Однако чаще всего приложения не доверяют друг другу и содержат ошибки, поэтому зачастую требуется более строгая изоляция, чем обеспечивает кооперативная схема.

Для достижения строгой изоляции полезно запретить приложениям напрямую обращаться к чувствительным аппаратным ресурсам и вместо этого предоставлять доступ к ресурсам через сервисы. Например, Unix-приложения взаимодействуют с хранилищем только через системные вызовы файловой системы — open, read, write и close, — а не путём прямого чтения и записи диска. Это даёт приложению удобство работы с путями к файлам и позволяет операционной системе (как реализатору интерфейса) управлять диском. Даже если изоляция не является приоритетом, программы, которые намеренно взаимодействуют (или просто хотят не мешать друг другу), скорее всего, найдут файловую систему более удобной абстракцией, чем прямую работу с диском.

Аналогично, Unix прозрачно переключает аппаратные процессоры между процессами, сохраняя и восстанавливая состояние регистров по мере необходимости, так что приложениям не нужно знать о разделении времени. Эта прозрачность позволяет операционной системе делить процессоры даже в том случае, если некоторые приложения находятся в бесконечных циклах.

В качестве ещё одного примера: Unix-процессы используют exec для формирования своего образа в памяти, вместо того чтобы напрямую работать с физической памятью. Это позволяет операционной системе решать, куда поместить процесс в памяти; если

памяти мало, операционная система может даже выгрузить часть данных процесса на диск. exec также предоставляет пользователям удобство файловой системы для хранения исполняемых образов программ.

Многие формы взаимодействия между Unix-процессами осуществляются через файловые дескрипторы. Файловые дескрипторы не только абстрагируют многие детали (например, где хранятся данные в канале или файле), но и определены таким образом, что упрощают взаимодействие. Например, если одно приложение в конвейере завершается аварийно, ядро генерирует сигнал конца файла для следующего процесса в конвейере.

Интерфейс системных вызовов из Figure 1.2 тщательно спроектирован, чтобы обеспечить как удобство для программиста, так и возможность строгой изоляции. Интерфейс Unix — не единственный способ абстрагировать ресурсы, но он зарекомендовал себя как весьма удачный.

2.2 Пользовательский режим, режим супервизора и системные вызовы

Строгая изоляция требует жёсткой границы между приложениями и операционной системой. Если приложение допускает ошибку, мы не хотим, чтобы операционная система или другие приложения вышли из строя. Вместо этого операционная система должна уметь завершить аварийное приложение и продолжить выполнение остальных. Для достижения строгой изоляции операционная система должна обеспечить, чтобы приложения не могли изменять (и даже читать) структуры данных и инструкции операционной системы, а также не могли обращаться к памяти других процессов.

Процессоры предоставляют аппаратную поддержку строгой изоляции. Например, RISC-V имеет три режима, в которых процессор может выполнять инструкции: машинный режим, режим супервизора и пользовательский режим. Инструкции, выполняемые в машинном режиме, обладают полными привилегиями; процессор запускается в машинном режиме. Машинный режим предназначен в основном для конфигурирования компьютера. Xv6 выполняет несколько строк в машинном режиме, а затем переключается в режим супервизора.

В режиме супервизора процессору разрешено выполнять привилегированные инструкции: например, включать и отключать прерывания, читать и записывать регистр, содержащий адрес таблицы страниц, и т.д. Если приложение в пользовательском режиме пытается выполнить привилегированную инструкцию, процессор не выполняет её, а переключается в режим супервизора, чтобы код режима супервизора мог завершить приложение, совершившее недопустимое действие. Figure 1.1 в Chapter 1 иллюстрирует эту организацию. Приложение может выполнять только пользовательские инструкции (например, арифметические операции и т.п.) и считается работающим в пространстве пользователя, тогда как программное обеспечение в режиме супервизора может также выполнять привилегированные инструкции и считается работающим в пространстве ядра. Программное обеспечение, выполняемое в пространстве ядра (или в режиме су-

первизора), называется ядром.

Приложение, желающее вызвать функцию ядра (например, системный вызов read в xv6), должно перейти в ядро; приложение не может вызвать функцию ядра напрямую. Процессоры предоставляют специальную инструкцию, которая переключает процессор из пользовательского режима в режим супервизора и передаёт управление ядру в точку входа, заданную самим ядром. (RISC-V предоставляет инструкцию ecall для этой цели.) После переключения процессора в режим супервизора ядро может проверить аргументы системного вызова (например, убедиться, что адрес, переданный системному вызову, принадлежит памяти приложения), решить, разрешено ли приложению выполнять запрошенную операцию (например, проверить, разрешено ли приложению записывать в указанный файл), и затем отклонить запрос или выполнить его. Принципиально важно, чтобы ядро контролировало точку входа для переходов в режим супервизора; если бы приложение могло задавать точку входа в ядро, вредоносное приложение могло бы, например, войти в ядро в месте, где проверка аргументов пропускается.

2.3 Организация ядра

Ключевой вопрос проектирования —какая часть операционной системы должна выполняться в режиме супервизора. Один из вариантов: вся операционная система размещается в ядре, так что реализации всех системных вызовов выполняются в режиме супервизора. Такая организация называется монолитным ядром.

При такой организации вся операционная система работает с полными аппаратными привилегиями. Это удобно, поскольку разработчику ОС не приходится решать, какой части операционной системы полные аппаратные привилегии не нужны. Кроме того, различным частям операционной системы проще взаимодействовать. Например, операционная система может иметь буферный кеш, совместно используемый как файловой системой, так и системой виртуальной памяти.

Недостаток монолитной организации состоит в том, что интерфейсы между различными частями операционной системы нередко оказываются сложными (как мы увидим в остальных главах этой книги), что облегчает разработчику ОС допущение ошибок. В монолитном ядре ошибка фатальна: сбой в режиме супервизора, как правило, приводит к отказу ядра. При отказе ядра компьютер прекращает работу, и все приложения также завершаются аварийно. Для восстановления работы требуется перезагрузка.

Чтобы снизить риск ошибок в ядре, разработчики ОС могут минимизировать количество кода операционной системы, выполняемого в режиме супервизора, и перенести основную часть операционной системы в пользовательский режим. Такая организация ядра называется микроядром.

Figure 2.1 иллюстрирует архитектуру микроядра. На рисунке файловая система работает как процесс пользовательского уровня. Сервисы ОС, запущенные как процессы, называются серверами. Чтобы приложения могли взаимодействовать с сервером файловой системы, ядро предоставляет механизм межпроцессного взаимодействия для передачи сообщений из одного пользовательского процесса другому. Например, если

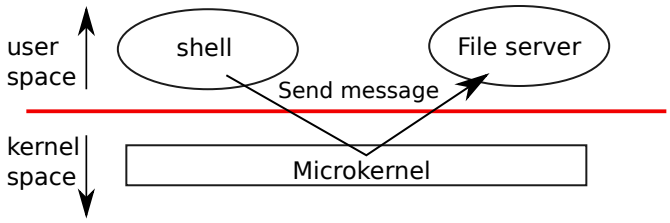

Рис. 2.1: Микроядро с сервером файловой системы

такое приложение, как командная оболочка, хочет прочитать или записать файл, оно посылает сообщение серверу файловой системы и ждёт ответа.

В микроядре интерфейс ядра состоит из нескольких низкоуровневых функций для запуска приложений, передачи сообщений, доступа к аппаратным устройствам и т.д. Такая организация позволяет ядру быть относительно простым, поскольку большая часть операционной системы находится в серверах пользовательского уровня.

В реальном мире широко применяются как монолитные ядра, так и микроядра. Многие Unix-ядра являются монолитными. Например, Linux имеет монолитное ядро, хотя некоторые функции ОС выполняются в виде серверов пользовательского уровня (например, оконная система). Linux обеспечивает высокую производительность для приложений, интенсивно использующих ОС, отчасти потому, что подсистемы ядра могут быть тесно интегрированы.

Операционные системы, такие как Minix, L4 и QNX, организованы как микроядро с серверами и получили широкое распространение во встраиваемых системах. Вариант L4 —seL4 —достаточно мал, что позволило формально верифицировать его безопасность по памяти и другие свойства безопасности [8].

Среди разработчиков операционных систем ведутся споры о том, какая организация лучше, и однозначного ответа нет. Кроме того, многое зависит от того, что именно понимается под «лучше»: более высокая производительность, меньший объём кода, надёжность ядра, надёжность всей операционной системы (включая сервисы пользовательского уровня) и т.д.

Существуют также практические соображения, которые могут оказаться важнее вопроса об организации. Некоторые операционные системы имеют микроядро, но запускают часть сервисов пользовательского уровня в пространстве ядра из соображений производительности. Некоторые операционные системы используют монолитные ядра просто потому, что именно так они были созданы, и нет достаточных стимулов переходить к чистой архитектуре микроядра: новые функции могут быть важнее переписывания существующей операционной системы в соответствии с проектом микроядра.

С точки зрения этой книги, операционные системы на основе микроядра и монолитные системы разделяют многие ключевые идеи. Они реализуют системные вызовы, используют таблицы страниц, обрабатывают прерывания, поддерживают процессы, применяют блокировки для управления параллелизмом, реализуют файловую систему и т.д. На этих основных идеях и сосредоточена данная книга.

Xv6 реализован как монолитное ядро, как большинство Unix-операционных систем.

Файл	Описание
bio.c	Кеш дисковых блоков для файловой системы.
console.c	Подключение к клавиатуре и экрану пользователя.
entry.S	Первые инструкции загрузки.
exec.c	Системный вызов exec().
file.c	Поддержка файловых дескрипторов.
fs.c	Файловая система.
kalloc.c	Распределитель физических страниц.
kernelvec.S	Обработка ловушек из ядра и прерываний таймера.
log.c	Журналирование файловой системы и восстановление после сбоев.
main.c	Управление инициализацией остальных модулей при загрузке.
pipe.c	Каналы (pipes).
plic.c	Контроллер прерываний RISC-V.
printf.c	Форматированный вывод на консоль.
proc.c	Процессы и планировщик.
sleeplock.c	Блокировки с уступкой процессора.
spinlock.c	Блокировки без уступки процессора.
start.c	Ранний загрузочный код машинного режима.
string.c	Библиотека строк и байтовых массивов C.
swtch.S	Переключение потоков.
syscall.c	Диспетчеризация системных вызовов к обработчикам.
sysfile.c	Системные вызовы, связанные с файлами.
sysproc.c	Системные вызовы, связанные с процессами.
trampoline.S	Ассемблерный код для переключения между пользователем и ядром.
trap.c	Код C для обработки ловушек и прерываний и возврата из них.
uart.c	Драйвер устройства консоли последовательного порта.
virtio_disk.c	Драйвер дискового устройства.
vm.c	Управление таблицами страниц и адресными пространствами.

Рис. 2.2: Исходные файлы ядра xv6.

Таким образом, интерфейс ядра xv6 соответствует интерфейсу операционной системы, а ядро реализует всю операционную систему. Поскольку xv6 не предоставляет большого количества сервисов, его ядро меньше некоторых микроядер, однако концептуально xv6 является монолитным.

2.4 Код: организация xv6

Исходный код ядра xv6 находится в подкаталоге kernel/. Исходный код разбит на файлы по приблизительному принципу модульности; Figure 2.2 перечисляет эти файлы. Межмодульные интерфейсы определены в defs.h (kernel/defs.h).

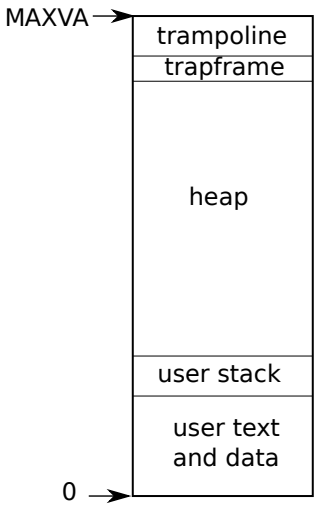

Рис. 2.3: Схема виртуального адресного пространства процесса

2.5 Обзор процессов

Единицей изоляции в xv6 (как и в других Unix-системах) является процесс. Абстракция процесса не позволяет одному процессу разрушать или прослушивать память, процессор, файловые дескрипторы и т.д. другого процесса. Она также не позволяет процессу разрушить само ядро, так что процесс не может обойти механизмы изоляции ядра. Ядро должно реализовывать абстракцию процесса с осторожностью, поскольку ошибочное или вредоносное приложение может обмануть ядро или аппаратуру, заставив их совершить нечто нежелательное (например, обойти изоляцию). К механизмам, используемым ядром для реализации процессов, относятся флаг пользовательского режима/режима супервизора, адресные пространства и квантование времени потоков.

Для обеспечения изоляции абстракция процесса создаёт у программы иллюзию наличия собственной выделенной машины. Процесс предоставляет программе то, что выглядит как собственная система памяти, или адресное пространство, которое другие процессы не могут читать или записывать. Процесс также предоставляет программе то, что выглядит как собственный процессор для выполнения инструкций программы.

Xv6 использует таблицы страниц (реализованные аппаратно), чтобы дать каждому процессу собственное адресное пространство. Таблица страниц RISC-V транслирует (или «отображает») виртуальный адрес (адрес, которым манипулирует инструкция RISC-V) в физический адрес (адрес, который процессор передаёт в оперативную память).

Xv6 поддерживает отдельную таблицу страниц для каждого процесса, определяющую его адресное пространство. Как показано на Figure 2.3, адресное пространство включает память пользователя процесса, начиная с виртуального адреса ноль. Сначала идут инструкции, затем глобальные переменные, затем стек и наконец область «кучи» (для malloc), которую процесс может расширять по мере необходимости. Существует

ряд факторов, ограничивающих максимальный размер адресного пространства процесса: указатели в RISC-V имеют ширину 64 бита; аппаратура использует только младшие 39 бит при поиске виртуальных адресов в таблицах страниц; а xv6 использует лишь 38 из этих 39 бит. Таким образом, максимальный адрес равен $2^{38} - 1 = 0x3ffffffff$, что соответствует MAXVA (kernel/riscv.h:380). В верхней части адресного пространства xv6 резервирует страницу для трамплина и страницу для отображения фрейма ловушки процесса. Xv6 использует эти две страницы для перехода в ядро и обратно; страница трамплина содержит код перехода в ядро и возврата из него, а отображение фрейма ловушки необходимо для сохранения/восстановления состояния пользовательского процесса, как будет объяснено в Chapter 4.

Ядро xv6 хранит множество элементов состояния для каждого процесса, собирая их в структуру struct proc (kernel/proc.h:85). Наиболее важными элементами состояния процесса в ядре являются его таблица страниц, стек ядра и состояние выполнения. Для обращения к элементам структуры proc используется нотация p->xxx; например, p->pagetable —указатель на таблицу страниц процесса.

Каждый процесс имеет поток выполнения (или просто поток), выполняющий инструкции процесса. Поток может быть приостановлен и впоследствии возобновлён. Для прозрачного переключения между процессами ядро приостанавливает текущий выполняемый поток и возобновляет поток другого процесса. Большая часть состояния потока (локальные переменные, адреса возврата функций) хранится в стеках потока. Каждый процесс имеет два стека: пользовательский стек и стек ядра (p->kstack). Когда процесс выполняет пользовательские инструкции, используется только его пользовательский стек, а стек ядра пуст. Когда процесс входит в ядро (при системном вызове или прерывании), код ядра выполняется на стеке ядра процесса; пока процесс находится в ядре, его пользовательский стек по-прежнему содержит сохранённые данные, но активно не используется. Поток процесса попеременно использует пользовательский стек и стек ядра. Стек ядра отделён (и защищён от пользовательского кода), чтобы ядро могло выполняться даже в случае, если процесс разрушил свой пользовательский стек.

Процесс может выполнить системный вызов, используя инструкцию RISC-V ecall. Эта инструкция повышает уровень аппаратных привилегий и изменяет счётчик команд на точку входа, определённую ядром. Код в точке входа переключается на стек ядра и выполняет инструкции ядра, реализующие системный вызов. По завершении системного вызова ядро переключается обратно на пользовательский стек и возвращается в пространство пользователя, выполняя инструкцию sret, которая понижает уровень аппаратных привилегий и возобновляет выполнение пользовательских инструкций сразу после инструкции системного вызова. Поток процесса может «заблокироваться» в ядре в ожидании ввода-вывода и возобновить работу с прерванного места после завершения ввода-вывода.

p->state указывает, выделен ли процесс, готов ли к выполнению, выполняется ли, ожидает ли ввода-вывода или завершается.

p->pagetable хранит таблицу страниц процесса в формате, ожидаемом аппаратурой RISC-V. Xv6 заставляет аппаратуру подкачки страниц использовать p->pagetable при

выполнении данного процесса в пространстве пользователя. Таблица страниц процесса также служит записью адресов физических страниц, выделенных для хранения памяти процесса.

Таким образом, процесс объединяет две проектные идеи: адресное пространство, создающее у процесса иллюзию собственной памяти, и поток, создающий у процесса иллюзию собственного процессора. В xv6 процесс состоит из одного адресного пространства и одного потока. В реальных операционных системах процесс может иметь несколько потоков для использования нескольких процессоров.

2.6 Код: запуск xv6, первый процесс и системный вызов

Чтобы xv6 стал более конкретным, рассмотрим, как ядро запускается и выполняет первый процесс. В последующих главах механизмы, упомянутые в этом обзоре, будут описаны подробнее.

При включении питания компьютера RISC-V выполняет инициализацию и запускает загрузчик, хранящийся в постоянной памяти. Загрузчик загружает ядро xv6 в память. Затем в машинном режиме процессор начинает выполнять xv6, начиная с _entry (kernel/entry.S:7). RISC-V запускается с отключённой аппаратурой подкачки страниц: виртуальные адреса напрямую отображаются в физические.

Загрузчик загружает ядро xv6 в память по физическому адресу 0x80000000. Причина, по которой ядро помещается по адресу 0x80000000, а не по адресу 0x0, заключается в том, что диапазон адресов 0x0:0x80000000 занят устройствами ввода-вывода.

Инструкции по адресу _entry устанавливают стек, чтобы xv6 мог выполнять код на C. Xv6 объявляет пространство для начального стека, stack0, в файле start.c (kernel/start.c:11). Код по адресу _entry загружает в регистр указателя стека sp адрес stack0+4096, т.е. вершину стека, поскольку стек на RISC-V растёт вниз. После того как у ядра есть стек, _entry вызывает код C по адресу start (kernel/start.c:15).

Функция start выполняет некоторые настройки, доступные только в машинном режиме, а затем переключается в режим супервизора. Для входа в режим супервизора RISC-V предоставляет инструкцию mret. Эта инструкция чаще всего используется для возврата из предыдущего вызова из режима супервизора в машинный режим. start не возвращается из такого вызова, а вместо этого имитирует его: устанавливает предыдущий режим привилегий в режим супервизора в регистре mstatus, задаёт адрес возврата равным main, записывая адрес main в регистр mepc, отключает трансляцию виртуальных адресов в режиме супервизора, записывая 0 в регистр таблицы страниц satp, и делегирует все прерывания и исключения в режим супервизора.

Перед переходом в режим супервизора start выполняет ещё одну задачу: программирует микросхему часов для генерации прерываний таймера. Завершив подготовительные действия, start «возвращается» в режим супервизора, вызывая mret. Это приводит к изменению счётчика команд на адрес main (kernel/main.c:11).

После того как main (kernel/main.c:11) инициализирует несколько устройств и подсистем, он создаёт первый процесс, вызывая userinit (kernel/proc.c:220). Первый процесс

выполняет небольшую программу на ассемблере RISC-V, которая выполняет первый системный вызов в xv6. initcode.S (user/initcode.S:3) загружает номер системного вызова exec, SYS_EXEC (kernel/syscall.h:8), в регистр a7, а затем вызывает ecall для повторного входа в ядро.

Ядро использует номер в регистре a7 в функции syscall (kernel/syscall.c:132) для вызова нужного системного вызова. Таблица системных вызовов (kernel/syscall.c:107) отображает SYS_EXEC на sys_exec, которую ядро и вызывает. Как мы видели в Chapter 1, exec заменяет память и регистры текущего процесса новой программой (в данном случае /init).

После того как ядро завершило выполнение exec, оно возвращается в пространство пользователя в процессе /init. init (user/init.c:15) при необходимости создаёт новый файл консольного устройства и открывает его как файловые дескрипторы 0, 1 и 2. Затем запускает командную оболочку на консоли. Система готова к работе.

2.7 Модель безопасности

Может возникнуть вопрос: как операционная система справляется с ошибочным или вредоносным кодом? Поскольку противодействие злому умыслу сложнее, чем борьба с случайными ошибками, эту тему разумно рассматривать в контексте безопасности. Ниже дан общий обзор типичных предположений о безопасности и целей в проектировании операционных систем.

Операционная система должна исходить из того, что пользовательский код процесса будет стараться всеми силами нарушить работу ядра или других процессов. Пользовательский код может пытаться разыменовывать указатели за пределами разрешённого адресного пространства; пытаться выполнять любые инструкции RISC-V, в том числе не предназначенные для пользовательского кода; пытаться читать и записывать любые управляющие регистры RISC-V; пытаться напрямую обращаться к аппаратным устройствам; и передавать хитроумно сформированные значения системным вызовам, стремясь вынудить ядро аварийно завершиться или совершить нежелательное действие. Цель ядра — ограничить каждый пользовательский процесс так, чтобы тот мог лишь читать/записывать/выполнять собственную пользовательскую память, использовать 32 регистра общего назначения RISC-V и влиять на ядро и другие процессы только теми способами, которые предусмотрены системными вызовами. Ядро должно предотвращать любые другие действия. Как правило, это абсолютное требование в проектировании ядра.

Требования к собственному коду ядра существенно отличаются. Предполагается, что код ядра написан добросовестными и внимательными программистами. Ожидается, что код ядра не содержит ошибок и уж точно не содержит ничего вредоносного. Это предположение влияет на то, как мы анализируем код ядра. Например, существует множество внутренних функций ядра (например, спин-блокировки), которые могут вызвать серьёзные проблемы при неправильном использовании кодом ядра. При рассмотрении любого конкретного фрагмента кода ядра мы будем стремиться убедиться в

его корректности. Однако мы исходим из того, что код ядра в целом написан правильно и следует всем правилам использования собственных функций и структур данных ядра. На аппаратном уровне предполагается, что процессор RISC-V, ОЗУ, диск и т.д. работают так, как описано в документации, без аппаратных ошибок.

Конечно, в реальной жизни всё не так однозначно. Трудно помешать хитроумному пользовательскому коду сделать систему непригодной для использования (или вызвать панику), потребляя защищённые ядром ресурсы — дисковое пространство, процессорное время, слоты таблицы процессов и т.д. Обычно невозможно написать код без ошибок или создать аппаратуру без дефектов; если создатели вредоносного пользовательского кода знают об ошибках в ядре или аппаратуре, они воспользуются ими. Даже в зрелых, широко используемых ядрах, таких как Linux, новые уязвимости обнаруживаются постоянно [1]. Стоит встраивать защитные механизмы в ядро на случай наличия ошибок: утверждения, проверки типов, страницы-охранники стека и т.д. Наконец, граница между пользовательским и ядерным кодом порой размыта: некоторые привилегированные процессы пользовательского уровня могут предоставлять важные сервисы и фактически являться частью операционной системы, а в некоторых операционных системах привилегированный пользовательский код может добавлять новый код в ядро (как в случае загружаемых модулей ядра Linux).

2.8 Реальный мир

Большинство операционных систем приняли концепцию процесса, и большинство процессов похожи на процессы xv6. Современные операционные системы, однако, поддерживают несколько потоков внутри процесса, чтобы один процесс мог использовать несколько процессоров. Поддержка нескольких потоков в процессе требует значительного механизма, которого нет в xv6, включая возможные изменения интерфейса (например, Linux использует clone, вариант fork), для управления тем, какие аспекты процесса совместно используются потоками.

2.9 Упражнения

1. Добавьте в xv6 системный вызов, возвращающий количество свободной доступной памяти.

Глава 3

Таблицы страниц

Таблицы страниц — наиболее распространённый механизм, с помощью которого операционная система предоставляет каждому процессу собственное изолированное адресное пространство и память. Таблицы страниц определяют смысл адресов памяти и то, какие части физической памяти доступны. Они позволяют xv6 изолировать адресные пространства разных процессов и мультиплексировать их на единую физическую память. Таблицы страниц широко применяются в системном программировании, поскольку обеспечивают уровень косвенности, дающий операционным системам возможность использовать различные приёмы. Xv6 использует несколько таких приёмов: отображение одной и той же памяти (страница трамплина) в нескольких адресных пространствах, а также защита стеков ядра и пользователя с помощью незамапленных страниц-ограничителей. В остальной части главы описываются таблицы страниц, предоставляемые оборудованием RISC-V, и то, как xv6 их использует.

3.1 Аппаратная поддержка страничной адресации

Для справки напомним, что инструкции RISC-V (как пользовательские, так и ядра) работают с виртуальными адресами. Оперативная память машины, или физическая память, индексируется физическими адресами. Аппаратура таблиц страниц RISC-V связывает эти два вида адресов, отображая каждый виртуальный адрес на физический.

Xv6 работает на RISC-V в режиме Sv39, то есть используются только младшие 39 битов 64-битного виртуального адреса; старшие 25 битов не используются. В конфигурации Sv39 таблица страниц RISC-V логически представляет собой массив из 2^{27} (134 217 728) записей таблицы страниц (PTE). Каждая PTE содержит 44-битный номер физической страницы (PPN) и несколько флагов. Аппаратура страничной адресации транслирует виртуальный адрес, используя старшие 27 битов из 39 для поиска PTE в таблице страниц, и формирует 56-битный физический адрес, у которого старшие 44 бита берутся из PPN в PTE, а младшие 12 битов копируются из исходного виртуального адреса. На рисунке 3.1 показан этот процесс в виде логического представления таблицы страниц как простого массива PTE (более полная картина — на рисунке 3.2). Таблица

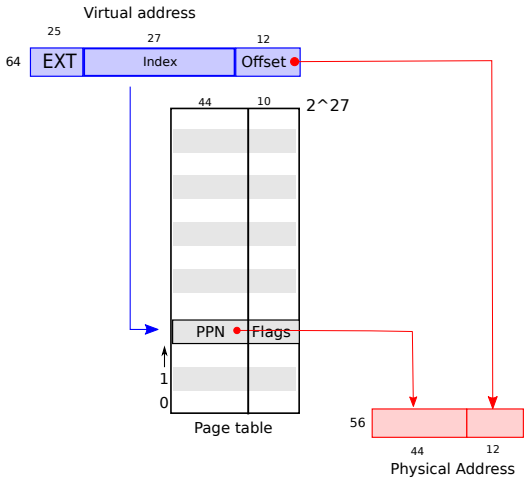

Рис. 3.1: Виртуальные и физические адреса RISC-V с упрощённой логической таблицей страниц.

страниц предоставляет операционной системе контроль над трансляцией виртуальных адресов в физические с гранулярностью выровненных блоков по 4096 (2^{12}) байт. Такой блок называется страницей.

В RISC-V Sv39 старшие 25 битов виртуального адреса не используются при трансляции. Физический адрес также имеет запас для роста: в формате PTE есть место для расширения номера физической страницы ещё на 10 битов. Разработчики RISC-V выбрали эти числа на основе технологических прогнозов. 2^{39} байт — это 512 ГБ, что должно быть достаточным адресным пространством для приложений, работающих на компьютерах RISC-V. 2^{56} — достаточный объём физической памяти на ближайшее будущее для размещения многочисленных устройств ввода-вывода и микросхем DRAM. Если потребуется больше, разработчики RISC-V определили Sv48 с 48-битными виртуальными адресами [3].

Как показано на рисунке 3.2, процессор RISC-V транслирует виртуальный адрес в физический за три шага. Таблица страниц хранится в физической памяти в виде трёхуровневого дерева. Корень дерева —это страница таблицы страниц размером 4096 байт, содержащая 512 PTE, которые хранят физические адреса страниц таблицы страниц следующего уровня дерева. Каждая из этих страниц содержит 512 PTE для финального уровня дерева. Аппаратура страничной адресации использует старшие 9 битов из 27 для выбора PTE на корневой странице таблицы страниц, средние 9 битов — для выбора PTE на странице таблицы страниц следующего уровня дерева, а младшие 9 битов —для выбора финальной PTE. (В RISC-V Sv48 таблица страниц имеет четыре уровня, а биты 39–47 виртуального адреса индексируют верхний уровень.)

Если любая из трёх PTE, необходимых для трансляции адреса, отсутствует, аппаратура страничной адресации генерирует исключение страничного отказа, предоставляя ядру возможность его обработать (см. главу 4).

Трёхуровневая структура рисунка 3.2 обеспечивает экономичный способ хранения

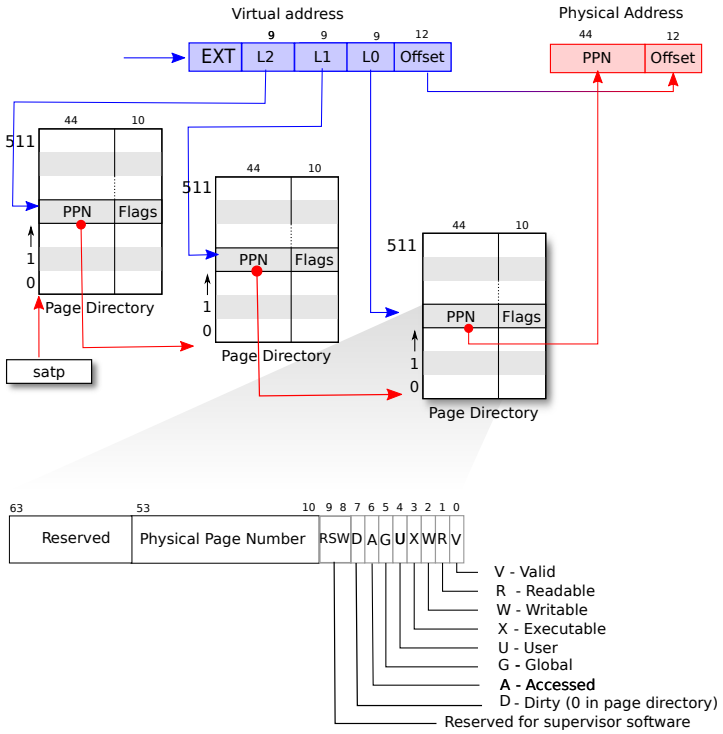

Рис. 3.2: Детали трансляции адресов RISC-V.

PTE по сравнению с одноуровневой схемой рисунка 3.1. В типичной ситуации, когда большие диапазоны виртуальных адресов не отображены, трёхуровневая структура позволяет полностью исключить целые каталоги страниц. Например, если приложение использует лишь несколько страниц, начиная с нулевого адреса, то записи с 1 по 511 каталога страниц верхнего уровня недействительны, и ядру не нужно выделять страницы для 511 промежуточных каталогов страниц. Более того, ядру также не нужно выделять страницы для нижних каталогов страниц, соответствующих этим 511 промежуточным каталогам. Таким образом, в данном примере трёхуровневая схема экономит 511 страниц для промежуточных каталогов и 511×512 страниц для нижних каталогов страниц.

Хотя процессор аппаратно обходит трёхуровневую структуру при выполнении инструкции загрузки или сохранения, потенциальный недостаток трёх уровней состоит в том, что процессору приходится загружать три PTE из памяти для трансляции виртуального адреса из инструкции загрузки/сохранения в физический адрес. Чтобы избежать затрат на загрузку PTE из физической памяти, процессор RISC-V кэширует записи таблицы страниц в буфере ассоциативной трансляции (TLB).

Каждая PTE содержит флаговые биты, сообщающие аппаратуре страничной адресации о допустимых способах использования соответствующего виртуального адреса. PTE_V указывает, присутствует ли PTE: если этот бит не установлен, обращение к странице вызывает исключение (т.е. не допускается). PTE_R управляет тем, могут ли

инструкции читать из страницы. PTE_W управляет тем, могут ли инструкции записывать на страницу. PTE_X управляет тем, может ли процессор интерпретировать содержимое страницы как инструкции и выполнять их. PTE_U управляет тем, могут ли инструкции в режиме пользователя обращаться к странице; если PTE_U не установлен, PTE может использоваться только в режиме супервизора. Рисунок 3.2 показывает, как всё это работает. Флаги и все остальные структуры, связанные с аппаратурой страниц, определены в (kernel/riscv.h)

Чтобы указать процессору использовать таблицу страниц, ядро должно записать физический адрес корневой страницы таблицы страниц в регистр satp. Процессор будет транслировать все адреса, генерируемые последующими инструкциями, используя таблицу страниц, на которую указывает его собственный satp. У каждого процессора есть свой satp, что позволяет разным процессорам выполнять разные процессы, каждый с собственным изолированным адресным пространством, описанным отдельной таблицей страниц.

Как правило, ядро отображает всю физическую память в свою таблицу страниц, чтобы иметь возможность читать и записывать данные в любое место физической памяти с помощью инструкций загрузки/сохранения. Поскольку каталоги страниц находятся в физической памяти, ядро может изменять содержимое PTE в каталоге страниц, записывая по виртуальному адресу этой PTE с помощью обычной инструкции сохранения.

Несколько замечаний о терминологии. Физическая память — это ячейки хранения в DRAM. Каждый байт физической памяти имеет адрес, называемый физическим адресом. Инструкции работают только с виртуальными адресами, которые аппаратура страничной адресации транслирует в физические адреса, а затем передаёт в аппаратуру DRAM для чтения или записи данных. В отличие от физической памяти и виртуальных адресов, виртуальная память — это не физический объект, а совокупность абстракций и механизмов, предоставляемых ядром для управления физической памятью и виртуальными адресами.

3.2 Адресное пространство ядра

Xv6 хранит по одной таблице страниц на каждый процесс, описывающей пользовательское адресное пространство этого процесса, а также единую таблицу страниц, описывающую адресное пространство ядра. Ядро конфигурирует своё адресное пространство таким образом, чтобы иметь доступ к физической памяти и различным аппаратным ресурсам по предсказуемым виртуальным адресам. Рисунок 3.3 показывает, как это отображение проецирует виртуальные адреса ядра на физические. Файл (kernel/memlayout.h) объявляет константы для разметки памяти ядра xv6.

QEMU эмулирует компьютер, включающий оперативную память (физическую память), начиная с физического адреса 0x80000000 и продолжающуюся как минимум до 0x88000000, который xv6 называет PHYSTOP. Эмулятор QEMU также включает устройства ввода-вывода, например дисковый интерфейс. QEMU предоставляет про-

граммному обеспечению интерфейсы устройств в виде отображённых в память регистров управления, расположенных ниже 0x80000000 в физическом адресном пространстве. Ядро взаимодействует с устройствами путём чтения/записи этих специальных физических адресов; такие операции передают данные аппаратуре устройства, а не оперативной памяти. В главе 4 объясняется, как xv6 взаимодействует с устройствами.

Ядро получает доступ к оперативной памяти и отображённым в память регистрам устройств через «прямое отображение», то есть отображая ресурсы по виртуальным адресам, равным физическим. Например, само ядро расположено по адресу KERNBASE=0x80000000 как в виртуальном адресном пространстве, так и в физической памяти. Прямое отображение упрощает код ядра, читающий или записывающий физическую память. Например, когда fork выделяет пользовательскую память для дочернего процесса, аллокатор возвращает физический адрес этой памяти; fork использует этот адрес непосредственно как виртуальный при копировании пользовательской памяти родителя в дочерний процесс.

Есть несколько виртуальных адресов ядра, которые не отображены напрямую:

- Страница трамплина. Она отображается в самом верху виртуального адресного пространства; пользовательские таблицы страниц имеют то же отображение. В главе 4 рассматривается роль страницы трамплина, но здесь мы видим интересный пример использования таблиц страниц: физическая страница (содержащая код трамплина) отображена дважды в виртуальном адресном пространстве ядра: один раз — в самом верху виртуального адресного пространства, и один раз — через прямое отображение.

- Страницы стека ядра. У каждого процесса есть собственный стек ядра, отображённый на высокие адреса, чтобы ниже него xv6 мог оставить незамапленную страницу-ограничитель. PTE страницы-ограничителя недействительна (т.е. PTE_V не установлен), так что при переполнении стека ядра, скорее всего, возникнет исключение и ядро завершится с паникой. Без страницы-ограничителя переполнение стека перезаписало бы другую память ядра, что привело бы к некорректной работе. Аварийное завершение с паникой предпочтительнее.

Хотя ядро обращается к стекам через отображение на высокие адреса, они также доступны ядру через прямой адрес. Альтернативная схема могла бы использовать только прямое отображение и работать со стеками по прямым адресам. Однако в таком случае создание страниц-ограничителей потребовало бы снятия отображений виртуальных адресов, которые иначе ссылались бы на физическую память, что затрудняет её использование.

Ядро отображает страницы трамплина и текст ядра с правами PTE_R и PTE_X. Ядро читает и выполняет инструкции с этих страниц. Остальные страницы ядро отображает с правами PTE_R и PTE_W, чтобы иметь возможность читать и записывать память на этих страницах. Отображения для страниц-ограничителей недействительны.

3.3 Код: создание адресного пространства

Большая часть кода xv6 для работы с адресными пространствами и таблицами страниц находится в vm.c (kernel/vm.c:1). Центральная структура данных — pagetable_t, которая по сути является указателем на корневую страницу таблицы страниц RISC-V; pagetable_t может быть как таблицей страниц ядра, так и одной из таблиц страниц конкретного процесса. Ключевые функции: walk, которая находит PTE для виртуального адреса, и mappages, устанавливающая PTE для новых отображений. Функции, начинающиеся с kvm, работают с таблицей страниц ядра; функции, начинающиеся с uvm, работают с таблицей страниц пользователя; остальные функции применяются для обеих целей. copyout и copyin копируют данные из/в виртуальные адреса пользователя, передаваемые как аргументы системных вызовов; они находятся в vm.c, потому что им нужно явно транслировать эти адреса для нахождения соответствующей физической памяти.

В начале последовательности загрузки main вызывает kvminit (kernel/vm.c:66) для создания таблицы страниц ядра с помощью kvmmake (kernel/vm.c:22). Этот вызов происходит до того, как xv6 включает страничную адресацию на RISC-V, поэтому адреса ссылаются непосредственно на физическую память. kvmmake сначала выделяет страницу физической памяти для хранения корневой страницы таблицы страниц. Затем она вызывает kvmmap для установки трансляций, необходимых ядру. Трансляции включают инструкции и данные ядра, физическую память вплоть до PHYSTOP, и диапазоны памяти, фактически являющиеся устройствами. proc_mapstacks (kernel/proc.c:33) выделяет стек ядра для каждого процесса. Она вызывает kvmmap для отображения каждого стека по виртуальному адресу, сгенерированному макросом KSTACK, оставляя место для недействительных страниц-ограничителей стека.

kvmmap (kernel/vm.c:58) вызывает mappages (kernel/vm.c:146), которая устанавливает отображения в таблице страниц для диапазона виртуальных адресов на соответствующий диапазон физических адресов. Это делается отдельно для каждого виртуального адреса в диапазоне с шагом в одну страницу. Для каждого отображаемого виртуального адреса mappages вызывает walk для нахождения адреса PTE этого адреса. Затем она инициализирует PTE, записывая соответствующий номер физической страницы, желаемые права доступа (PTE_W, PTE_X, и/или PTE_R), а также PTE_V для пометки PTE как действительной (kernel/vm.c:167).

walk (kernel/vm.c:98) имитирует работу аппаратуры страничной адресации RISC-V при поиске PTE для виртуального адреса (см. рисунок 3.2). walk обходит трёхуровневую таблицу страниц по 9 битов за раз. Она использует 9 битов виртуального адреса на каждом уровне для нахождения PTE либо таблицы страниц следующего уровня, либо конечной страницы (kernel/vm.c:104). Если PTE недействительна, значит, необходимая страница ещё не была выделена; если аргумент alloc установлен, walk выделяет новую страницу таблицы страниц и записывает её физический адрес в PTE. Функция возвращает адрес PTE на нижнем уровне дерева (kernel/vm.c:114).

Приведённый выше код опирается на то, что физическая память прямо отображена в виртуальное адресное пространство ядра. Например, когда walk обходит уровни таб-

лицы страниц, она берёт (физический) адрес таблицы страниц следующего уровня из PTE (kernel/vm.c:106), а затем использует этот адрес как виртуальный для чтения PTE следующего уровня (kernel/vm.c:104).

main вызывает kvminithart (kernel/vm.c:74) для установки таблицы страниц ядра. Она записывает физический адрес корневой страницы таблицы страниц в регистр satp. После этого процессор будет транслировать адреса, используя таблицу страниц ядра. Поскольку ядро использует отображение один-к-одному, виртуальный адрес следующей инструкции будет соответствовать правильному физическому адресу в памяти.

Каждый процессор RISC-V кэширует записи таблицы страниц в буфере ассоциативной трансляции (TLB), и когда xv6 изменяет таблицу страниц, она должна сообщить процессору об инвалидации соответствующих кэшированных записей TLB. Иначе TLB в какой-то момент мог бы использовать устаревшее кэшированное отображение, указывающее на физическую страницу, которая тем временем была выделена другому процессу, и в результате один процесс мог бы записывать данные в память другого процесса. В RISC-V есть инструкция sfence.vma, сбрасывающая TLB текущего процессора. Xv6 выполняет sfence.vma в kvminithart после перезагрузки регистра satp, а также в коде трамплина, который переключается на таблицу страниц пользователя перед возвратом в пространство пользователя (kernel/trampoline.S:89).

Также необходимо выполнять sfence.vma перед изменением satp, чтобы дождаться завершения всех ожидающих операций загрузки и сохранения. Это ожидание гарантирует, что предшествующие обновления таблицы страниц завершены, а предшествующие загрузки и сохранения используют старую таблицу страниц, а не новую.

Чтобы избежать полного сброса TLB, процессоры RISC-V могут поддерживать идентификаторы адресных пространств (ASID) [3]. Ядро тогда может сбрасывать только записи TLB для конкретного адресного пространства. Xv6 не использует эту возможность.

3.4 Выделение физической памяти

Ядро должно во время работы выделять и освобождать физическую память для таблиц страниц, пользовательской памяти, стеков ядра и буферов каналов.

Xv6 использует физическую память между концом ядра и PHYSTOP для динамического выделения. Оно выделяет и освобождает целые страницы по 4096 байт за раз. Отслеживание свободных страниц ведётся через связанный список, пронизывающий сами страницы. Выделение памяти состоит в удалении страницы из связанного списка; освобождение — в добавлении освобождённой страницы в список.

3.5 Код: аллокатор физической памяти

Аллокатор расположен в kalloc.c (kernel/kalloc.c:1). Структура данных аллокатора — список свободных страниц (free list) физической памяти, доступных для выделения.

Элемент списка каждой свободной страницы — это struct run (kernel/kalloc.c:17). Откуда аллокатор берёт память для хранения этой структуры данных? Он хранит структуру run каждой свободной страницы прямо в самой свободной странице, так как там ничего больше не хранится. Список свободных страниц защищён спин-блокировкой (kernel/kalloc.c:21-24). Список и блокировка обёрнуты в структуру, чтобы явно показать, что блокировка защищает поля в этой структуре. Пока что не обращайте внимания на блокировку и вызовы acquire и release; детальное рассмотрение блокировок — в главе 6.

Функция main вызывает kinit для инициализации аллокатора (kernel/kalloc.c:27). kinit инициализирует список свободных страниц, включая в него все страницы от конца ядра до PHYSTOP. Xv6 должна была бы определять объём доступной физической памяти, анализируя конфигурационную информацию, предоставляемую аппаратурой. Вместо этого xv6 предполагает, что машина имеет 128 мегабайт оперативной памяти. kinit вызывает freerange для добавления памяти в список свободных страниц через постраничные вызовы kfree. PTE может ссылаться только на физический адрес, выровненный по границе 4096 байт (кратный 4096), поэтому freerange использует PGROUNDUP для обеспечения того, чтобы освобождались только выровненные физические адреса. Изначально аллокатор не имеет памяти; эти вызовы kfree передают ему память для управления.

Аллокатор иногда трактует адреса как целые числа для выполнения арифметических операций над ними (например, при обходе всех страниц в freerange), а иногда использует адреса как указатели для чтения и записи памяти (например, при работе со структурой run, хранящейся в каждой странице); это двойное использование адресов — главная причина, по которой код аллокатора насыщен приведениями типов C. Другая причина — то, что освобождение и выделение памяти по своей природе изменяют тип памяти.

Функция kfree (kernel/kalloc.c:47) начинает с записи значения 1 в каждый байт освобождаемой памяти. Это приведёт к тому, что код, использующий память после её освобождения (так называемые «висячие ссылки»), будет читать мусор вместо прежнего корректного содержимого; можно надеяться, что это ускорит обнаружение ошибок в таком коде. Затем kfree добавляет страницу в начало списка свободных страниц: она приводит pa к указателю на struct run, записывает прежнее начало списка свободных страниц в r->next, и устанавливает список свободных страниц равным r. kalloc удаляет и возвращает первый элемент списка свободных страниц.

3.6 Адресное пространство процесса

У каждого процесса есть отдельная таблица страниц, и когда xv6 переключается между процессами, она также меняет таблицы страниц. На рисунке 3.4 адресное пространство процесса показано подробнее, чем на рисунке 2.3. Пользовательская память процесса начинается с виртуального адреса ноль и может расти до MAXVA (kernel/riscv.h:377), что теоретически позволяет процессу адресовать 256 гигабайт памяти.

Адресное пространство процесса состоит из страниц, содержащих текст программы (которые xv6 отображает с правами PTE_R, PTE_X и PTE_U), страниц с предварительно инициализированными данными программы, страницы стека и страниц кучи. Xv6 отображает данные, стек и кучу с правами PTE_R, PTE_W и PTE_U.

Использование прав доступа в пользовательском адресном пространстве — распространённый приём для повышения устойчивости пользовательских процессов. Если текст был бы отображён с PTE_W, то процесс мог бы случайно изменить свою собственную программу; например, ошибка программирования может привести к записи по нулевому указателю, изменению инструкций по адресу 0, а затем к продолжению выполнения, возможно с ещё более серьёзными последствиями. Чтобы немедленно выявлять такие ошибки, xv6 отображает текст без PTE_W; если программа случайно пытается записать по адресу 0, аппаратура откажет в выполнении записи и сгенерирует страничный отказ (см. раздел 4.6). Ядро затем завершит процесс и выведет информационное сообщение, которое поможет разработчику найти проблему.

Аналогично, отображая данные без PTE_X, пользовательская программа не сможет случайно перейти на адрес в области данных программы и начать выполнение с этого адреса.

На практике тщательная настройка прав доступа для защиты процессов также помогает в противостоянии атакам безопасности. Злоумышленник может передать специально сконструированные входные данные программе (например, веб-серверу), что приведёт к срабатыванию ошибки в программе в расчёте на то, чтобы превратить её в эксплойт [14]. Тщательная настройка прав доступа и другие методы, такие как рандомизация разметки пользовательского адресного пространства, делают такие атаки более сложными.

Стек занимает одну страницу и показан с начальным содержимым, созданным функцией exec. Строки, содержащие аргументы командной строки, а также массив указателей на них, находятся в самом верху стека. Чуть ниже расположены значения, позволяющие программе начать выполнение с main так, как если бы функция main(argc, argv) только что была вызвана.

Для обнаружения переполнения стека пользователя за выделенную память xv6 помещает недоступную страницу-ограничитель прямо под стеком, сбрасывая флаг PTE_U. Если стек пользователя переполняется и процесс пытается обратиться по адресу ниже стека, аппаратура сгенерирует исключение страничного отказа, поскольку страница-ограничитель недоступна для программы в режиме пользователя. Реальная операционная система могла бы вместо этого автоматически выделять дополнительную память для стека пользователя при его переполнении.

Когда процесс запрашивает у xv6 дополнительную пользовательскую память, xv6 увеличивает кучу процесса. Xv6 сначала использует kalloc для выделения физических страниц. Затем добавляет PTE в таблицу страниц процесса, указывающие на новые физические страницы. Xv6 устанавливает флаги PTE_W, PTE_R, PTE_U, и PTE_V в этих PTE. Большинство процессов не используют всё пользовательское адресное пространство; xv6 оставляет PTE_V сброшенным в неиспользуемых PTE.

Здесь можно увидеть несколько наглядных примеров применения таблиц страниц. Во-первых, таблицы страниц разных процессов транслируют пользовательские адреса в различные страницы физической памяти, так что каждый процесс имеет изолированную пользовательскую память. Во-вторых, каждый процесс видит свою память как непрерывную область виртуальных адресов, начиная с нуля, тогда как физическая память процесса может быть нефрагментированной. В-третьих, ядро отображает страницу с кодом трамплина в верхней части пользовательского адресного пространства (без PTE_U), так что одна физическая страница присутствует во всех адресных пространствах, но может использоваться только ядром.

3.7 Код: sbrk

sbrk — это системный вызов для уменьшения или увеличения памяти процесса. Системный вызов реализован функцией growproc (kernel/proc.c:237). growproc вызывает uvmalloc или uvmdealloc, в зависимости от того, является ли n положительным или отрицательным. uvmalloc (kernel/vm.c:217) выделяет физическую память с помощью kalloc и добавляет PTE в таблицу страниц пользователя с помощью mappages. uvmdealloc вызывает uvmunmap (kernel/vm.c:193), который использует walk для нахождения PTE и kfree для освобождения физической памяти, на которую они ссылаются.

Xv6 использует таблицу страниц процесса не только для указания аппаратуре, как отображать пользовательские виртуальные адреса, но и как единственный журнал о том, какие страницы физической памяти выделены данному процессу. Именно поэтому освобождение пользовательской памяти (в uvmunmap) требует просмотра таблицы страниц пользователя.

3.8 Код: exec

exec —системный вызов, заменяющий пользовательское адресное пространство процесса данными, считанными из файла, называемого двоичным или исполняемым файлом. Двоичный файл обычно является результатом работы компилятора и компоновщика и содержит машинные инструкции и данные программы. указанный двоичный файл path с помощью namei (kernel/exec.c:41), что объясняется в главе 8. Затем читается заголовок ELF. Двоичные файлы xv6 отформатированы в широко распространённом формате ELF, определённом в (kernel/elf.h). Двоичный файл ELF состоит из заголовка ELF, struct elfhdr (kernel/elf.h:6), за которым следует последовательность заголовков программных секций, struct proghdr (kernel/elf.h:25). Каждый progvhdr описывает секцию приложения, которую необходимо загрузить в память; программы xv6 имеют два заголовка программных секций: один для инструкций и один для данных.

Первый шаг —быстрая проверка того, что файл, вероятно, содержит двоичный файл ELF. Двоичный файл ELF начинается с четырёхбайтового «магического числа» 0x7F, ‘E’, ‘L’, ‘F’, или ELF_MAGIC (kernel/elf.h:3). Если заголовок ELF содержит правильное

магическое число, exec считает, что двоичный файл корректен.

exec выделяет новую таблицу страниц без пользовательских отображений с помощью proc_pagetable (kernel/exec.c:55), выделяет память для каждого сегмента ELF с помощью uvmalloc (kernel/exec.c:71), и загружает каждый сегмент в память с помощью loadseg (kernel/exec.c:10). loadseg использует walkaddr для нахождения физического адреса выделенной памяти, по которому следует записывать каждую страницу сегмента ELF, и readi для чтения из файла.

Заголовок программной секции для /init, первой пользовательской программы, созданной с помощью exec, выглядит следующим образом:

```
# objdump -p user/_init

user/_init:     file format elf64-little

Program Header:
0x70000003 off    0x0000000000006bb0 vaddr 0x0000000000000000
                                     paddr 0x0000000000000000 align 2**0
         filesz 0x000000000000004a memsz 0x0000000000000000 flags r--
    LOAD off    0x0000000000001000 vaddr 0x0000000000000000
                                     paddr 0x0000000000000000 align 2**12
         filesz 0x0000000000001000 memsz 0x0000000000001000 flags r-x
    LOAD off    0x0000000000002000 vaddr 0x0000000000001000
                                     paddr 0x0000000000001000 align 2**12
         filesz 0x0000000000000010 memsz 0x0000000000000030 flags rw-
   STACK off    0x0000000000000000 vaddr 0x0000000000000000
                                     paddr 0x0000000000000000 align 2**4
         filesz 0x0000000000000000 memsz 0x0000000000000000 flags rw-
```

Мы видим, что текстовый сегмент должен быть загружен по виртуальному адресу 0x0 в памяти (без прав записи) из содержимого со смещением 0x1000 в файле. Также видно, что данные должны быть загружены по адресу 0x1000, который находится на границе страницы, и без прав выполнения.

Поле filesz заголовка программной секции может быть меньше memsz, что означает, что разрыв между ними должен быть заполнен нулями (для глобальных переменных C), а не читаться из файла. Для /init значение filesz данных составляет 0x10 байт, а memsz равен 0x30 байтам, и поэтому uvmalloc выделяет достаточно физической памяти для хранения 0x30 байт, но считывает только 0x10 байт из файла /init.

Теперь exec выделяет и инициализирует стек пользователя. Выделяется только одна страница стека. exec копирует строки аргументов на вершину стека по одной, записывая указатели на них в ustack. В конец будущего списка argv, передаваемого в main, помещается нулевой указатель. Первые три записи в ustack — это фиктивный адрес возврата, argc, и указатель argv.

exec помещает недоступную страницу прямо под страницей стека, чтобы программы, пытающиеся использовать более одной страницы, получали ошибку. Эта недоступная страница также позволяет exec обрабатывать слишком большие аргументы; в такой ситуации функция copyout (kernel/vm.c:343), которую использует exec для копирования аргументов в стек, обнаружит, что целевая страница недоступна, и вернёт -1.

Во время подготовки нового образа памяти, если exec обнаруживает ошибку, например неверный программный сегмент, оно переходит на метку bad, освобождает новый образ и возвращает -1. exec должна дождаться освобождения старого образа, пока не убедится, что системный вызов завершится успешно: если старый образ уже удалён, системный вызов не может вернуть -1 в него. Единственные случаи ошибок в exec происходят во время создания образа. Как только образ готов, exec может перейти на новую таблицу страниц (kernel/exec.c:132) и освободить старую (kernel/exec.c:136).

exec загружает байты из файла ELF в память по адресам, указанным в файле ELF. Пользователи или процессы могут помещать в файл ELF любые адреса. Поэтому exec сопряжён с риском, поскольку адреса в файле ELF могут ссылаться на ядро — намеренно или случайно. Последствия для неосторожного ядра могут варьироваться от аварийного завершения до злонамеренного подрыва механизмов изоляции ядра (т.е. эксплойта безопасности). Xv6 выполняет ряд проверок для снижения этих рисков. Например, if (ph.vaddr + ph.memsz < ph.vaddr) проверяет, не происходит ли переполнение суммы при сложении 64-битных целых чисел. Опасность состоит в том, что пользователь мог бы создать ELF-файл с ph.vaddr, указывающим на выбранный пользователем адрес, и ph.memsz, достаточно большим, чтобы при переполнении сумма дала 0x1000, что выглядит как допустимое значение. В более ранней версии xv6, где пользовательское адресное пространство также содержало ядро (но недоступное для чтения/записи в режиме пользователя), пользователь мог бы выбрать адрес, соответствующий памяти ядра, и таким образом скопировать данные из ELF-файла в ядро. В версии xv6 для RISC-V это невозможно, поскольку ядро имеет собственную отдельную таблицу страниц; loadseg загружает данные в таблицу страниц процесса, а не в таблицу страниц ядра.

Разработчику ядра легко упустить важную проверку, и реальные ядра имеют долгую историю пропущенных проверок, отсутствие которых может быть использовано пользовательскими программами для получения привилегий ядра. Вполне вероятно, что xv6 не выполняет полную валидацию пользовательских данных, передаваемых в ядро, что вредоносная пользовательская программа могла бы использовать для обхода механизмов изоляции xv6.

3.9 Реальный мир

Как и большинство операционных систем, xv6 использует аппаратуру страничной адресации для защиты памяти и отображения. Большинство операционных систем применяют страничную адресацию значительно более изощрённо, чем xv6, сочетая страничную адресацию и исключения страничных отказов, что мы обсудим в главе 4.

Xv6 упрощена за счёт прямого отображения между виртуальными и физическими адресами в ядре, а также за счёт предположения о наличии физической оперативной памяти по адресу 0x8000000, куда ядро ожидает быть загруженным. Это работает с QEMU, но на реальном оборудовании оказывается неудачным решением; реальное оборудование размещает оперативную память и устройства по непредсказуемым физиче-

ским адресам, так что (например) по адресу 0x8000000 может не оказаться оперативной памяти, где xv6 рассчитывает хранить ядро. В более серьёзных разработках ядра таблица страниц используется для преобразования произвольной физической разметки аппаратной памяти в предсказуемую разметку виртуальных адресов ядра.

RISC-V поддерживает защиту на уровне физических адресов, но xv6 не использует эту возможность.

На машинах с большим объёмом памяти имеет смысл использовать поддержку RISC-V «суперстраниц». Маленькие страницы имеют смысл при небольшом объёме физической памяти, чтобы обеспечить точное выделение и выгрузку на диск. Например, если программа использует лишь 8 килобайт памяти, выделять ей целую суперстраницу в 4 мегабайта физической памяти расточительно. Большие страницы целесообразны на машинах с большим объёмом оперативной памяти и могут снизить накладные расходы на работу с таблицами страниц.

Отсутствие в ядре xv6 аллокатора типа malloc, способного предоставлять память для небольших объектов, не позволяет ядру использовать сложные структуры данных, требующие динамического выделения. Более развитое ядро, скорее всего, выделяло бы блоки многих различных малых размеров, а не (как в xv6) только блоки по 4096 байт; настоящий аллокатор ядра должен обрабатывать как малые, так и большие выделения.

Выделение памяти — традиционно горячая тема, а основные проблемы связаны с эффективным использованием ограниченной памяти и подготовкой к неизвестным будущим запросам [9]. Сегодня скорость важнее пространственной эффективности.

3.10 Упражнения

1. Разберите дерево устройств RISC-V, чтобы определить объём физической памяти компьютера.

2. Напишите пользовательскую программу, увеличивающую своё адресное пространство на один байт с помощью вызова sbrk(1). Запустите программу и изучите таблицу страниц до вызова sbrk и после вызова sbrk. Сколько памяти выделило ядро? Что содержит PTE для новой памяти?

3. Модифицируйте xv6 для использования суперстраниц в ядре.

4. Реализации exec в Unix традиционно включают специальную обработку shell-скриптов. Если исполняемый файл начинается с текста #!, то первая строка считается программой для интерпретации файла. Например, если exec вызывается для запуска myprog arg1 и первая строка myprog — #!/interp, то exec запускает /interp с командной строкой /interp myprog arg1. Реализуйте поддержку этого соглашения в xv6.

5. Реализуйте рандомизацию разметки адресного пространства ядра.

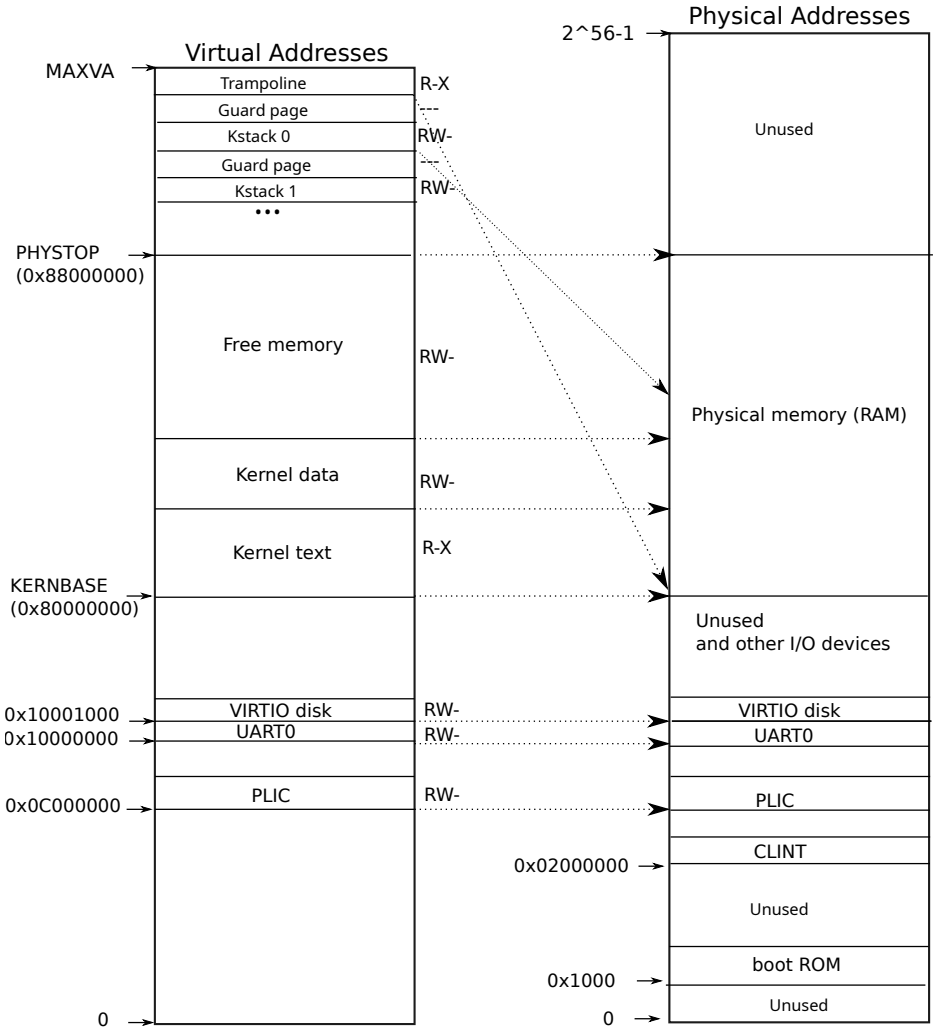

Рис. 3.3: Слева — адресное пространство ядра xv6. RWX обозначают права доступа PTE: чтение, запись и выполнение. Справа — физическое адресное пространство RISC-V, которое ожидает увидеть xv6.

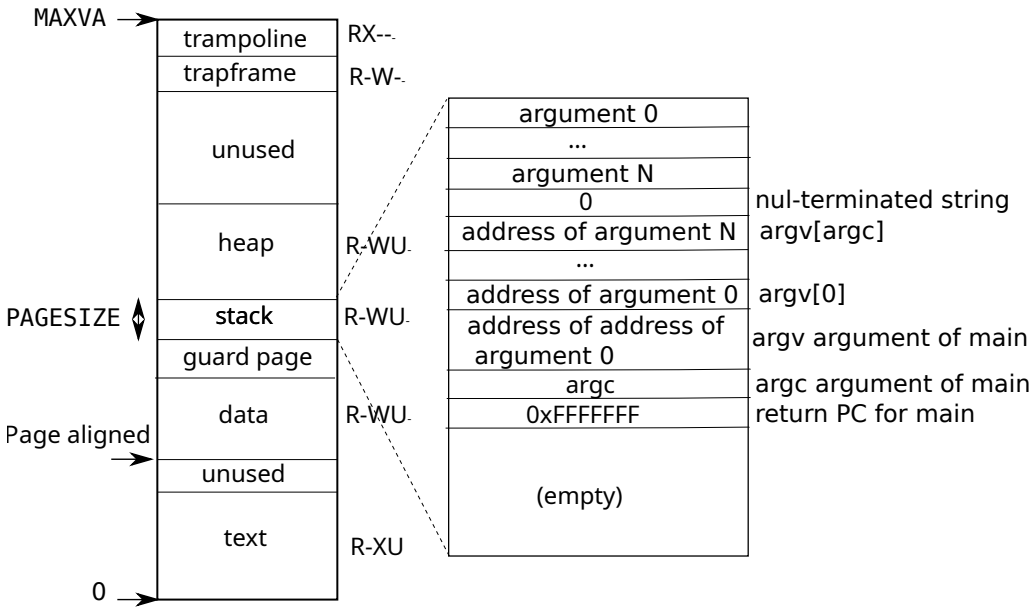

Рис. 3.4: Пользовательское адресное пространство процесса с его начальным стеком.

Глава 4

Ловушки и системные вызовы

Существуют три вида событий, при которых процессор прерывает обычное выполнение инструкций и принудительно передаёт управление специальному коду, обрабатывающему данное событие. Первая ситуация — системный вызов, когда пользовательская программа выполняет инструкцию ecall, чтобы попросить ядро выполнить какое-либо действие. Вторая ситуация — исключение: инструкция (пользовательская или ядерная) совершает что-то недопустимое, например делит на ноль или обращается по недействительному виртуальному адресу. Третья ситуация — прерывание устройства, когда устройство сигнализирует о необходимости обслуживания, например когда дисковое оборудование завершает запрос на чтение или запись.

В данной книге для обозначения всех этих ситуаций используется общий термин ловушка. Как правило, код, выполнявшийся в момент ловушки, должен впоследствии возобновить работу и не обязан знать, что произошло что-то особенное. То есть зачастую желательно, чтобы ловушки были прозрачными; это особенно важно для прерываний устройств, которых прерванный код обычно не ожидает. Обычная последовательность событий такова: ловушка принудительно передаёт управление в ядро; ядро сохраняет регистры и другое состояние, чтобы можно было возобновить выполнение; ядро выполняет соответствующий код обработчика (например, реализацию системного вызова или драйвер устройства); ядро восстанавливает сохранённое состояние и возвращается из ловушки; и исходный код возобновляется с прерванного места.

Xv6 обрабатывает все ловушки в ядре; ловушки не доставляются пользовательскому коду. Обработка ловушек в ядре естественна для системных вызовов. Это разумно для прерываний, поскольку изоляция требует, чтобы только ядро имело право использовать устройства, а также потому, что ядро является удобным механизмом для совместного использования устройств несколькими процессами. Это также разумно для исключений, поскольку xv6 реагирует на все исключения из пространства пользователя, уничтожая виновный процесс.

Обработка ловушек в xv6 проходит четыре этапа: аппаратные действия, выполняемые процессором RISC-V; несколько ассемблерных инструкций, подготавливающих почву для кода ядра на C; функция на C, определяющая, что делать с ловушкой; и сервисная подпрограмма системного вызова или драйвера устройства. Несмотря на то

что общность трёх типов ловушек предполагает возможность обработки всех ловушек единым путём, на практике удобнее иметь отдельный код для трёх различных случаев: ловушки из пространства пользователя, ловушки из пространства ядра и прерывания таймера. Код ядра (ассемблерный или на C), обрабатывающий ловушку, часто называют обработчиком; первые инструкции обработчика обычно пишутся на ассемблере (а не на C) и иногда называются вектором.

4.1 Механизм ловушек RISC-V

Каждый процессор RISC-V имеет набор управляющих регистров, в которые ядро записывает инструкции для процессора о том, как обрабатывать ловушки, и из которых ядро может считать информацию о произошедшей ловушке. Полное описание содержится в документации RISC-V [3]. riscv.h (kernel/riscv.h:1) содержит определения, используемые xv6. Ниже приведён обзор наиболее важных регистров:

- stvec: Ядро записывает сюда адрес своего обработчика ловушек; RISC-V переходит по адресу в stvec для обработки ловушки.
- sepc: При возникновении ловушки RISC-V сохраняет здесь счётчик команд (поскольку pc затем перезаписывается значением из stvec). Инструкция sret (возврат из ловушки) копирует sepc в pc. Ядро может записывать в sepc, управляя тем, куда перейдёт sret.
- scause: RISC-V записывает сюда число, описывающее причину ловушки.
- sscratch: Код обработчика ловушек использует sscratch, чтобы не затереть пользовательские регистры до их сохранения.
- sstatus: Бит SIE в sstatus управляет тем, включены ли прерывания устройств. Если ядро сбросит SIE, RISC-V будет откладывать прерывания устройств до тех пор, пока ядро не установит SIE. Бит SPP указывает, из пользовательского или из режима супервизора пришла ловушка, и определяет, в какой режим вернётся sret.

Перечисленные выше регистры относятся к ловушкам, обрабатываемым в режиме супервизора, и не могут быть прочитаны или записаны в пользовательском режиме. Существует аналогичный набор управляющих регистров для ловушек, обрабатываемых в машинном режиме; xv6 использует их только для особого случая прерываний таймера.

Каждый процессор многоядерного чипа имеет собственный набор этих регистров, и в любой момент времени более одного процессора может обрабатывать ловушку.

Когда необходимо инициировать ловушку, аппаратура RISC-V выполняет следующее для всех типов ловушек (кроме прерываний таймера):

1. Если ловушка является прерыванием устройства, а бит SIE в sstatus сброшен, не выполнять ни одного из следующих шагов.

2. Отключить прерывания, сбросив бит SIE в sstatus.
3. Скопировать pc в sepc.
4. Сохранить текущий режим (пользовательский или режим супервизора) в бите SPP в sstatus.
5. Установить scause в соответствии с причиной ловушки.
6. Установить режим супервизора.
7. Скопировать stvec в pc.
8. Начать выполнение по новому pc.

Обратите внимание, что процессор не переключает таблицу страниц на ядерную, не переключается на стек в ядре и не сохраняет никаких регистров, кроме pc. Эти задачи должно выполнить программное обеспечение ядра. Одна из причин, по которым процессор выполняет минимальный объём работы при ловушке, — обеспечение гибкости для программного обеспечения; например, некоторые операционные системы пропускают переключение таблицы страниц в ряде случаев, чтобы повысить производительность обработки ловушек.

Стоит подумать, можно ли опустить какие-либо из перечисленных выше шагов, возможно, в поисках более быстрых ловушек. Хотя существуют ситуации, где более простая последовательность может работать, многие из шагов было бы опасно опускать в общем случае. Например, предположим, что процессор не переключает счётчик команд. Тогда ловушка из пространства пользователя могла бы переключить его в режим супервизора, продолжая выполнять пользовательские инструкции. Эти пользовательские инструкции могли бы нарушить изоляцию пользователя/ядра, например изменив регистр satp так, чтобы он указывал на таблицу страниц, предоставляющую доступ ко всей физической памяти. Поэтому важно, чтобы процессор переходил по адресу инструкции, заданному ядром, то есть stvec.

4.2 Ловушки из пространства пользователя

Xv6 обрабатывает ловушки по-разному в зависимости от того, произошла ли ловушка во время выполнения в ядре или в пользовательском коде. Здесь описываются ловушки из пользовательского кода; раздел 4.5 описывает ловушки из кода ядра.

Ловушка может возникнуть при выполнении в пространстве пользователя, если пользовательская программа выполняет системный вызов (инструкция ecall), совершает что-то недопустимое или если поступает прерывание устройства. Высокоуровневый путь ловушки из пространства пользователя: uservec (kernel/trampoline.S:22), затем usertrap (kernel/trap.c:38); а при возврате — usertrapret и затем userret (kernel/trampoline.S:101).

Существенным ограничением при проектировании обработки ловушек в xv6 является тот факт, что аппаратура RISC-V не переключает таблицы страниц при инициировании ловушки. Это означает, что адрес обработчика ловушек в stvec должен иметь действительное отображение в пользовательской таблице страниц, поскольку именно она активна в момент начала выполнения кода обработки ловушек. Кроме того, код обработки ловушек xv6 должен переключиться на ядерную таблицу страниц; чтобы суметь продолжить выполнение после этого переключения, ядерная таблица страниц также должна содержать отображение для обработчика, на который указывает stvec.

Xv6 удовлетворяет этим требованиям, используя страницу трамплина. Страница трамплина содержит uservec — код обработки ловушек xv6, на который указывает stvec. Страница трамплина отображена в таблицах страниц каждого процесса по адресу TRAMPOLINE, который находится в верхней части виртуального адресного пространства, выше памяти, используемой программами. Страница трамплина также отображена по адресу TRAMPOLINE в ядерной таблице страниц. Смотрите Figure 2.3 и Figure 3.3. Поскольку страница трамплина отображена в пользовательской таблице страниц без флага PTE_U, ловушки могут начать выполняться там в режиме супервизора. Поскольку страница трамплина отображена по одному и тому же адресу в ядерном адресном пространстве, обработчик ловушек может продолжить выполнение после переключения на ядерную таблицу страниц.

Код обработчика ловушек uservec находится в trampoline.S (kernel/trampoline.S:22). Когда uservec начинает работу, все 32 регистра содержат значения, принадлежащие прерванному пользовательскому коду. Эти 32 значения необходимо сохранить где-то в памяти, чтобы их можно было восстановить при возврате из ловушки в пространство пользователя. Для записи в память нужен регистр, содержащий адрес, но в данный момент нет доступных регистров общего назначения! К счастью, RISC-V приходит на помощь в виде регистра sscratch. Инструкция csrw в начале uservec сохраняет a0 в sscratch. Теперь uservec имеет один регистр (a0) для работы.

Следующая задача uservec —сохранить 32 пользовательских регистра. Для каждого процесса ядро выделяет страницу памяти для структуры trapframe, в которой (помимо прочего) есть место для сохранения 32 пользовательских регистров (kernel/proc.h:43). Поскольку satp по-прежнему указывает на пользовательскую таблицу страниц, uservec требует, чтобы фрейм ловушки был отображён в пользовательском адресном пространстве. Xv6 отображает фрейм ловушки каждого процесса по виртуальному адресу TRAPFRAME в пользовательской таблице страниц этого процесса; TRAPFRAME расположен сразу ниже TRAMPOLINE. Указатель p->trapframe процесса также указывает на фрейм ловушки, но по его физическому адресу, чтобы ядро могло использовать его через ядерную таблицу страниц.

Таким образом uservec загружает адрес TRAPFRAME в a0 и сохраняет туда все пользовательские регистры, включая пользовательский a0, считанный обратно из sscratch.

trapframe содержит адрес стека ядра текущего процесса, идентификатор hartid текущего процессора, адрес функции usertrap и адрес ядерной таблицы страниц. uservec

извлекает эти значения, переключает satp на ядерную таблицу страниц и вызывает usertrap.

Задача usertrap — определить причину ловушки, обработать её и вернуться (kernel/trap.c:38). Сначала она изменяет stvec так, чтобы ловушка, возникающая в ядре, обрабатывалась kernelvec, а не uservec. Она сохраняет регистр sepc (сохранённый счётчик команд пользователя), поскольку usertrap может вызвать yield, чтобы переключиться на поток ядра другого процесса, а тот процесс может вернуться в пространство пользователя, изменив при этом sepc. Если ловушка является системным вызовом, usertrap вызывает syscall для его обработки; если прерыванием устройства — devintr; в остальных случаях это исключение, и ядро уничтожает аварийный процесс. Путь системного вызова добавляет четыре к сохранённому счётчику команд пользователя, поскольку RISC-V в случае системного вызова оставляет указатель команд на инструкции ecall, тогда как пользовательскому коду нужно возобновить выполнение со следующей инструкции. На выходе usertrap проверяет, был ли процесс уничтожен или должен уступить процессор (если данная ловушка является прерыванием таймера).

Первый шаг при возврате в пространство пользователя — вызов usertrapret Эта функция настраивает управляющие регистры RISC-V в подготовку к будущей ловушке из пространства пользователя. Это включает изменение stvec для указания на uservec, подготовку полей фрейма ловушки, на которые опирается uservec, и установку sepc в ранее сохранённый счётчик команд пользователя. В конце usertrapret вызывает userret на странице трамплина, отображённой как в пользовательской, так и в ядерной таблицах страниц; это необходимо потому, что ассемблерный код в userret будет переключать таблицы страниц.

Вызов userret из usertrapret передаёт указатель на пользовательскую таблицу страниц процесса в a0 (kernel/trampoline.S:101). userret переключает satp на пользовательскую таблицу страниц процесса. Вспомним, что пользовательская таблица страниц отображает как страницу трамплина, так и TRAPFRAME, но ничего другого из ядра. Одинаковое отображение страницы трамплина по одному и тому же виртуальному адресу в пользовательской и ядерной таблицах страниц позволяет userret продолжать выполнение после изменения satp. После этого момента userret может использовать только содержимое регистров и содержимое фрейма ловушки. userret загружает адрес TRAPFRAME в a0, восстанавливает сохранённые пользовательские регистры из фрейма ловушки через a0, восстанавливает сохранённый пользовательский a0 и выполняет sret для возврата в пространство пользователя.

4.3 Код: вызов системных вызовов

Chapter 2 завершился тем, что initcode.S вызывает системный вызов exec Рассмотрим, как пользовательский вызов достигает реализации системного вызова exec в ядре.

initcode.S помещает аргументы для exec в регистры a0 и a1, а номер системного вызова — в a7. Номера системных вызовов соответствуют записям в массиве syscalls — таблице указателей на функции (kernel/syscall.c:107). Инструкция ecall производит

ловушку в ядро и вызывает выполнение uservec, usertrap и затем syscall, как описано выше.

syscall (kernel/syscall.c:132) извлекает номер системного вызова из сохранённого a7 во фрейме ловушки и использует его как индекс в syscalls. Для первого системного вызова a7 содержит SYS_exec (kernel/syscall.h:8), что приводит к вызову функции реализации системного вызова sys_exec.

Когда sys_exec возвращается, syscall записывает возвращаемое значение в p->trapframe->a0. Это приведёт к тому, что исходный вызов exec() в пространстве пользователя вернёт это значение, поскольку соглашение о вызовах C на RISC-V помещает возвращаемые значения в a0. Системные вызовы по соглашению возвращают отрицательные числа для индикации ошибок и ноль или положительные числа при успехе. Если номер системного вызова недопустим, syscall выводит сообщение об ошибке и возвращает -1.

4.4 Код: аргументы системных вызовов

Реализации системных вызовов в ядре должны находить аргументы, переданные пользовательским кодом. Поскольку пользовательский код вызывает функции-обёртки системных вызовов, аргументы изначально находятся там, куда их помещает соглашение о вызовах RISC-V C: в регистрах. Код ядра для обработки ловушек сохраняет пользовательские регистры во фрейм ловушки текущего процесса, откуда код ядра может их получить. Функции ядра argint, argaddr и argfd извлекают n-й аргумент системного вызова из фрейма ловушки как целое число, указатель или файловый дескриптор соответственно. Все они вызывают argraw для получения соответствующего сохранённого пользовательского регистра (kernel/syscall.c:34).

Некоторые системные вызовы передают указатели в качестве аргументов, и ядро должно использовать эти указатели для чтения или записи пользовательской памяти. Системный вызов exec, например, передаёт ядру массив указателей на строковые аргументы в пространстве пользователя. Эти указатели создают две проблемы. Во-первых, пользовательская программа может содержать ошибки или быть вредоносной и передать ядру недействительный указатель или указатель, намеренно направляющий ядро к обращению к ядерной памяти вместо пользовательской. Во-вторых, отображения ядерной таблицы страниц xv6 не совпадают с отображениями пользовательской таблицы страниц, поэтому ядро не может использовать обычные инструкции загрузки или записи по адресам, предоставленным пользователем.

Ядро реализует функции, безопасно передающие данные по адресам, предоставленным пользователем. fetchstr является примером (kernel/syscall.c:25). Системные вызовы файловой системы, такие как exec, используют fetchstr для получения строковых аргументов с именами файлов из пространства пользователя. fetchstr вызывает copyinstr для выполнения основной работы.

copyinstr (kernel/vm.c:410) копирует до max байт в dst из виртуального адреса srcva в пользовательской таблице страниц pagetable. Поскольку pagetable не является текущей

таблицей страниц, copyinstr использует walkaddr (который вызывает walk) для поиска srcva в pagetable, получая физический адрес pa0. Ядро отображает каждый адрес физической ОЗУ в соответствующий виртуальный адрес ядра, поэтому copyinstr может непосредственно скопировать байты строки из pa0 в dst. walkaddr (kernel/vm.c:121) проверяет, принадлежит ли предоставленный пользователем виртуальный адрес пользовательскому адресному пространству процесса, чтобы программы не могли обмануть ядро и заставить его читать чужую память. Аналогичная функция copyout копирует данные из ядра по адресу, предоставленному пользователем.

4.5 Ловушки из пространства ядра

Xv6 несколько по-разному настраивает регистры ловушек процессора в зависимости от того, выполняется ли пользовательский или ядерный код. Когда ядро выполняется на процессоре, ядро направляет stvec на ассемблерный код по адресу kernelvec (kernel/kernelvec.S:12). Поскольку xv6 уже находится в ядре, kernelvec может полагаться на то, что satp указывает на ядерную таблицу страниц, а указатель стека ссылается на действительный стек ядра. kernelvec сохраняет все 32 регистра в стек, откуда позднее восстанавливает их, чтобы прерванный код ядра мог возобновить работу без помех.

kernelvec сохраняет регистры в стек прерванного потока ядра, что логично, поскольку значения регистров принадлежат этому потоку. Это особенно важно, если ловушка вызывает переключение на другой поток — в таком случае ловушка фактически вернётся из стека нового потока, оставив сохранённые регистры прерванного потока в безопасности на его стеке.

kernelvec переходит к kerneltrap (kernel/trap.c:136) после сохранения регистров. kerneltrap подготовлен для двух типов ловушек: прерываний устройств и исключений. Он вызывает devintr (kernel/trap.c:186) для проверки и обработки первых. Если ловушка не является прерыванием устройства, значит, это исключение, которое в ядре xv6 всегда является фатальной ошибкой; ядро вызывает panic и прекращает выполнение.

Если kerneltrap был вызван прерыванием таймера и при этом выполняется поток ядра процесса (в отличие от потока планировщика), kerneltrap вызывает yield, чтобы дать другим потокам возможность поработать. В какой-то момент один из этих потоков уступит управление, позволив нашему потоку и его kerneltrap возобновить работу. Chapter 7 объясняет, что происходит в yield.

Когда работа kerneltrap завершена, ему нужно вернуться в код, прерванный ловушкой. Поскольку yield мог нарушить sepc и предыдущий режим в sstatus, kerneltrap сохраняет их при старте. Теперь он восстанавливает эти управляющие регистры и возвращается в kernelvec (kernel/kernelvec.S:38). kernelvec извлекает сохранённые регистры из стека и выполняет sret, который копирует sepc в pc и возобновляет прерванный код ядра.

Стоит обдумать, как происходит возврат из ловушки, если kerneltrap вызвал yield из-за прерывания таймера.

Xv6 устанавливает stvec процессора в kernelvec, когда этот процессор входит в ядро

из пространства пользователя; это можно видеть в usertrap (kernel/trap.c:29). Существует промежуток времени, когда ядро уже начало выполняться, но stvec ещё указывает на uservec, и принципиально важно, чтобы в течение этого промежутка не возникло ни одного прерывания устройства. К счастью, RISC-V всегда отключает прерывания при начале обработки ловушки, а xv6 не включает их снова до тех пор, пока не установит stvec.

4.6 Исключения при обращении к страницам

Реакция xv6 на исключения весьма незатейлива: если исключение происходит в пространстве пользователя, ядро уничтожает виновный процесс. Если исключение происходит в ядре, ядро вызывает панику. Реальные операционные системы нередко реагируют значительно интереснее.

В качестве примера: многие ядра используют ошибки страниц для реализации форка с копированием при записи (COW). Чтобы объяснить форк с копированием при записи, рассмотрим fork в xv6, описанный в Chapter 3. fork приводит к тому, что начальное содержимое памяти дочернего процесса совпадает с памятью родительского на момент форка. Xv6 реализует форк с помощью uvmcopy (kernel/vm.c:297), которая выделяет физическую память для дочернего процесса и копирует туда память родителя. Было бы эффективнее, если бы дочерний и родительский процессы могли совместно использовать физическую память родителя. Однако прямолинейная реализация этого не работает, поскольку тогда родительский и дочерний процессы мешали бы выполнению друг друга своими записями в совместный стек и кучу.

Родительский и дочерний процессы могут безопасно совместно использовать физическую память путём надлежащего применения разрешений таблицы страниц и ошибок страниц. Процессор генерирует исключение при обращении к странице, когда используется виртуальный адрес, не имеющий отображения в таблице страниц, или имеющий отображение с очищенным флагом PTE_V, или отображение, биты разрешений которого (PTE_R, PTE_W, PTE_X, PTE_U) запрещают выполняемую операцию. RISC-V различает три вида ошибок страниц: ошибки страниц при загрузке (когда инструкция загрузки не может транслировать свой виртуальный адрес), ошибки страниц при записи (когда инструкция записи не может транслировать свой виртуальный адрес) и ошибки страниц инструкций (когда адрес в счётчике команд не транслируется). Регистр scause указывает тип ошибки страницы, а регистр stval содержит адрес, который не удалось транслировать.

Основная идея COW-форка состоит в том, что родитель и дочерний процесс изначально совместно используют все физические страницы, но каждый отображает их только для чтения (с очищенным флагом PTE_W). Родительский и дочерний процессы могут читать из общей физической памяти. Если кто-либо из них записывает в данную страницу, процессор RISC-V генерирует исключение при обращении к странице. Обработчик ловушек ядра реагирует, выделяя новую страницу физической памяти и копируя в неё физическую страницу, на которую отображён адрес, вызвавший ошибку.

Ядро изменяет соответствующую запись PTE в таблице страниц аварийного процесса, чтобы та указывала на копию и разрешала как запись, так и чтение, после чего возобновляет выполнение аварийного процесса с инструкции, вызвавшей ошибку. Поскольку PTE теперь разрешает запись, повторно выполненная инструкция завершится без ошибки. Копирование при записи требует ведения учёта, чтобы определять, когда физические страницы могут быть освобождены, поскольку каждая страница может быть ссылаемой из разного числа таблиц страниц в зависимости от истории форков, ошибок страниц, exec и завершений. Этот учёт позволяет выполнить важную оптимизацию: если процесс получает ошибку страницы при записи и физическая страница ссылается только из таблицы страниц этого процесса, копирование не нужно.

Копирование при записи делает fork быстрее, поскольку fork не нужно копировать память. Часть памяти придётся скопировать позже при записи, но зачастую большая часть памяти так и не копируется. Типичный пример — fork с последующим exec: несколько страниц могут быть записаны после fork, но затем exec дочернего процесса освобождает основную часть памяти, унаследованной от родителя. Форк с копированием при записи исключает необходимость когда-либо копировать эту память. Кроме того, COW-форк прозрачен: никаких изменений в приложениях не требуется для получения его преимуществ.

Сочетание таблиц страниц и исключений при обращении к страницам открывает широкие возможности помимо COW-форка. Другой широко используемой функцией является ленивое выделение памяти, состоящее из двух частей. Во-первых, когда приложение запрашивает дополнительную память, вызывая sbrk, ядро отмечает увеличение размера, но не выделяет физическую память и не создаёт PTE для нового диапазона виртуальных адресов. Во-вторых, при ошибке страницы на одном из этих новых адресов ядро выделяет страницу физической памяти и отображает её в таблице страниц. Как и COW-форк, ядро может реализовать ленивое выделение памяти прозрачно для приложений.

Поскольку приложения нередко запрашивают больше памяти, чем используют, ленивое выделение выгодно: ядро вообще не тратит усилий на страницы, которые приложение никогда не использует. Кроме того, если приложение запрашивает значительное увеличение адресного пространства, то sbrk без ленивого выделения обходится дорого: если приложение запрашивает гигабайт памяти, ядру придётся выделить и обнулить 262144 страницы по 4096 байт. Ленивое выделение позволяет распределить эти затраты во времени. С другой стороны, ленивое выделение создаёт дополнительные накладные расходы на ошибки страниц, которые сопровождаются переходом между ядром и пользователем. Операционные системы могут снизить эти расходы, выделяя группу последовательных страниц на одну ошибку страницы вместо одной страницы и специализируя код входа/выхода ядра для таких ошибок страниц.

Ещё одна широко используемая функция, эксплуатирующая ошибки страниц, — подкачка страниц по требованию. В exec, xv6 загружает весь текст и данные приложения в память заранее. Поскольку приложения могут быть большими и чтение с диска обходится дорого, эти начальные затраты могут быть заметны пользователям: когда

пользователь запускает крупное приложение из командной оболочки, до появления отклика может пройти немало времени. Для улучшения времени отклика современные ядра создают таблицу страниц для пользовательского адресного пространства, но помечают PTE страниц как недействительные. При ошибке страницы ядро читает содержимое страницы с диска и отображает её в пользовательское адресное пространство. Как COW-форк и ленивое выделение, ядро может реализовать эту функцию прозрачно для приложений.

Программы, выполняемые на компьютере, могут нуждаться в большем объёме памяти, чем имеется ОЗУ. Для корректной работы в таких условиях операционная система может реализовать подкачку на диск. Идея состоит в хранении только части пользовательских страниц в ОЗУ, а остальных — на диске в области подкачки. Ядро помечает PTE, соответствующие памяти, хранящейся в области подкачки (и, следовательно, не в ОЗУ), как недействительные. Если приложение пытается использовать одну из выгруженных на диск страниц, возникает ошибка страницы, и страницу необходимо подкачать: обработчик ловушек ядра выделяет страницу физического ОЗУ, читает страницу с диска в ОЗУ и изменяет соответствующую PTE, чтобы та указывала на ОЗУ.

Что происходит, если страницу нужно подкачать, но нет свободного физического ОЗУ? В таком случае ядро должно сначала освободить физическую страницу, выгрузив её или вытеснив в область подкачки на диске, и пометить PTE, ссылающиеся на эту физическую страницу, как недействительные. Вытеснение обходится дорого, поэтому подкачка работает лучше, когда она редкая: если приложения используют только часть своих страниц памяти и объединение этих подмножеств умещается в ОЗУ. Это свойство часто называют хорошей локальностью обращений. Как и многие другие техники виртуальной памяти, ядра обычно реализуют подкачку на диск прозрачно для приложений.

Компьютеры нередко работают с малым количеством или вовсе без свободной физической памяти, независимо от объёма ОЗУ. Например, облачные провайдеры мультиплексируют многих клиентов на одной машине, чтобы экономно использовать аппаратные затраты. Другой пример: пользователи запускают множество приложений на смартфонах при небольшом объёме физической памяти. В таких условиях выделение страницы может потребовать предварительного вытеснения существующей страницы. Таким образом, при дефиците свободной физической памяти выделение памяти обходится дорого.

Ленивое выделение и подкачка по требованию особенно выгодны при нехватке свободной памяти. Заблаговременное выделение памяти в sbrk или exec влечёт дополнительные затраты на вытеснение для освобождения памяти. Кроме того, есть риск, что заблаговременная работа окажется напрасной, поскольку до того как приложение воспользуется страницей, операционная система может её уже вытеснить.

Другие возможности, сочетающие подкачку страниц и исключения при обращении к страницам, включают автоматическое расширение стеков и отображение файлов в память.

4.7 Реальный мир

Трамплин и фрейм ловушки могут показаться чрезмерно сложными. Основная причина — то, что RISC-V намеренно делает как можно меньше при инициировании ловушки, чтобы обеспечить возможность очень быстрой обработки ловушек, что оказывается важным на практике. В результате первые несколько инструкций обработчика ловушек ядра фактически должны выполняться в пользовательской среде: с пользовательской таблицей страниц и содержимым пользовательских регистров. При этом обработчик ловушек изначально не знает таких полезных фактов, как идентификатор выполняющегося процесса или адрес ядерной таблицы страниц. Решение возможно благодаря тому, что RISC-V предоставляет защищённые места, в которых ядро может хранить информацию перед входом в пространство пользователя: регистр sscratch и записи пользовательской таблицы страниц, указывающие на ядерную память, но защищённые отсутствием PTE_U. Трамплин и фрейм ловушки xv6 используют именно эти возможности RISC-V.

Необходимость в специальных страницах трамплина исчезла бы, если бы ядерная память была отображена в пользовательскую таблицу страниц каждого процесса (с надлежащими флагами разрешений PTE). Это также исключило бы необходимость переключения таблицы страниц при ловушке из пространства пользователя в ядро. Это, в свою очередь, позволило бы реализациям системных вызовов в ядре использовать преимущество отображённой пользовательской памяти текущего процесса, позволяя ядерному коду напрямую разыменовывать пользовательские указатели. Многие операционные системы применяли эти идеи для повышения эффективности. Xv6 их избегает, чтобы снизить вероятность уязвимостей безопасности в ядре из-за непреднамеренного использования пользовательских указателей, а также чтобы уменьшить часть сложности, которая потребовалась бы для обеспечения отсутствия пересечения виртуальных адресов пользователя и ядра.

Производственные операционные системы реализуют форк с копированием при записи, ленивое выделение памяти, подкачку по требованию, подкачку на диск, отображение файлов в память и т.д. Кроме того, производственные операционные системы стремятся использовать всю физическую память — либо для приложений, либо для кешей (например, буферный кеш файловой системы, который мы рассмотрим позже в Section 8.2). Xv6 наивен в этом отношении: вы хотите, чтобы ваша операционная система использовала купленную вами физическую память, но xv6 этого не делает. Более того, если xv6 исчерпает память, она возвращает ошибку работающему приложению или уничтожает его, вместо того чтобы, например, вытеснить страницу другого приложения.

4.8 Упражнения

1. Функции copyin и copyinstr обходят пользовательскую таблицу страниц программно. Настройте ядерную таблицу страниц так, чтобы ядро имело отображение поль-

зовательской программы, и copyin и copyinstr могли использовать memcpy для копирования аргументов системных вызовов в пространство ядра, полагаясь на аппаратный обход таблицы страниц.

2. Реализуйте ленивое выделение памяти.

3. Реализуйте COW-форк.

4. Можно ли исключить специальное отображение страницы TRAPFRAME в каждом пользовательском адресном пространстве? Например, можно ли изменить uservec так, чтобы просто помещать 32 пользовательских регистра в стек ядра или хранить их в структуре proc?

5. Можно ли изменить xv6, чтобы исключить специальное отображение страницы TRAMPOLINE?

Глава 5

Прерывания и драйверы устройств

Драйвер —это код в операционной системе, управляющий конкретным устройством: он конфигурирует аппаратуру устройства, отдаёт устройству команды на выполнение операций, обрабатывает возникающие прерывания и взаимодействует с процессами, которые могут ожидать ввода-вывода от устройства. Код драйвера бывает непростым, поскольку драйвер выполняется параллельно с управляемым им устройством. Кроме того, драйвер должен понимать аппаратный интерфейс устройства, который может быть сложным и слабо задокументированным.

Устройства, требующие внимания операционной системы, как правило, можно настроить на генерацию прерываний —одного из видов ловушек. Код обработки ловушек в ядре обнаруживает, что устройство выставило прерывание, и вызывает обработчик прерывания драйвера; в xv6 эта диспетчеризация происходит в devintr (kernel/trap.c:186).

Многие драйверы устройств выполняют код в двух контекстах: верхняя половина работает в потоке ядра процесса, а нижняя половина выполняется во время обработки прерывания. Верхняя половина вызывается через системные вызовы, такие как read и write, когда устройству нужно выполнить ввод-вывод. Этот код может попросить аппаратуру начать операцию (например, запросить чтение блока с диска), а затем ждать её завершения. В конечном счёте устройство завершает операцию и выставляет прерывание. Обработчик прерывания драйвера, выступая в роли нижней половины, определяет, какая операция завершилась, при необходимости пробуждает ожидающий процесс и сообщает аппаратуре, что можно приступать к следующей ожидающей операции.

5.1 Код: ввод с консоли

Драйвер консоли (kernel/console.c) —это простой пример структуры драйвера. Консольный драйвер принимает символы, вводимые пользователем, через последовательный порт UART, подключённый к RISC-V. Драйвер консоли накапливает ввод по одной строке, обрабатывая специальные символы, такие как backspace и control-u. Пользовательские процессы, например оболочка, используют системный вызов read для получения строк ввода с консоли. При вводе данных в xv6 под QEMU нажатые клавиши

передаются в xv6 через эмулируемую QEMU аппаратуру UART.

Аппаратура UART, с которой работает драйвер, — это микросхема 16550 [13], эмулируемая QEMU. На реальном компьютере 16550 управляла бы последовательным каналом RS232, соединяющим с терминалом или другим компьютером. При работе под QEMU она подключена к вашей клавиатуре и дисплею.

Аппаратура UART представлена для программного обеспечения в виде набора регистров управления, отображённых на память. То есть существуют физические адреса, по которым аппаратура RISC-V подключена к устройству UART, так что загрузки и сохранения взаимодействуют с аппаратурой устройства, а не с ОЗУ. Адреса отображения в память для UART начинаются с 0x10000000, или UART0 (kernel/memlayout.h:21). Имеется несколько регистров управления UART, каждый размером в байт. Их смещения от UART0 определены в (kernel/uart.c:24). Например, регистр LSR содержит биты, показывающие, есть ли входные символы, ожидающие чтения программой. Эти символы (если они есть) доступны для чтения из регистра RHR. При каждом чтении аппаратура UART удаляет его из внутренней FIFO-очереди ожидающих символов и сбрасывает бит «готовности» в LSR, когда FIFO пуста. Передающая часть UART во многом независима от приёмной; если программное обеспечение записывает байт в THR, UART передаёт этот байт.

main в xv6 вызывает consoleinit (kernel/console.c:188) для инициализации аппаратуры UART. Этот код настраивает UART на генерацию прерывания при приёме каждого байта входных данных и прерывания о завершении передачи каждый раз, когда UART заканчивает отправку байта выходных данных (kernel/uart.c:49).

Оболочка xv6 читает с консоли через файловый дескриптор, открытый в init.c (user/init.c:19). Вызовы системного вызова read проходят через ядро до consoleread (kernel/console.c:86). consoleread ожидает поступления ввода (через прерывания) и его буферизации в cons.buf, копирует ввод в пространство пользователя и (после поступления полной строки) возвращает управление процессу пользователя. Если пользователь ещё не ввёл полную строку, все читающие процессы будут ожидать в вызове sleep (kernel/console.c:102) (подробности о sleep объясняет Глава 7).

Когда пользователь набирает символ, аппаратура UART запрашивает у RISC-V выставление прерывания, которое активирует обработчик ловушек xv6. Обработчик ловушек вызывает devintr (kernel/trap.c:186), который проверяет регистр scause RISC-V и обнаруживает, что прерывание получено от внешнего устройства. Затем он обращается к аппаратному блоку PLIC [3], чтобы узнать, какое устройство вызвало прерывание (kernel/trap.c:194). Если это был UART, devintr вызывает uartintr.

uartintr (kernel/uart.c:142) считывает все ожидающие входные символы из аппаратуры UART и передаёт их в consoleintr (kernel/console.c:142); он не ждёт символов, поскольку будущий ввод породит новое прерывание. Задача consoleintr — накапливать входные символы в cons.buf до поступления полной строки. consoleintr особым образом обрабатывает backspace и несколько других символов. Когда поступает символ новой строки, consoleintr пробуждает ожидающий consoleread (если он есть).

После пробуждения consoleread обнаруживает полную строку в cons.buf, копирует

её в пространство пользователя и возвращает управление (через механизм системного вызова) в пространство пользователя.

5.2 Код: вывод на консоль

Системный вызов write на файловом дескрипторе, подключённом к консоли, в итоге поступает в uartputc (kernel/uart.c:106). Драйвер устройства поддерживает выходной буфер (uart_tx_buf), чтобы пишущим процессам не нужно было ждать, пока UART закончит отправку; вместо этого uartputc добавляет каждый символ в буфер, вызывает uartstart для запуска передачи устройством (если она ещё не ведётся) и возвращает управление. Единственный случай, когда uartputc ждёт, —это когда буфер уже заполнен.

Каждый раз, когда UART заканчивает передачу байта, он генерирует прерывание. uartintr вызывает uartstart, который проверяет, что устройство действительно завершило отправку, и передаёт устройству следующий буферизованный выходной символ. Таким образом, если процесс записывает в консоль несколько байтов, как правило, первый байт будет отправлен вызовом uartstart из uartputc, а оставшиеся буферизованные байты будут отправлены вызовами uartstart из uartintr по мере поступления прерываний о завершении передачи.

Стоит отметить общий паттерн: разделение активности устройства и активности процесса с помощью буферизации и прерываний. Драйвер консоли может обрабатывать ввод, даже когда ни один процесс не ожидает его чтения; последующее чтение увидит этот ввод. Аналогично, процессы могут отправлять вывод, не ожидая готовности устройства. Такое разделение может повышать производительность, позволяя процессам выполняться параллельно с вводом-выводом устройства, и особенно важно, когда устройство медленное (как UART) или требует немедленной реакции (как эхо набираемых символов). Эта идея иногда называется конкурентным вводом-выводом.

5.3 Конкурентность в драйверах

Вы, вероятно, заметили вызовы acquire в consoleread и в consoleintr. Эти вызовы захватывают блокировку, которая защищает структуры данных драйвера консоли от конкурентного доступа. Здесь присутствуют три опасности, связанные с конкурентностью: два процесса на разных процессорах могут одновременно вызвать consoleread; аппаратура может попросить процессор доставить прерывание консоли (точнее, UART), пока этот процессор уже выполняется внутри consoleread; и аппаратура может доставить прерывание консоли на другом процессоре, пока consoleread выполняется. Эти опасности могут привести к гонкам или взаимоблокировкам. Глава 6 исследует эти проблемы и то, как блокировки могут их решить.

Ещё один аспект, требующий осторожности при конкурентном программировании драйверов, состоит в том, что один процесс может ожидать ввода от устройства, но

прерывание, сигнализирующее о поступлении ввода, может прийти, когда выполняется другой процесс (или не выполняется никакой). Поэтому обработчики прерываний не вправе делать предположения о процессе или коде, который они прервали. Например, обработчик прерывания не может безопасно вызывать copyout с таблицей страниц текущего процесса. Обработчики прерываний обычно делают сравнительно мало работы (например, просто копируют входные данные в буфер) и пробуждают код верхней половины для выполнения остального.

5.4 Прерывания таймера

Xv6 использует прерывания таймера для поддержания своих часов и для переключения между вычислительно-интенсивными процессами; вызовы yield в usertrap и kerneltrap обеспечивают это переключение. Прерывания таймера поступают от тактовой аппаратуры, подключённой к каждому процессору RISC-V. Xv6 программирует эту тактовую аппаратуру на периодическое прерывание каждого процессора.

RISC-V требует, чтобы прерывания таймера обрабатывались в машинном режиме, а не в режиме супервизора. Машинный режим RISC-V работает без подкачки страниц и с отдельным набором управляющих регистров, поэтому запускать обычный код ядра xv6 в машинном режиме нецелесообразно. В результате xv6 обрабатывает прерывания таймера полностью отдельно от механизма ловушек, описанного выше.

Код, выполняемый в машинном режиме в start.c, до main, настраивает приём прерываний таймера (kernel/start.c:53). Часть работы — запрограммировать аппаратуру CLINT (core-local interruptor) на генерацию прерывания через определённую задержку. Другая часть —подготовить область сохранения состояния, аналогичную trapframe, для помощи обработчику прерывания таймера в сохранении регистров и адреса регистров CLINT. Наконец, start устанавливает mtvec на timervec и включает прерывания таймера.

Прерывание таймера может произойти в любой момент выполнения пользовательского или ядерного кода; у ядра нет возможности отключить прерывания таймера во время критических операций. Поэтому обработчик прерывания таймера должен выполнять свою работу так, чтобы гарантированно не нарушить прерванный код ядра. Основная стратегия —попросить RISC-V выставить «программное прерывание» и немедленно вернуться. RISC-V доставляет программные прерывания ядру через обычный механизм ловушек и позволяет ядру их отключать. Код обработки программного прерывания, порождённого прерыванием таймера, можно

Обработчик прерывания таймера в машинном режиме —это timervec Он сохраняет несколько регистров в области сохранения состояния, подготовленной start, сообщает CLINT, когда генерировать следующее прерывание таймера, просит RISC-V выставить программное прерывание, восстанавливает регистры и возвращает управление. В обработчике прерывания таймера нет кода на C.

5.5 Реальный мир

Xv6 допускает прерывания устройств и таймера как во время выполнения кода ядра, так и при выполнении пользовательских программ. Прерывания таймера принудительно вызывают переключение потока (вызов yield) из обработчика прерывания таймера, даже при выполнении в ядре. Возможность справедливого разделения процессорного времени между потоками ядра полезна, если потоки ядра иногда долго вычисляют, не возвращаясь в пространство пользователя. Однако необходимость для кода ядра учитывать, что он может быть приостановлен (из-за прерывания таймера) и позднее продолжить выполнение на другом процессоре, —источник некоторой сложности в xv6 (см. Раздел 6.6). Ядро можно было бы упростить, если бы прерывания устройств и таймера происходили только во время выполнения пользовательского кода.

Поддержка всех устройств типичного компьютера в полном объёме требует большого труда, поскольку устройств много, у них масса функций, а протокол взаимодействия устройства с драйвером может быть сложным и слабо задокументированным. Во многих операционных системах на драйверы приходится больше кода, чем на ядро.

Драйвер UART извлекает данные по одному байту, читая управляющие регистры UART; этот паттерн называется программируемым вводом-выводом, поскольку программное обеспечение управляет перемещением данных. Программируемый ввод-вывод прост, но слишком медленен для использования при высоких скоростях передачи данных. Устройства, которым нужно перемещать большие объёмы данных с высокой скоростью, как правило, используют прямой доступ к памяти (DMA). Аппаратура устройства DMA непосредственно записывает принятые данные в ОЗУ и читает отправляемые данные из ОЗУ. Современные дисковые и сетевые устройства используют DMA. Драйвер устройства DMA подготавливает данные в ОЗУ, а затем одной записью в управляющий регистр сообщает устройству о необходимости обработать подготовленные данные.

Прерывания целесообразны, когда устройство требует внимания в непредсказуемые моменты и не слишком часто. Однако прерывания имеют высокие накладные расходы на процессоре. Поэтому быстродействующие устройства, такие как сетевые и дисковые контроллеры, применяют приёмы, снижающие потребность в прерываниях. Один из приёмов —выставлять одно прерывание на целый пакет входящих или исходящих запросов. Другой приём —полностью отключить прерывания в драйвере и периодически опрашивать устройство, нуждается ли оно во внимании. Эта техника называется опросом (polling). Опрос целесообразен, если устройство выполняет операции очень быстро, но расходует процессорное время впустую, если устройство в основном простаивает. Некоторые драйверы динамически переключаются между опросом и прерываниями в зависимости от текущей нагрузки на устройство.

Драйвер UART копирует входные данные сначала в буфер в ядре, а затем в пространство пользователя. Это оправдано при низких скоростях передачи данных, но такое двойное копирование может существенно снизить производительность для устройств, которые генерируют или потребляют данные очень быстро. Некоторые операционные системы способны напрямую перемещать данные между буферами в пространстве пользователя и аппаратурой устройства, нередко с использованием DMA.

Как упоминается в Главе 1, консоль представляется приложениям как обычный файл, и приложения читают ввод и пишут вывод с помощью системных вызовов read и write. Приложения могут захотеть управлять аспектами устройства, которые нельзя выразить через стандартные вызовы файловой системы (например, включение/выключение построчной буферизации в драйвере консоли). Unix-системы поддерживают системный вызов ioctl для таких случаев.

В некоторых применениях компьютеров система должна реагировать в течение ограниченного времени. Например, в системах, критически важных для безопасности, пропуск временного дедлайна может привести к катастрофе. Xv6 не подходит для жёстких систем реального времени. Операционные системы для жёсткого реального времени, как правило, являются библиотеками, компонуемыми с приложением таким образом, что позволяет анализировать наихудшее время отклика. Xv6 также не подходит для мягких приложений реального времени, где изредка допустимо пропускать дедлайн, поскольку планировщик xv6 слишком примитивен, а в пути выполнения кода ядра прерывания отключаются на длительное время.

5.6 Упражнения

1. Измените uart.c, чтобы полностью отказаться от прерываний. Возможно, потребуется также изменить console.c.

2. Добавьте драйвер для сетевой карты Ethernet.

Глава 6

Блокировки

Большинство ядер, включая xv6, чередуют выполнение нескольких видов деятельности. Одним из источников такого чередования является многопроцессорное оборудование: компьютеры с несколькими процессорами, работающими независимо, как, например, RISC-V в xv6. Эти процессоры совместно используют физическую оперативную память, и xv6 использует это совместное использование для поддержания структур данных, которые все процессоры читают и записывают. Совместное использование порождает возможность того, что один процессор читает структуру данных, пока другой находится в середине её обновления, или даже несколько процессоров одновременно обновляют одни и те же данные; без тщательного проектирования такой параллельный доступ скорее всего приведёт к некорректным результатам или повреждению структуры данных. Даже на однопроцессорной машине ядро может переключать процессор между несколькими потоками, вызывая чередование их выполнения. Наконец, обработчик прерывания устройства, изменяющий те же данные, что и прерываемый код, может повредить данные, если прерывание произойдёт в самый неподходящий момент. Термин конкурентность обозначает ситуации, в которых несколько потоков инструкций выполняются вперемежку из-за параллелизма на многопроцессорных системах, переключения потоков или прерываний.

Ядра насыщены данными, к которым происходит конкурентный доступ. Например, два процессора могут одновременно вызвать kalloc, тем самым конкурентно извлекая элементы из начала списка свободных страниц. Разработчики ядер стремятся допускать высокий уровень конкурентности, поскольку это может повысить производительность за счёт параллелизма и улучшить отзывчивость. Однако в результате разработчики ядра должны убедиться в корректности кода несмотря на такую конкурентность. Существует множество способов достичь корректного кода, одни легче поддаются анализу, чем другие. Стратегии, направленные на обеспечение корректности в условиях конкурентности, а также абстракции, поддерживающие их, называются техниками управления конкурентностью.

Xv6 использует ряд техник управления конкурентностью в зависимости от ситуации; возможных техник значительно больше. В этой главе рассматривается широко применяемая техника: блокировка. Блокировка обеспечивает взаимное исключение, га-

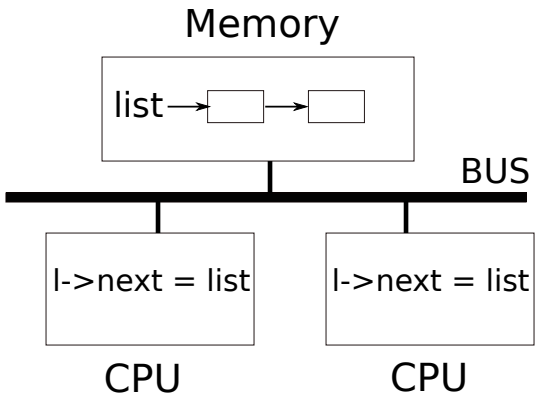

Рис. 6.1: Упрощённая архитектура SMP

рантируя, что в каждый момент времени только один процессор может удерживать блокировку. Если программист ассоциирует блокировку с каждым совместно используемым элементом данных, и код всегда удерживает соответствующую блокировку при работе с элементом, то элемент будет использоваться только одним процессором в каждый момент времени. В такой ситуации говорят, что блокировка защищает элемент данных. Несмотря на то что блокировки являются легко понятным механизмом управления конкурентностью, их недостаток состоит в том, что они могут ограничивать производительность, поскольку упорядочивают конкурентные операции последовательно.

В остальной части этой главы объясняется, почему xv6 нуждается в блокировках, как xv6 реализует их и как использует.

6.1 Гонки

В качестве примера того, почему нужны блокировки, рассмотрим два процесса с завершившимися дочерними процессами, вызывающих wait на двух разных процессорах. wait освобождает память дочернего процесса. Таким образом, на каждом процессоре ядро вызовет kfree для освобождения страниц памяти дочерних процессов. Распределитель памяти ядра поддерживает связный список: kalloc() (kernel/kalloc.c:69) извлекает страницу памяти из списка свободных страниц, а kfree() (kernel/kalloc.c:47) помещает страницу в список свободных. Для наилучшей производительности хотелось бы, чтобы вызовы kfree двух родительских процессов выполнялись параллельно, не ожидая друг друга, но это было бы некорректно при данной реализации kfree в xv6.

Рисунок 6.1 иллюстрирует ситуацию подробнее: связный список свободных страниц находится в памяти, совместно используемой двумя процессорами, которые работают со списком с помощью инструкций загрузки и сохранения. (В действительности процессоры имеют кэши, но концептуально многопроцессорные системы ведут себя так, словно существует единая разделяемая память.) Если бы не было конкурентных запросов, операцию добавления в список push можно было бы реализовать следующим образом:

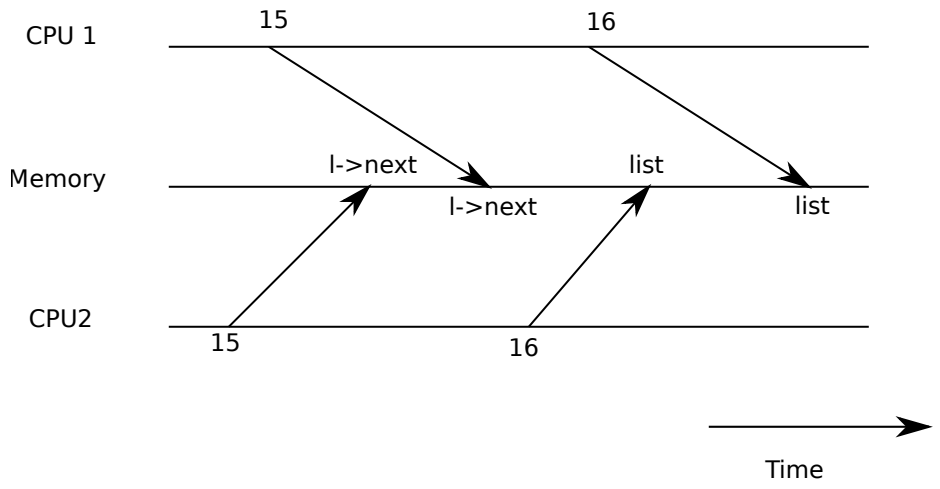

Рис. 6.2: Пример гонки

```
1      struct element {
2        int data;
3        struct element *next;
4      };
5
6      struct element *list = 0;
7
8      void
9      push(int data)
10     {
11       struct element *l;
12
13       l = malloc(sizeof *l);
14       l->data = data;
15       l->next = list;
16       list = l;
17     }
```

Эта реализация корректна при изолированном выполнении. Однако код некорректен, если более одного экземпляра выполняется конкурентно. Если два процессора выполняют push одновременно, оба могут выполнить строку 15, как показано на рис. 6.1, прежде чем любой из них выполнит строку 16, что приводит к некорректному результату, показанному на рисунке 6.2. После этого окажутся два элемента списка, у которых next указывает на прежнее значение list. Когда два присваивания list произойдут в строке 16, второе перезапишет первое; элемент, участвовавший в первом присваивании, будет потерян.

Потеря обновления в строке 16 является примером гонки. Гонка — это ситуация, при которой к ячейке памяти происходит конкурентный доступ, причём хотя бы один из доступов является записью. Гонка часто является признаком ошибки: либо потери

обновления (если оба доступа являются записью), либо чтения неполностью обновлённой структуры данных. Результат гонки зависит от машинного кода, сгенерированного компилятором, временного соотношения двух задействованных процессоров и порядка, в котором память обрабатывает их операции, что может затруднить воспроизведение и отладку ошибок, вызванных гонками. Например, добавление операторов вывода во время отладки push может изменить временную картину выполнения настолько, что гонка исчезнет.

Обычный способ избежать гонок — использовать блокировку. Блокировки обеспечивают взаимное исключение, позволяя только одному процессору в каждый момент времени выполнять критические строки push; это делает описанный выше сценарий невозможным. Корректная версия приведённого кода с блокировкой добавляет всего несколько строк (выделены жёлтым):

```
 6    struct element *list = 0;
 7    struct lock listlock;
 8
 9    void
10    push(int data)
11    {
12      struct element *l;
13      l = malloc(sizeof *l);
14      l->data = data;
15
16      acquire(&listlock);
17      l->next = list;
18      list = l;
19      release(&listlock);
20    }
```

Последовательность инструкций между acquire и release обычно называется критической секцией. Принято говорить, что блокировка защищает list .

Когда мы говорим, что блокировка защищает данные, мы на самом деле имеем в виду, что блокировка защищает некоторый набор инвариантов, применимых к данным. Инварианты — это свойства структур данных, которые сохраняются в ходе операций. Как правило, корректное поведение операции зависит от того, выполняются ли инварианты в момент начала операции. Операция может временно нарушить инварианты, но должна восстановить их перед завершением. Например, в случае связного списка инвариант состоит в том, что list указывает на первый элемент списка и что поле next каждого элемента указывает на следующий элемент. Реализация push временно нарушает этот инвариант: в строке 17 l указывает на следующий элемент списка, но list ещё не указывает на l (это исправляется в строке 18). Рассмотренная выше гонка произошла потому, что второй процессор выполнял код, опирающийся на инварианты списка, пока они были (временно) нарушены. Правильное использование блокировки гарантирует, что только один процессор в каждый момент может работать со структурой данных в критической секции, поэтому ни один процессор не выполнит операцию

над структурой данных, когда инварианты структуры данных не выполняются.

Можно представить блокировку как механизм упорядочивания конкурентных критических секций — они выполняются по одной в каждый момент времени, тем самым сохраняя инварианты (при условии, что критические секции корректны при изолированном выполнении). Также можно считать, что критические секции, защищённые одной и той же блокировкой, являются атомарными по отношению друг к другу, так что каждая из них видит только полный набор изменений от предшествующих критических секций и никогда не видит частично завершённых обновлений.

Несмотря на полезность для корректности, блокировки по своей природе ограничивают производительность. Например, если два процесса вызывают kfree конкурентно, блокировки упорядочат эти две критические секции последовательно, так что не будет никакой выгоды от их выполнения на разных процессорах. Говорят, что несколько процессов конфликтуют, если они хотят получить одну и ту же блокировку одновременно, или что блокировка испытывает конкуренцию. Серьёзным вызовом при проектировании ядра является избегание конкуренции за блокировки ради достижения параллелизма. Xv6 делает для этого немного, но сложные ядра организуют структуры данных и алгоритмы специально, чтобы избежать конкуренции за блокировки. В примере со списком ядро может поддерживать отдельный список свободных страниц для каждого процессора и обращаться к списку другого процессора только тогда, когда список текущего процессора пуст и ему нужно похитить память у другого процессора. В других случаях могут потребоваться более сложные решения.

Расположение блокировок также важно для производительности. Например, было бы корректно переместить acquire ранее в push, перед строкой 13. Но это, вероятно, снизило бы производительность, поскольку тогда вызовы malloc стали бы выполняться последовательно. В разделе «Использование блокировок» ниже приводятся некоторые рекомендации по выбору мест для вставки acquire и release.

6.2 Код: блокировки

Xv6 имеет два вида блокировок: спин-блокировки и блокировки сна. Начнём со спин-блокировок. Xv6 представляет спин-блокировку как struct spinlock (kernel/spinlock.h:2). Важным полем структуры является locked — слово, равное нулю, когда блокировка свободна, и ненулевое, когда она удерживается. Логически, xv6 должна захватывать блокировку, выполняя код вроде

```
void
acquire(struct spinlock *lk) // does not work!
{
  for(;;) {
    if(lk->locked == 0) {
      lk->locked = 1;
      break;
    }
```

```
29          }
30      }
```

К сожалению, эта реализация не гарантирует взаимного исключения на многопроцессорной машине. Может случиться так, что два процессора одновременно достигнут строки 25, увидят, что lk->locked равно нулю, и оба захватят блокировку, выполнив строку 26. В этот момент два разных процессора удерживают блокировку, что нарушает свойство взаимного исключения. Нам нужен способ сделать так, чтобы строки 25 и 26 выполнялись как одно атомарное (т.е. неделимое) действие.

Поскольку блокировки широко используются, многоядерные процессоры обычно предоставляют инструкции, реализующие атомарный аналог строк 25 и 26. На RISC-V такой инструкцией является amoswap r, a. amoswap читает значение по адресу памяти a, записывает содержимое регистра r по этому адресу и помещает прочитанное значение в r. То есть она обменивает содержимое регистра и ячейки памяти. Она выполняет эту последовательность атомарно, используя специальное аппаратное обеспечение для предотвращения использования другими процессорами данного адреса памяти между чтением и записью.

Функция acquire (kernel/spinlock.c:22) в xv6 использует переносимый вызов библиотеки C __sync_lock_test_and_set, который сводится к инструкции amoswap; возвращаемое значение — старое (обменянное) содержимое lk->locked. Функция acquire оборачивает обмен в цикл, повторяя попытки (крутясь), пока не захватит блокировку. На каждой итерации в lk->locked помещается единица и проверяется предыдущее значение; если предыдущее значение равно нулю, блокировка захвачена, и обмен установит lk->locked в единицу. Если предыдущее значение равно единице, какой-то другой процессор удерживает блокировку, а атомарная запись единицы в lk->locked не изменила её значения.

После захвата блокировки acquire для отладки записывает процессор, захвативший блокировку. Поле lk->cpu защищено блокировкой и должно изменяться только при удерживаемой блокировке.

Функция release (kernel/spinlock.c:47) является противоположностью acquire: она очищает поле lk->cpu и затем освобождает блокировку. Концептуально освобождение требует лишь присвоения нуля lk->locked. Стандарт C позволяет компиляторам реализовать присваивание несколькими инструкциями сохранения, поэтому присваивание на C может быть неатомарным по отношению к конкурентному коду. Вместо этого release использует функцию библиотеки C __sync_lock_release, выполняющую атомарное присваивание. Эта функция также сводится к инструкции RISC-V amoswap.

6.3 Код: использование блокировок

Xv6 использует блокировки во многих местах для избежания гонок. Как описано выше, kalloc (kernel/kalloc.c:69) и kfree (kernel/kalloc.c:47) являются хорошим примером. Попробуйте упражнения 1 и 2, чтобы увидеть, что произойдёт, если эти функции не используют блокировки. Вы, скорее всего, обнаружите, что трудно спровоцировать некорректное поведение, что говорит о сложности надёжного тестирования кода на наличие

ошибок блокировок и гонок. В xv6 вполне могут быть ещё не обнаруженные гонки.

Трудная часть использования блокировок — решить, сколько блокировок использовать и какие данные и инварианты должна защищать каждая блокировка. Есть несколько основных принципов. Во-первых, всякий раз, когда переменная может быть записана одним процессором одновременно с тем, как другой процессор может читать или записывать её, следует использовать блокировку, чтобы эти две операции не перекрывались. Во-вторых, помните, что блокировки защищают инварианты: если инвариант охватывает несколько ячеек памяти, как правило, все они должны быть защищены одной блокировкой для обеспечения выполнения инварианта.

Приведённые выше правила говорят, когда блокировки необходимы, но ничего не говорят о том, когда блокировки излишни, а для эффективности важно не блокировать слишком много, поскольку блокировки снижают параллелизм. Если параллелизм не важен, можно организовать работу в одном потоке и не беспокоиться о блокировках. Простое ядро может сделать это на многопроцессорной машине, имея единственную блокировку, которая должна быть захвачена при входе в ядро и освобождена при выходе из него (хотя блокирующие системные вызовы, такие как чтение из канала или wait, создадут проблему). Многие однопроцессорные операционные системы были преобразованы для работы на многопроцессорных машинах с использованием этого подхода, иногда называемого «большой блокировкой ядра», но такой подход жертвует параллелизмом: только один процессор может выполнять код ядра в каждый момент времени. Если ядро выполняет тяжёлые вычисления, было бы эффективнее использовать более широкий набор мелкозернистых блокировок, чтобы ядро могло выполняться на нескольких процессорах одновременно.

В качестве примера крупнозернистых блокировок: распределитель в kalloc.c у xv6 имеет единственный список свободных страниц, защищённый одной блокировкой. Если несколько процессов на разных процессорах пытаются выделить страницы одновременно, каждый должен ждать своей очереди, вращаясь в цикле acquire. Вращение расходует процессорное время впустую, поскольку не производит полезной работы. Если конкуренция за блокировку расходует значительную долю процессорного времени, возможно, производительность можно повысить, изменив дизайн распределителя так, чтобы он имел несколько списков свободных страниц, каждый со своей блокировкой, что позволит по-настоящему параллельно выделять память.

В качестве примера мелкозернистых блокировок: в xv6 есть отдельная блокировка для каждого файла, так что процессы, работающие с разными файлами, часто могут продолжать работу, не ожидая блокировок друг друга. Схема блокировки файлов могла бы быть ещё более мелкозернистой, если бы требовалось позволить процессам одновременно записывать в разные области одного файла. В конечном счёте решения о гранулярности блокировок должны определяться как измерениями производительности, так и соображениями сложности.

По мере того как последующие главы объясняют каждую часть xv6, они будут приводить примеры использования блокировок в xv6 для работы с конкурентностью. Для предварительного ознакомления рисунок 6.3 перечисляет все блокировки в xv6.

Блокировка	Описание
bcache.lock	Защищает выделение записей кэша блочных буферов
cons.lock	Упорядочивает доступ к аппаратуре консоли, предотвращает перемешивание выв
ftable.lock	Упорядочивает выделение struct file в таблице файлов
itable.lock	Защищает выделение записей индексных узлов в памяти
vdisk_lock	Упорядочивает доступ к дисковой аппаратуре и очереди дескрипторов DMA
kmem.lock	Упорядочивает выделение памяти
log.lock	Упорядочивает операции с журналом транзакций
pipe's pi->lock	Упорядочивает операции с каждым каналом
pid_lock	Упорядочивает инкременты next_pid
proc's p->lock	Упорядочивает изменения состояния процесса
wait_lock	Помогает wait избежать потерянных пробуждений
tickslock	Упорядочивает операции со счётчиком ticks
inode's ip->lock	Упорядочивает операции с каждым индексным узлом и его содержимым
buf's b->lock	Упорядочивает операции с каждым блочным буфером

Рис. 6.3: Блокировки в xv6

6.4 Взаимная блокировка и порядок захвата блокировок

Если путь выполнения в ядре должен удерживать несколько блокировок одновременно, важно, чтобы все пути выполнения захватывали эти блокировки в одном и том же порядке. Если этого не придерживаться, возникает риск взаимной блокировки. Допустим, два пути выполнения в xv6 нуждаются в блокировках A и B, но путь 1 захватывает блокировки в порядке A, затем B, а другой путь захватывает их в порядке B, затем A. Предположим, что поток T1 выполняет путь 1 и захватывает блокировку A, а поток T2 выполняет путь 2 и захватывает блокировку B. Затем T1 попытается захватить блокировку B, а T2 —блокировку A. Оба захвата будут заблокированы бесконечно, поскольку в обоих случаях другой поток удерживает нужную блокировку и не освободит её, пока его захват не завершится. Чтобы избежать таких взаимных блокировок, все пути выполнения должны захватывать блокировки в одном и том же порядке. Необходимость глобального порядка захвата блокировок означает, что блокировки фактически являются частью спецификации каждой функции: вызывающие должны вызывать функции так, чтобы блокировки захватывались в согласованном порядке.

В xv6 существует много цепочек порядка блокировок длиной два с участием блокировок отдельных процессов (блокировка в каждом struct proc) из-за способа работы sleep (см. главу 7). Например, consoleintr (kernel/console.c:142) является процедурой обработки прерываний, обрабатывающей введённые символы. Когда приходит символ новой строки, любой процесс, ожидающий ввода с консоли, должен быть разбужен. Для этого consoleintr удерживает cons.lock при вызове wakeup, который захватывает блокировку ожидающего процесса, чтобы разбудить его. Следовательно, глобальный порядок во избежание взаимных блокировок включает правило, что cons.lock должна быть захвачена раньше любой блокировки процесса. Код файловой системы содержит самые длинные

цепочки блокировок в xv6. Например, создание файла требует одновременно удержания блокировки на каталог, блокировки на индексный узел нового файла, блокировки на буфер дискового блока, vdisk_lock дискового драйвера и блокировки p->lock вызывающего процесса. Чтобы избежать взаимной блокировки, код файловой системы всегда захватывает блокировки в порядке, упомянутом в предыдущем предложении.

Соблюдение глобального порядка во избежание взаимных блокировок может оказаться на удивление трудным. Иногда порядок блокировок вступает в противоречие с логической структурой программы, например, модуль M1 вызывает модуль M2, но порядок блокировок требует, чтобы блокировка в M2 была захвачена раньше блокировки в M1. Иногда идентификаторы блокировок заранее не известны, например, потому что одна блокировка должна удерживаться, чтобы определить идентификатор следующей захватываемой блокировки. Такая ситуация возникает в файловой системе при последовательном просмотре компонентов пути, а также в коде wait и exit при поиске дочерних процессов в таблице процессов. Наконец, опасность взаимной блокировки часто ограничивает степень мелкозернистости схемы блокировки, поскольку больше блокировок означает больше возможностей для взаимной блокировки. Необходимость избегать взаимных блокировок часто является важнейшим фактором при реализации ядра.

6.5 Реентерабельные блокировки

Может показаться, что некоторых взаимных блокировок и проблем с порядком захвата можно избежать, используя реентерабельные блокировки, которые также называются рекурсивными блокировками. Идея состоит в том, что если блокировка удерживается процессом и этот процесс пытается захватить блокировку снова, ядро может просто разрешить это (поскольку процесс уже имеет блокировку), вместо того чтобы вызвать панику, как это делает ядро xv6.

Однако оказывается, что реентерабельные блокировки усложняют анализ конкурентности: они разрушают интуицию о том, что блокировки делают критические секции атомарными по отношению к другим критическим секциям. Рассмотрим следующие две функции f и g:

```
struct spinlock lock;
int data = 0; // protected by lock

f() {
  acquire(&lock);
  if(data == 0){
    call_once();
    h();
    data = 1;
  }
  release(&lock);
}
```

```
g() {
  aquire(&lock);
  if(data == 0){
    call_once();
    data = 1;
  }
  release(&lock);
}
```

Глядя на этот фрагмент кода, интуиция подсказывает, что call_once будет вызвана только один раз: либо из f, либо из g, но не из обеих.

Но если разрешены реентерабельные блокировки и h вызывает g, то call_once будет вызвана дважды.

Если реентерабельные блокировки не разрешены, вызов g из h приведёт к взаимной блокировке, что тоже нехорошо. Но, при условии что двойной вызов call_once является серьёзной ошибкой, взаимная блокировка предпочтительнее. Разработчик ядра заметит взаимную блокировку (ядро паникует) и сможет исправить код, тогда как двойной вызов call_once может незаметно привести к труднообнаруживаемой ошибке.

По этой причине xv6 использует более понятные нереентерабельные блокировки. При условии что программисты помнят о правилах блокировок, оба подхода можно сделать работоспособными. Если бы xv6 использовала реентерабельные блокировки, потребовалось бы изменить acquire так, чтобы она замечала, что блокировка уже удерживается вызывающим потоком. Также пришлось бы добавить счётчик вложенных захватов в struct spinlock по образцу push_off, который рассматривается далее.

6.6 Блокировки и обработчики прерываний

Некоторые спин-блокировки xv6 защищают данные, используемые как потоками, так и обработчиками прерываний. Например, обработчик прерывания таймера clockintr может инкрементировать ticks (kernel/trap.c:165) примерно в тот же момент, когда поток ядра читает ticks в sys_sleep (kernel/sysproc.c:77). Блокировка tickslock упорядочивает эти два доступа.

Взаимодействие спин-блокировок и прерываний создаёт потенциальную опасность. Предположим, sys_sleep удерживает tickslock, и её процессор прерывается прерыванием таймера. clockintr попытается захватить tickslock, увидит, что она удерживается, и будет ждать её освобождения. В этой ситуации tickslock никогда не будет освобождена: только sys_sleep может её освободить, но sys_sleep не продолжит выполнение, пока clockintr не вернётся. Таким образом, процессор войдёт во взаимную блокировку, и любой код, которому нужна та или иная блокировка, также зависнет.

Чтобы избежать этой ситуации, если спин-блокировка используется обработчиком прерываний, процессор никогда не должен удерживать эту блокировку при включённых прерываниях. Xv6 использует более консервативный подход: когда процессор за-

хватывает любую блокировку, xv6 всегда отключает прерывания на этом процессоре. Прерывания всё ещё могут возникать на других процессорах, поэтому acquire в обработчике прерываний может ожидать, пока поток освободит спин-блокировку, — но только не на том же процессоре.

Xv6 повторно включает прерывания, когда процессор не удерживает ни одной спин-блокировки; при этом ему необходимо вести небольшой учёт для работы с вложенными критическими секциями. acquire вызывает push_off (kernel/spinlock.c:89), а release вызывает pop_off (kernel/spinlock.c:103) для отслеживания уровня вложенности блокировок на текущем процессоре. Когда этот счётчик достигает нуля, pop_off восстанавливает состояние разрешения прерываний, существовавшее в начале самой внешней критической секции. Функции intr_off и intr_on выполняют инструкции RISC-V для отключения и включения прерываний соответственно.

Важно, чтобы acquire вызывал push_off строго перед установкой lk->locked (kernel/spinlock.c:28). Если бы они были переставлены местами, существовало бы короткое окно, когда блокировка удерживалась бы при включённых прерываниях, и неудачно расположенное прерывание привело бы к взаимной блокировке системы. Аналогично, важно, чтобы release вызывал pop_off только после освобождения блокировки (kernel/spinlock.c:66).

6.7 Упорядочивание инструкций и памяти

Естественно думать, что программы выполняются в том порядке, в котором расположены операторы исходного кода. Это разумная ментальная модель для однопоточного кода, но она неверна, когда несколько потоков взаимодействуют через общую память [2, 4]. Одна из причин — компиляторы генерируют инструкции загрузки и сохранения в порядке, отличном от подразумеваемого исходным кодом, и могут полностью опустить их (например, кэшируя данные в регистрах). Другая причина — процессор может выполнять инструкции не по порядку для повышения производительности. Например, процессор может заметить, что в последовательности инструкций A и B не зависят друг от друга. Процессор может начать выполнение B первым, либо потому что его входные данные готовы раньше, чем у A, либо для того чтобы перекрыть выполнение A и B.

В качестве примера того, что может пойти не так, в коде push было бы катастрофой, если бы компилятор или процессор переместили сохранение, соответствующее строке 4, в точку после release в строке 6:

```
1        l = malloc(sizeof *l);
2        l->data = data;
3        acquire(&listlock);
4        l->next = list;
5        list = l;
6        release(&listlock);
```

Если бы такое переупорядочивание произошло, существовало бы окно, в течение которого другой процессор мог бы захватить блокировку и наблюдать обновлённый list, но видеть неинициализированный list->next.

Хорошая новость состоит в том, что компиляторы и процессоры помогают программистам, работающим с конкурентным кодом, следуя набору правил, называемых моделью памяти, и предоставляя некоторые примитивы для управления переупорядочиванием.

Чтобы указать аппаратуре и компилятору не выполнять переупорядочивание, xv6 использует __sync_synchronize() как в acquire (kernel/spinlock.c:22), так и в release (kernel/spinlock.c:47). __sync_synchronize() является барьером памяти: он указывает компилятору и процессору не переупорядочивать загрузки или сохранения через барьер. Барьеры в acquire и release в xv6 обеспечивают правильный порядок почти во всех случаях, где это важно, поскольку xv6 использует блокировки вокруг обращений к разделяемым данным. В главе 9 обсуждаются несколько исключений.

6.8 Блокировки сна

Иногда xv6 нужно удерживать блокировку долгое время. Например, файловая система (глава 8) держит файл заблокированным во время чтения и записи его содержимого на диск, и эти дисковые операции могут занимать десятки миллисекунд. Удерживание спин-блокировки так долго приводило бы к потерям, если другой процесс захочет её захватить, поскольку захватывающий процесс будет впустую тратить процессорное время, вращаясь в цикле. Ещё один недостаток спин-блокировок — процесс не может уступить процессор, удерживая спин-блокировку; хотелось бы иметь возможность это делать, чтобы другие процессы могли использовать процессор, пока процесс с блокировкой ждёт диска. Уступка процессора при удерживании спин-блокировки недопустима, поскольку это может привести к взаимной блокировке, если второй поток затем попытается захватить спин-блокировку: поскольку acquire не уступает процессор, вращение второго потока может не дать первому потоку выполниться и освободить блокировку. Уступка при удерживании блокировки также нарушила бы требование о том, что при удерживании спин-блокировки прерывания должны быть отключены. Таким образом, желателен такой тип блокировки, который уступает процессор при ожидании захвата и допускает уступки (и прерывания) пока блокировка удерживается.

Xv6 предоставляет такие блокировки в виде блокировок сна. acquiresleep (kernel/sleeplock.c:22) уступает процессор во время ожидания, используя техники, которые будут объяснены в главе 7. На высоком уровне блокировка сна имеет поле locked, защищённое спин-блокировкой, и вызов sleep в acquiresleep атомарно уступает процессор и освобождает спин-блокировку. В результате другие потоки могут выполняться, пока acquiresleep ждёт.

Поскольку блокировки сна оставляют прерывания включёнными, они не могут использоваться в обработчиках прерываний. Поскольку acquiresleep может уступить процессор, блокировки сна нельзя использовать внутри критических секций спин-

блокировок (хотя спин-блокировки можно использовать внутри критических секций блокировок сна).

Спин-блокировки лучше всего подходят для коротких критических секций, поскольку ожидание в них расходует процессорное время; блокировки сна хорошо работают для длительных операций.

6.9 Реальный мир

Программирование с блокировками остаётся сложным, несмотря на многолетние исследования примитивов конкурентности и параллелизма. Часто лучше скрывать блокировки внутри высокоуровневых конструкций, таких как синхронизированные очереди, хотя xv6 этого не делает. Если вы программируете с блокировками, разумно использовать инструмент, который пытается находить гонки, поскольку легко пропустить инвариант, требующий блокировки.

Большинство операционных систем поддерживают потоки POSIX (Pthreads), позволяющие пользовательскому процессу иметь несколько потоков, работающих конкурентно на разных процессорах. Pthreads поддерживает блокировки уровня пользователя, барьеры и т.д. Pthreads также позволяет программисту при желании указать, что блокировка должна быть реентерабельной.

Поддержка Pthreads на уровне пользователя требует поддержки со стороны операционной системы. Например, должно выполняться условие, что если один поток pthread заблокируется в системном вызове, другой поток pthread того же процесса должен иметь возможность выполняться на этом процессоре. В качестве другого примера: если поток pthread изменяет адресное пространство своего процесса (например, отображает или снимает отображение памяти), ядро должно организовать обновление аппаратных таблиц страниц другими процессорами, выполняющими потоки того же процесса, чтобы отразить изменение адресного пространства.

Блокировки можно реализовать без атомарных инструкций [10], но это дорогостоящие, и большинство операционных систем используют атомарные инструкции.

Блокировки могут быть дорогостоящими, если многие процессоры пытаются захватить одну и ту же блокировку одновременно. Если один процессор имеет блокировку в своём локальном кэше, а другому процессору нужно захватить блокировку, то атомарная инструкция обновления строки кэша, содержащей блокировку, должна переместить строку из кэша одного процессора в кэш другого и, возможно, аннулировать все другие копии этой строки кэша. Извлечение строки кэша из кэша другого процессора может быть на несколько порядков дороже, чем извлечение строки из локального кэша.

Чтобы избежать затрат, связанных с блокировками, многие операционные системы используют структуры данных и алгоритмы без блокировок [6, 12]. Например, можно реализовать связный список, подобный тому, что был в начале главы, не требующий блокировок при поиске в списке и одной атомарной инструкции для вставки элемента в список. Однако программирование без блокировок сложнее, чем программирование с блокировками; например, нужно думать о переупорядочивании инструкций и памя-

ти. Программирование с блокировками уже достаточно сложно, поэтому xv6 избегает дополнительной сложности программирования без блокировок.

6.10 Упражнения

1. Закомментируйте вызовы acquire и release в kalloc (kernel/kalloc.c:69). Это должно создать проблемы для кода ядра, вызывающего kalloc; какие симптомы вы ожидаете увидеть? При запуске xv6 видите ли вы эти симптомы? А при запуске usertests? Если проблемы не наблюдаются, почему нет? Попробуйте спровоцировать проблему, вставив фиктивные циклы в критическую секцию kalloc.

2. Предположим, что вместо этого вы закомментировали блокировку в kfree (восстановив блокировку в kalloc). Что теперь может пойти не так? Является ли отсутствие блокировок в kfree менее опасным, чем в kalloc?

3. Если два процессора вызывают kalloc одновременно, один должен ждать другого, что плохо для производительности. Измените kalloc.c для обеспечения большего параллелизма, чтобы одновременные вызовы kalloc с разных процессоров могли выполняться без ожидания друг друга.

4. Напишите параллельную программу с использованием потоков POSIX, которые поддерживаются большинством операционных систем. Например, реализуйте параллельную хеш-таблицу и измерьте, масштабируется ли количество операций put/get с ростом числа ядер.

5. Реализуйте подмножество Pthreads в xv6. То есть реализуйте пользовательскую библиотеку потоков так, чтобы пользовательский процесс мог иметь более одного потока и эти потоки могли выполняться параллельно на разных процессорах. Разработайте дизайн, который корректно обрабатывает блокирующий системный вызов потока и изменение разделяемого адресного пространства.

Глава 7

Планирование

В любой операционной системе процессов, как правило, больше, чем процессоров, поэтому необходим план для разделения процессорного времени между процессами. В идеале это разделение должно быть прозрачным для пользовательских процессов. Распространённый подход состоит в том, чтобы создать для каждого процесса иллюзию собственного виртуального процессора с помощью мультиплексирования процессов на физических процессорах. В этой главе объясняется, как xv6 реализует такое мультиплексирование.

7.1 Мультиплексирование

Xv6 осуществляет мультиплексирование, переключая каждый процессор с одного процесса на другой в двух ситуациях. Во-первых, механизм sleep и wakeup в xv6 выполняет переключение, когда процесс ожидает завершения ввода-вывода устройства или канала, ожидает завершения дочернего процесса или вызывает системный вызов sleep. Во-вторых, xv6 периодически принудительно переключает процессы, чтобы обрабатывать те из них, которые долго вычисляют без перехода в состояние ожидания. Мультиплексирование создаёт иллюзию того, что у каждого процесса есть собственный процессор — примерно так же, как xv6 использует аллокатор памяти и аппаратные таблицы страниц, чтобы создать иллюзию собственной памяти у каждого процесса.

Реализация мультиплексирования сопряжена с рядом трудностей. Первая: как переключаться с одного процесса на другой? Хотя идея переключения контекста проста, реализация представляет собой один из наиболее трудных для понимания фрагментов кода в xv6. Вторая: как принудительно переключать процессы так, чтобы это было прозрачно для пользовательских процессов? Xv6 использует стандартный подход, при котором прерывания аппаратного таймера управляют переключением контекста. Третья: все процессоры переключаются между одним и тем же общим набором процессов, и для предотвращения состояний гонки необходим план блокировок. Четвёртая: когда процесс завершается, его память и другие ресурсы должны быть освобождены, но сам процесс не может сделать всё это самостоятельно — например, он не может освободить

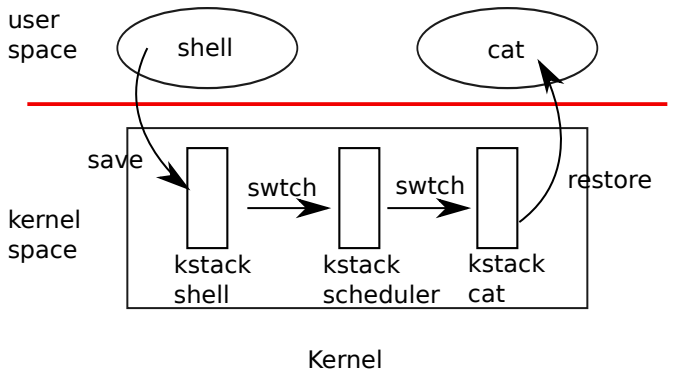

Рис. 7.1: Переключение с одного пользовательского процесса на другой. В этом примере xv6 работает с одним процессором (и, следовательно, одним потоком планировщика).

свой стек ядра, пока продолжает им пользоваться. Пятая: каждое ядро многоядерной машины должно помнить, какой процесс оно выполняет, чтобы системные вызовы влияли на состояние ядра правильного процесса. Наконец, sleep и wakeup позволяют процессу добровольно освободить процессор и ждать пробуждения от другого процесса или прерывания. Необходимо соблюдать осторожность, чтобы избежать состояний гонки, приводящих к потере уведомлений о пробуждении. Xv6 стремится решать эти проблемы как можно проще, однако итоговый код всё равно получается непростым.

7.2 Код: переключение контекста

На рисунке 7.1 схематично показаны шаги переключения с одного пользовательского процесса на другой: переход пользователь–ядро (системный вызов или прерывание) в поток ядра старого процесса, переключение контекста на поток планировщика текущего процессора, переключение контекста на поток ядра нового процесса и возврат из ловушки в пользовательский процесс. Планировщик xv6 имеет выделенный поток (сохранённые регистры и стек) на каждый процессор, поскольку выполнять планировщик на стеке ядра старого процесса небезопасно: какое-либо другое ядро может разбудить этот процесс и запустить его, а использование одного стека на двух разных ядрах привело бы к катастрофе. В этом разделе рассматривается механизм переключения между потоком ядра и потоком планировщика.

Переключение с одного потока на другой включает сохранение регистров процессора старого потока и восстановление ранее сохранённых регистров нового потока; тот факт, что указатель стека и счётчик команд сохраняются и восстанавливаются, означает, что процессор переключит стеки и перейдёт к выполнению другого кода.

Функция swtch выполняет сохранение и восстановление при переключении потока ядра. swtch не знает ничего о потоках напрямую; она просто сохраняет и восстанавливает наборы из 32 регистров RISC-V, называемых контекстами. Когда наступает время процессу освободить процессор, поток ядра этого процесса вызывает swtch, чтобы

сохранить свой контекст и вернуться к контексту планировщика. Каждый контекст содержится в структуре struct context (kernel/proc.h:2), которая, в свою очередь, входит в состав struct proc процесса или struct cpu процессора. swtch принимает два аргумента: struct context *old и struct context *new. Функция сохраняет текущие регистры в old, загружает регистры из new и выполняет возврат.

Проследим работу процесса через вызов swtch вплоть до планировщика. В главе 4 мы видели, что в конце обработки прерывания usertrap может вызвать yield. yield в свою очередь вызывает sched, который вызывает swtch для сохранения текущего контекста в p->context и переключения на контекст планировщика, ранее сохранённый в cpu->context (kernel/proc.c:488).

swtch (kernel/swtch.S:3) сохраняет только callee-сохраняемые регистры; компилятор C генерирует код в вызывающей функции для сохранения caller-сохраняемых регистров на стеке. swtch знает смещение поля каждого регистра в struct context. Счётчик команд при этом не сохраняется. Вместо этого swtch сохраняет регистр ra, который хранит адрес возврата из того места, откуда была вызвана swtch. Затем swtch восстанавливает регистры из нового контекста, в котором хранятся значения регистров, сохранённые предыдущим вызовом swtch. Когда swtch возвращает управление, она возвращается по адресу, указанному в восстановленном регистре ra, то есть в инструкцию, из которой новый поток ранее вызвал swtch. Кроме того, возврат происходит на стек нового потока, поскольку именно туда указывает восстановлённый sp.

В нашем примере sched вызывает swtch для переключения на cpu->context, контекст планировщика данного процессора. Этот контекст был сохранён в тот момент в прошлом, когда scheduler вызвал swtch (kernel/proc.c:449) для переключения на процесс, который сейчас освобождает процессор. Когда отслеживаемый нами вызов swtch возвращает управление, он возвращается не в sched, а в scheduler, причём указатель стека указывает на стек планировщика текущего процессора.

7.3 Код: планирование

В предыдущем разделе рассматривались низкоуровневые детали работы swtch; теперь будем считать swtch заданной функцией и рассмотрим переключение с потока ядра одного процесса через планировщик на другой процесс. Планировщик существует в виде специального потока на каждый процессор; каждый поток выполняет функцию scheduler. Эта функция отвечает за выбор следующего процесса для запуска. Процесс, который хочет освободить процессор, должен захватить собственную блокировку процесса p->lock, освободить все другие удерживаемые блокировки, обновить своё состояние (p->state) и затем вызвать sched. Эту последовательность можно видеть в yield (kernel/proc.c:494), sleep и exit. sched проверяет часть этих условий и затем проверяет следствие из них: если блокировка удерживается, прерывания должны быть отключены. Наконец, sched вызывает swtch для сохранения текущего контекста в p->context и переключения на контекст планировщика в cpu->context. swtch возвращает управление на стек планировщика, как если бы вызов swtch функции scheduler вернул управление

(kernel/proc.c:449). Планировщик продолжает свой цикл for, находит процесс для запуска, переключается на него, и цикл повторяется.

Мы только что видели, что xv6 удерживает p->lock на протяжении вызовов swtch: вызывающая сторона swtch должна уже удерживать блокировку, и управление блокировкой передаётся получившему управление коду. Это соглашение необычно для блокировок; обычно поток, захвативший блокировку, несёт ответственность и за её освобождение, что упрощает рассуждение о корректности. При переключении контекста необходимо нарушить это соглашение, поскольку p->lock защищает инварианты полей state и context процесса, которые не выполняются в процессе выполнения swtch. Пример проблемы, которая могла бы возникнуть, если бы p->lock не удерживалась во время swtch: другой процессор мог бы принять решение о запуске процесса после того, как yield установил его состояние в RUNNABLE, но до того, как swtch вынудил процесс прекратить использование его собственного стека ядра. Результатом было бы выполнение двух процессоров на одном стеке, что привело бы к хаосу.

Единственное место, где поток ядра освобождает процессор, —это sched, и он всегда переключается на одно и то же место в scheduler, который (почти) всегда переключается на какой-либо поток ядра, ранее вызвавший sched. Таким образом, если бы мы выводили номера строк, в которых xv6 переключает потоки, мы наблюдали бы следующий простой паттерн: (kernel/proc.c:449), (kernel/proc.c:488), (kernel/proc.c:449), (kernel/proc.c:488) и так далее. Процедуры, намеренно передающие управление друг другу через переключение потоков, иногда называют сопрограммами; в данном примере sched и scheduler являются сопрограммами по отношению друг к другу.

Есть один случай, когда вызов swtch в планировщике не заканчивается в sched. allocproc устанавливает регистр ra контекста нового процесса на forkret (kernel/proc.c:506), так что его первый вызов swtch «возвращается» в начало этой функции. forkret существует для освобождения p->lock; в остальном, поскольку новый процесс должен вернуться в пользовательское пространство, как при возврате из fork, он мог бы вместо этого начинать с usertrapret.

scheduler (kernel/proc.c:425) выполняет цикл: найти процесс для запуска, запустить его до тех пор, пока он не уступит процессор, повторить. Планировщик просматривает таблицу процессов в поисках готового к выполнению процесса с p->state == RUNNABLE. Найдя такой процесс, он устанавливает переменную текущего процесса данного процессора c->proc, помечает процесс как RUNNING и вызывает swtch для запуска процесса (kernel/proc.c:444-449).

Один из способов осмыслить структуру кода планирования состоит в том, что она поддерживает набор инвариантов для каждого процесса и удерживает p->lock всякий раз, когда эти инварианты не выполняются. Один инвариант: если процесс находится в состоянии RUNNING, yield при прерывании таймера должен иметь возможность безопасно переключиться с этого процесса; это означает, что регистры процессора должны хранить значения регистров процесса (то есть swtch ещё не перенёс их в context), и c->proc должен указывать на процесс. Другой инвариант: если процесс находится в состоянии RUNNABLE, простаивающий планировщик scheduler должен иметь возмож-

ность безопасно запустить его; это означает, что p->context должен хранить регистры процесса (то есть они на самом деле не находятся в реальных регистрах), что ни один процессор не выполняется на стеке ядра процесса, и что c->proc ни одного процессора не ссылается на данный процесс. Заметим, что эти свойства часто не выполняются, когда удерживается p->lock.

Поддержание вышеуказанных инвариантов объясняет, почему xv6 часто захватывает p->lock в одном потоке и освобождает в другом — например, захватывает в yield и освобождает в scheduler. Как только yield начинает изменять состояние выполняющегося процесса, переводя его в RUNNABLE, блокировка должна оставаться захваченной до восстановления инвариантов: самый ранний корректный момент освобождения — после того, как scheduler (выполняющийся на своём стеке) очистит c->proc. Аналогично, как только scheduler начинает переводить процесс из состояния RUNNABLE в RUNNING, блокировка не может быть освобождена, пока поток ядра не начнёт полноценно выполняться (после swtch, например, в yield).

7.4 Код: mycpu и myproc

Xv6 часто нуждается в указателе на структуру proc текущего процесса. На однопроцессорной машине можно было бы использовать глобальную переменную, указывающую на текущий proc. На многоядерной машине это не работает, поскольку каждое ядро выполняет разные процессы. Решение состоит в том, чтобы воспользоваться тем, что у каждого ядра есть собственный набор регистров; один из этих регистров можно использовать для хранения информации, специфичной для данного ядра.

Xv6 поддерживает структуру struct cpu для каждого процессора (kernel/proc.h:22), в которой записывается процесс, выполняющийся на данном процессоре (если таковой имеется), сохранённые регистры потока планировщика процессора и счётчик вложенных спинлоков, необходимый для управления отключением прерываний. Функция mycpu (kernel/proc.c:74) возвращает указатель на структуру struct cpu текущего процессора. RISC-V нумерует свои процессоры, присваивая каждому hartid. Xv6 обеспечивает хранение hartid каждого процессора в его регистре tp в режиме ядра. Это позволяет mycpu использовать tp в качестве индекса в массиве структур cpu для нахождения нужной записи.

Обеспечение того, чтобы регистр tp процессора всегда хранил его hartid, требует определённых усилий. start устанавливает регистр tp на раннем этапе загрузки процессора, ещё в машинном режиме (kernel/start.c:45). usertrapret сохраняет tp на странице трамплина, поскольку пользовательский процесс может изменить tp. Наконец, uservec восстанавливает сохранённый tp при входе в ядро из пользовательского пространства (kernel/trampoline.S:78). Компилятор гарантирует, что он никогда не использует регистр tp. Было бы удобнее, если бы xv6 мог запрашивать текущий hartid у аппаратуры RISC-V по мере необходимости, но RISC-V допускает это только в машинном режиме, но не в режиме супервизора.

Возвращаемые значения функций cpuid и mycpu нестабильны: если таймер прервёт

выполнение и поток уступит процессор, а затем переместится на другой процессор, ранее полученное значение станет неверным. Чтобы избежать этой проблемы, xv6 требует, чтобы вызывающие отключали прерывания и включали их снова лишь после того, как закончат использовать возвращённую структуру struct cpu.

Функция myproc (kernel/proc.c:83) возвращает указатель на struct proc процесса, выполняющегося на текущем процессоре. myproc отключает прерывания, вызывает mycpu, получает указатель на текущий процесс (c->proc) из структуры struct cpu и затем включает прерывания. Возвращаемое значение myproc безопасно использовать даже при включённых прерываниях: если прерывание таймера переместит вызывающий процесс на другой процессор, его указатель struct proc останется тем же.

7.5 Sleep и wakeup

Планирование и блокировки помогают скрыть действия одного потока от другого, но нам также нужны абстракции, позволяющие потокам намеренно взаимодействовать. Например, читатель канала в xv6 может ожидать, пока записывающий процесс произведёт данные; родительский вызов wait может ожидать завершения дочернего процесса; а процесс, читающий с диска, должен ждать, пока дисковое оборудование завершит чтение. Ядро xv6 использует механизм sleep и wakeup в этих ситуациях (и во многих других). Sleep позволяет потоку ядра ждать определённого события; другой поток может вызвать wakeup, чтобы сообщить, что потоки, ждущие этого события, должны возобновить работу. Sleep и wakeup часто называют механизмами координации последовательности или условной синхронизации.

Sleep и wakeup предоставляют относительно низкоуровневый интерфейс синхронизации. Чтобы мотивировать их устройство в xv6, воспользуемся ими для построения более высокоуровневого механизма синхронизации — семафора [5], координирующего производителей и потребителей (xv6 не использует семафоры). Семафор поддерживает счётчик и предоставляет две операции. Операция «V» (для производителя) увеличивает счётчик. Операция «P» (для потребителя) ожидает, пока счётчик не станет ненулевым, затем уменьшает его и возвращает управление. Если бы существовали только один поток-производитель и один поток-потребитель, выполняющиеся на разных процессорах, и компилятор не выполнял слишком агрессивных оптимизаций, следующая реализация была бы корректной:

```
struct semaphore {
  struct spinlock lock;
  int count;
};

void
V(struct semaphore *s)
{
    acquire(&s->lock);
```

```
109         s->count += 1;
110         release(&s->lock);
111     }
112
113     void
114     P(struct semaphore *s)
115     {
116         while(s->count == 0)
117             ;
118         acquire(&s->lock);
119         s->count -= 1;
120         release(&s->lock);
121     }
```

Приведённая выше реализация неэффективна. Если производитель действует редко, потребитель будет проводить большую часть времени в цикле while, ожидая ненулевого значения счётчика. Процессор потребителя, вероятно, мог бы выполнять более полезную работу, чем активное ожидание с помощью многократного опроса s->count. Для избежания активного ожидания необходим способ, позволяющий потребителю уступить процессор и возобновить работу только после того, как V увеличит счётчик.

Вот шаг в этом направлении, хотя, как мы увидим, его недостаточно. Представим пару вызовов, sleep и wakeup, работающих следующим образом. sleep(chan) переводит вызывающий процесс в сон на произвольном значении chan, называемом каналом ожидания. sleep переводит вызывающий процесс в состояние сна, освобождая процессор для другой работы. wakeup(chan) пробуждает все процессы, спящие на chan (если таковые имеются), заставляя их вызовы sleep вернуть управление. Если ни один процесс не ожидает на chan, wakeup ничего не делает. Реализацию семафора можно изменить, использовав sleep и wakeup (изменения выделены жёлтым):

```
200     void
201     V(struct semaphore *s)
202     {
203         acquire(&s->lock);
204         s->count += 1;
205         wakeup(s);
206         release(&s->lock);
207     }
208
209     void
210     P(struct semaphore *s)
211     {
212         while(s->count == 0)
213             sleep(s);
214         acquire(&s->lock);
215         s->count -= 1;
216         release(&s->lock);
```

```
217     }
```

P теперь уступает процессор вместо того, чтобы крутиться в цикле, что хорошо. Однако выясняется, что спроектировать sleep и wakeup с таким интерфейсом, не страдая от так называемой проблемы потерянного пробуждения, непросто. Предположим, что P обнаруживает, что s->count == 0 на строке 212. Пока P находится между строками 212 и 213, V выполняется на другом процессоре: он изменяет s->count на ненулевое и вызывает wakeup, который не находит спящих процессов и поэтому ничего не делает. Теперь P продолжает выполнение со строки 213: он вызывает sleep и засыпает. Это создаёт проблему: P спит в ожидании вызова V, который уже произошёл. Если нам не повезёт и производитель снова вызовет V, потребитель будет ждать вечно, хотя счётчик ненулевой.

Корень этой проблемы в том, что инвариант о том, что P засыпает только тогда, когда s->count == 0, нарушается выполнением V в самый неподходящий момент. Некорректным способом защиты инварианта было бы перенести захват блокировки (выделено жёлтым ниже) в P так, чтобы проверка счётчика и вызов sleep стали атомарными:

```
300     void
301     V(struct semaphore *s)
302     {
303       acquire(&s->lock);
304       s->count += 1;
305       wakeup(s);
306       release(&s->lock);
307     }
308
309     void
310     P(struct semaphore *s)
311     {
312       acquire(&s->lock);
313       while(s->count == 0)
314         sleep(s);
315       s->count -= 1;
316       release(&s->lock);
317     }
```

Можно было бы надеяться, что эта версия P позволит избежать потерянного пробуждения, поскольку блокировка не даёт V выполняться между строками 313 и 314. Это действительно так, но возникает взаимная блокировка: P удерживает блокировку, пока спит, поэтому V будет ждать блокировку вечно.

Мы исправим предыдущую схему, изменив интерфейс sleep: вызывающая сторона должна передать блокировку условия в sleep, чтобы та могла освободить её после того, как вызывающий процесс будет помечен как спящий и ожидающий на канале сна. Блокировка вынудит параллельно выполняющийся V ждать, пока P полностью перейдёт в состояние сна, чтобы wakeup нашёл спящего потребителя и разбудил его. Как только потребитель проснётся, sleep повторно захватит блокировку перед возвратом. Новая

корректная схема sleep/wakeup используется следующим образом (изменение выделено жёлтым):

```
void
V(struct semaphore *s)
{
  acquire(&s->lock);
  s->count += 1;
  wakeup(s);
  release(&s->lock);
}

void
P(struct semaphore *s)
{
  acquire(&s->lock);
  while(s->count == 0)
      sleep(s, &s->lock);
  s->count -= 1;
  release(&s->lock);
}
```

Тот факт, что P удерживает s->lock, не даёт V попытаться разбудить его между проверкой s->count функцией P и вызовом sleep. Однако заметим, что нам нужно, чтобы sleep атомарно освобождала s->lock и переводила потребляющий процесс в сон, во избежание потерянных пробуждений.

7.6 Код: Sleep и wakeup

Функции sleep (kernel/proc.c:543) и wakeup (kernel/proc.c:574) в xv6 предоставляют интерфейс, показанный в последнем примере выше, а их реализация (плюс правила использования) гарантирует отсутствие потерянных пробуждений. Основная идея состоит в том, чтобы sleep помечала текущий процесс как SLEEPING и затем вызывала sched для освобождения процессора; wakeup ищет процесс, спящий на указанном канале ожидания, и помечает его как RUNNABLE. Вызывающие функции sleep и wakeup могут использовать любое удобное число в качестве канала. Xv6 часто использует адрес структуры данных ядра, участвующей в ожидании.

sleep захватывает p->lock (kernel/proc.c:554). Теперь засыпающий процесс удерживает и p->lock, и lk. Удержание lk было необходимо у вызывающей стороны (в примере это P): оно гарантировало, что никакой другой процесс (в примере это процесс, выполняющий V) не мог начать вызов wakeup(chan). Теперь, когда sleep удерживает p->lock, безопасно освобождать lk: другой процесс может начать вызов wakeup(chan), но wakeup будет ждать захвата p->lock и, следовательно, будет ждать, пока sleep не завершит перевод процесса в состояние сна, не давая wakeup пропустить sleep.

Теперь, когда sleep удерживает p->lock и никакие другие блокировки, она может перевести процесс в состояние сна, записав канал сна, изменив состояние процесса на SLEEPING и вызвав sched (kernel/proc.c:558-561). Чуть позже станет понятно, почему критически важно, чтобы p->lock не освобождалась (функцией scheduler) до тех пор, пока процесс не будет помечен как SLEEPING.

В какой-то момент другой процесс захватит блокировку условия, установит условие, которого ждёт спящий процесс, и вызовет wakeup(chan). Важно, чтобы wakeup вызывалась при удержании блокировки условия[1]. wakeup обходит таблицу процессов (kernel/proc.c:574). Она захватывает p->lock каждого проверяемого процесса, как потому, что может изменить его состояние, так и потому, что p->lock гарантирует, что sleep и wakeup не пропустят друг друга. Когда wakeup находит процесс в состоянии SLEEPING с совпадающим chan, она изменяет его состояние на RUNNABLE. При следующем запуске планировщик увидит, что процесс готов к выполнению.

Почему правила блокировки для sleep и wakeup гарантируют, что спящий процесс не пропустит пробуждение? Спящий процесс удерживает либо блокировку условия, либо собственную p->lock, либо обе — с момента проверки условия до момента, когда он будет помечен как SLEEPING. Процесс, вызывающий wakeup, удерживает обе эти блокировки в цикле wakeup. Следовательно, пробудитель либо делает условие истинным до того, как потребляющий поток проверит его; либо wakeup пробудителя проверяет спящий поток строго после того, как он был помечен как SLEEPING. Тогда wakeup увидит спящий процесс и разбудит его (если только что-то другое не разбудит его раньше).

Иногда несколько процессов спят на одном канале; например, более одного процесса читают из канала. Один вызов wakeup разбудит их всех. Один из них выполнится первым, захватит блокировку, с которой был вызван sleep, и (в случае каналов) прочитает данные, ожидающие в канале. Остальные процессы обнаружат, что несмотря на пробуждение, данных для чтения нет. С их точки зрения пробуждение было «ложным», и им придётся снова заснуть. По этой причине sleep всегда вызывается внутри цикла, который повторно проверяет условие.

Никакого вреда нет, если два использования sleep/wakeup случайно выбирают один канал: они увидят ложные пробуждения, но цикл, описанный выше, справится с этой проблемой. Привлекательность sleep/wakeup во многом объясняется тем, что они одновременно легковесны (не нужно создавать специальные структуры данных в качестве каналов сна) и обеспечивают уровень косвенности (вызывающим не нужно знать, с каким конкретным процессом они взаимодействуют).

7.7 Код: Каналы

Более сложный пример использования sleep и wakeup для синхронизации производителей и потребителей — реализация каналов в xv6. Мы видели интерфейс каналов в главе 1: байты, записанные в один конец канала, копируются во внутрибуферную

[1] Строго говоря, достаточно, если wakeup просто следует за acquire (то есть можно вызвать wakeup после release).

область ядра, откуда их можно прочитать с другого конца канала. В последующих главах будет рассматриваться поддержка дескрипторов файлов для каналов, а сейчас обратимся к реализациям pipewrite и piperead.

Каждый канал представлен структурой struct pipe, которая содержит lock и буфер data. Поля nread и nwrite считают суммарное количество байт, прочитанных из и записанных в буфер. Буфер кольцевой: следующий байт после buf[PIPESIZE-1] записывается в buf[0]. Счётчики не зацикливаются. Это соглашение позволяет реализации различать полный буфер (nwrite == nread+PIPESIZE) от пустого (nwrite == nread), однако означает, что при индексировании в буфер нужно использовать buf[nread % PIPESIZE] вместо просто buf[nread] (аналогично для nwrite).

Предположим, что вызовы piperead и pipewrite происходят одновременно на двух разных процессорах. pipewrite (kernel/pipe.c:77) начинает с захвата блокировки канала, которая защищает счётчики, данные и связанные с ними инварианты. piperead (kernel/pipe.c:106) затем пытается захватить блокировку, но не может. Она вращается в acquire (kernel/spinlock.c:22) в ожидании блокировки. Пока piperead ждёт, pipewrite обходит записываемые байты (addr[0..n-1]), добавляя каждый в канал по очереди (kernel/pipe.c:95). В ходе этого цикла может произойти заполнение буфера (kernel/pipe.c:88). В этом случае pipewrite вызывает wakeup, чтобы оповестить спящих читателей о том, что в буфере есть данные, и затем засыпает на &pi->nwrite, ожидая, пока читатель не извлечёт байты из буфера. sleep освобождает pi->lock как часть перевода процесса pipewrite в состояние сна.

Теперь, когда pi->lock доступна, piperead захватывает её и входит в критическую секцию: она обнаруживает, что pi->nread != pi->nwrite (kernel/pipe.c:113) (pipewrite уснула, потому что pi->nwrite == pi->nread+PIPESIZE (kernel/pipe.c:88)), поэтому переходит к циклу for, копирует данные из канала (kernel/pipe.c:120) и увеличивает nread на количество скопированных байт. Теперь столько байт доступно для записи, и поэтому piperead вызывает wakeup (kernel/pipe.c:131) для пробуждения спящих записывающих процессов перед возвратом. wakeup находит процесс, спящий на &pi->nwrite, —процесс, выполнявший pipewrite, но остановившийся при заполнении буфера. Он помечает этот процесс как RUNNABLE.

Код канала использует отдельные каналы сна для читателей и записывающих (pi->nread и pi->nwrite); это может сделать систему более эффективной в маловероятном случае, когда на одном канале ожидает много читателей и записывающих одновременно. Код канала засыпает внутри цикла, проверяющего условие сна; если есть несколько читателей или записывающих, все процессы, кроме первого проснувшегося, увидят, что условие по-прежнему ложно, и снова заснут.

7.8 Код: Wait, exit и kill

sleep и wakeup можно использовать для многих видов ожидания. Интересный пример, представленный в главе 1, —взаимодействие между exit дочернего процесса и wait родительского. Ко времени гибели дочернего процесса родитель может уже спать в

wait или заниматься чем-то другим; в последнем случае последующий вызов wait должен зафиксировать гибель дочернего процесса, возможно, долго спустя после вызова exit. Способ, которым xv6 сохраняет сведения о гибели дочернего процесса до тех пор, пока wait не заметит её, состоит в том, что exit переводит вызывающего в состояние ZOMBIE, где он остаётся, пока wait родителя не заметит его, не изменит состояние дочернего процесса на UNUSED, не скопирует статус завершения дочернего процесса и не вернёт идентификатор дочернего процесса родителю. Если родитель завершается раньше дочернего, он передаёт дочерний процесс процессу init, который постоянно вызывает wait; таким образом, у каждого дочернего процесса есть родитель, который его «уберёт». Сложность состоит в том, чтобы избежать состояний гонки и взаимных блокировок между одновременными вызовами wait и exit родителя и дочернего, а также между одновременными вызовами exit и exit.

wait начинается с захвата wait_lock Причина в том, что wait_lock служит блокировкой условия, помогающей гарантировать, что родитель не пропустит wakeup от завершающегося дочернего процесса. Затем wait просматривает таблицу процессов. Если она находит дочерний процесс в состоянии ZOMBIE, она освобождает ресурсы этого дочернего процесса и его структуру proc, копирует статус завершения дочернего по адресу, переданному в wait (если он не равен 0), и возвращает идентификатор дочернего процесса. Если wait находит дочерние процессы, но ни один из них не завершился, она вызывает sleep в ожидании завершения любого из них (kernel/proc.c:413) и затем снова просматривает таблицу. wait часто удерживает две блокировки: wait_lock и pp->lock какого-либо процесса; порядок их захвата для предотвращения взаимной блокировки: сначала wait_lock, затем pp->lock.

освобождает часть ресурсов, вызывает reparent для передачи своих дочерних процессов процессу init, будит родителя на случай, если тот находится в wait, помечает вызывающего как зомби и навсегда уступает процессор. exit удерживает и wait_lock, и p->lock на протяжении этой последовательности. Она удерживает wait_lock, поскольку это блокировка условия для wakeup(p->parent), предотвращающая потерю пробуждения у родителя в wait. exit также должна удерживать p->lock в этой последовательности, чтобы не дать родителю в wait увидеть, что дочерний процесс находится в состоянии ZOMBIE до того, как дочерний процесс окончательно вызовет swtch. exit захватывает эти блокировки в том же порядке, что и wait, во избежание взаимной блокировки.

Может показаться некорректным, что exit будит родителя до того, как установит своё состояние в ZOMBIE, но это безопасно: хотя wakeup может заставить родителя выполняться, цикл в wait не может проверить дочерний процесс, пока p->lock дочернего не будет освобождена функцией scheduler, поэтому wait не может изучить завершающийся процесс до тех пор, пока exit не установит его состояние в ZOMBIE (kernel/proc.c:359).

Тогда как exit позволяет процессу завершить себя, kill (kernel/proc.c:622) позволяет одному процессу потребовать завершения другого. Было бы слишком сложно для kill напрямую уничтожить процесс-жертву, поскольку жертва может выполняться на другом процессоре, возможно в середине чувствительной последовательности обновлений структур данных ядра. Поэтому kill делает очень мало: он просто устанавливает

p->killed у жертвы и, если та спит, будит её. В конечном счёте жертва войдёт в ядро или покинет его, и тогда код в usertrap вызовет exit, если установлен флаг p->killed (это проверяется вызовом killed (kernel/proc.c:622)). Если жертва выполняется в пользовательском пространстве, она вскоре войдёт в ядро, сделав системный вызов или из-за того, что таймер (или другое устройство) вызвал прерывание.

Если процесс-жертва находится в состоянии sleep, вызов wakeup в kill заставит жертву вернуться из sleep. Это потенциально опасно, поскольку ожидаемое условие может оказаться ложным. Однако в xv6 все вызовы sleep обёрнуты в цикл while, повторно проверяющий условие после возврата sleep. Некоторые вызовы sleep также проверяют p->killed в цикле и прерывают текущую операцию, если флаг установлен. Это делается только тогда, когда такое прерывание корректно. Например, код чтения и записи канала возвращает управление, если установлен флаг killed; в конечном счёте код вернётся к ловушке, которая снова проверит p->killed и завершит процесс.

Некоторые циклы sleep в xv6 не проверяют p->killed, поскольку код находится в середине многошагового системного вызова, который должен быть атомарным. Пример —драйвер virtio (kernel/virtio_disk.c:285): он не проверяет p->killed, поскольку дисковая операция может быть частью набора записей, все из которых необходимы для оставления файловой системы в корректном состоянии. Процесс, которому поступила команда kill во время ожидания дискового ввода-вывода, не завершится, пока не завершит текущий системный вызов и usertrap не увидит флаг killed.

7.9 Блокировка процесса

Блокировка, связанная с каждым процессом (p->lock), —самая сложная блокировка в xv6. Простой способ думать о p->lock состоит в том, что она должна удерживаться при чтении или записи любого из следующих полей struct proc: p->state, p->chan, p->killed, p->xstate и p->pid. Эти поля могут использоваться другими процессами или потоками планировщика на других ядрах, поэтому естественно, что они должны быть защищены блокировкой.

Однако большинство применений p->lock защищают более высокоуровневые аспекты структур данных и алгоритмов процессов xv6. Вот полный перечень того, что делает p->lock:

- Совместно с p->state предотвращает гонки при выделении слотов proc[] для новых процессов.
- Скрывает процесс от посторонних взглядов, пока он создаётся или уничтожается.
- Не даёт функции wait родителя собрать процесс, который установил своё состояние в ZOMBIE, но ещё не уступил процессор.
- Не позволяет планировщику другого ядра принять решение о запуске уступающего процесса после того, как он установил своё состояние в RUNNABLE, но до завершения swtch.

- Гарантирует, что только планировщик одного ядра принимает решение о запуске процесса в состоянии RUNNABLE.
- Не даёт прерыванию таймера заставить процесс уступить процессор, пока он находится в swtch.
- Совместно с блокировкой условия помогает предотвратить пропуск wakeup процесса, вызывающего sleep, но ещё не завершившего уступку процессора.
- Не позволяет процессу-жертве kill завершиться и, возможно, быть повторно выделенным между проверкой kill p->pid и установкой p->killed.
- Делает проверку и запись p->state в kill атомарными.

Поле p->parent защищено глобальной блокировкой wait_lock, а не p->lock. Только родитель процесса изменяет p->parent, хотя это поле читается как самим процессом, так и другими процессами, ищущими своих дочерних. Назначение wait_lock — служить блокировкой условия, когда wait спит в ожидании завершения любого дочернего процесса. Завершающийся дочерний процесс удерживает либо wait_lock, либо p->lock до тех пор, пока не установит своё состояние в ZOMBIE, не разбудит родителя и не уступит процессор. wait_lock также сериализует одновременные вызовы exit родителем и дочерним, гарантируя тем самым, что процесс init (который наследует дочерний) будет гарантированно разбужен от своего wait. wait_lock является глобальной, а не перпроцессной блокировкой в каждом родителе, потому что до захвата этой блокировки процесс не может знать, кто является его родителем.

7.10 Реальный мир

Планировщик xv6 реализует простую политику планирования — запуск каждого процесса по очереди. Эта политика называется круговой очерёдностью. Реальные операционные системы реализуют более сложные политики, позволяющие, например, назначать процессам приоритеты. Идея состоит в том, что готовый к выполнению процесс с высоким приоритетом предпочтительнее для планировщика, чем готовый процесс с низким приоритетом. Такие политики могут быстро усложниться, поскольку нередко возникают конкурирующие цели: например, операционная система может также стремиться гарантировать справедливость и высокую пропускную способность. Кроме того, сложные политики могут приводить к нежелательным взаимодействиям, таким как инверсия приоритетов и конвои. Инверсия приоритетов может произойти, когда процессы с низким и высоким приоритетом используют одну и ту же блокировку: если её захватывает процесс с низким приоритетом, он может блокировать продвижение процесса с высоким приоритетом. Когда многие процессы с высоким приоритетом ожидают процесс с низким приоритетом, захвативший общую блокировку, может образоваться длинный конвой ожидающих процессов; образовавшись, он способен сохраняться долгое

время. Для предотвращения подобных проблем в сложных планировщиках необходимы дополнительные механизмы.

sleep и wakeup — простой и эффективный метод синхронизации, хотя существуют и другие подходы. Первая трудность во всех них состоит в том, чтобы избежать проблемы «потерянного пробуждения», рассмотренной в начале главы. Исходное ядро Unix реализовывало sleep, просто отключая прерывания, что было достаточно, поскольку Unix работал на однопроцессорной системе. Поскольку xv6 работает на многопроцессорных системах, он добавляет явную блокировку в sleep. msleep FreeBSD использует тот же подход. sleep в Plan 9 использует функцию обратного вызова, которая выполняется при удержании блокировки планировщика непосредственно перед переходом в сон; эта функция служит проверкой условия сна в последний момент для предотвращения потерянных пробуждений. sleep ядра Linux использует явную очередь процессов, называемую очередью ожидания, вместо канала ожидания; у очереди есть собственная внутренняя блокировка.

Обход всего набора процессов в wakeup неэффективен. Лучшим решением было бы заменить chan в sleep и wakeup на структуру данных, хранящую список процессов, спящих на этой структуре, например очередь ожидания Linux. В Plan 9 sleep и wakeup называют эту структуру точкой рандеву. Многие библиотеки потоков называют ту же структуру переменной условия; в этом контексте операции sleep и wakeup называются wait и signal. Все эти механизмы разделяют одну общую черту: условие сна защищено некоторой блокировкой, атомарно снимаемой во время сна.

Реализация wakeup будит все процессы, ожидающие на конкретном канале, и может оказаться, что таких процессов много. Операционная система планирует выполнение всех этих процессов, и они наперегонки проверяют условие сна. Процессы, ведущие себя таким образом, иногда называют стадом громов, и его лучше избегать. Большинство переменных условия имеют два примитива для wakeup: signal, который будит один процесс, и broadcast, который будит все ожидающие процессы.

Семафоры часто используются для синхронизации. Счётчик обычно соответствует чему-либо вроде количества доступных байт в буфере канала или числа дочерних зомби, имеющихся у процесса. Использование явного счётчика как части абстракции позволяет избежать проблемы «потерянного пробуждения»: существует явный счётчик числа произошедших пробуждений. Счётчик также позволяет избежать проблем ложного пробуждения и «стада громов».

Завершение процессов и их очистка вносят значительную сложность в xv6. В большинстве операционных систем это ещё сложнее, поскольку, например, процесс-жертва может находиться глубоко в ядре в состоянии сна, и раскрутка его стека требует осторожности, так как каждая функция в стеке вызовов может нуждаться в выполнении некоторой очистки. Некоторые языки облегчают это с помощью механизма исключений, но не C. Кроме того, существуют другие события, которые могут разбудить спящий процесс, даже если ожидаемое им событие ещё не произошло. Например, когда Unix-процесс спит, другой процесс может отправить ему сигнал. В этом случае процесс вернётся из прерванного системного вызова со значением -1 и кодом ошибки EINTR.

Приложение может проверить эти значения и решить, что делать. Xv6 не поддерживает сигналы, и эта сложность не возникает.

Поддержка kill в xv6 не является полностью удовлетворительной: есть циклы sleep, которые, вероятно, должны проверять p->killed. Связанная проблема состоит в том, что даже для циклов sleep, проверяющих p->killed, существует гонка между sleep и kill; последний может установить p->killed и попытаться разбудить жертву сразу после того, как цикл жертвы проверил p->killed, но до того, как она вызвала sleep. Если это произойдёт, жертва не заметит p->killed до тех пор, пока не выполнится условие, которого она ожидает. Это может произойти значительно позже или даже никогда (например, если жертва ожидает ввода с консоли, но пользователь ничего не вводит).

Реальная операционная система находила бы свободные структуры proc с помощью явного свободного списка за постоянное время вместо линейного поиска в allocproc; xv6 использует линейный поиск ради простоты.

7.11 Упражнения

1. Sleep должна проверять lk != &p->lock во избежание взаимной блокировки Предположим, что особый случай был устранён путём замены

   ```
   if(lk != &p->lock){
     acquire(&p->lock);
     release(lk);
   }
   ```

   на

   ```
   release(lk);
   acquire(&p->lock);
   ```

   Это сломало бы sleep. Каким образом?

2. Реализуйте семафоры в xv6, не используя sleep и wakeup (но спинлоки использовать допускается). Замените использования sleep и wakeup в xv6 семафорами. Оцените результат.

3. Исправьте описанную выше гонку между kill и sleep, так чтобы вызов kill, происходящий после того, как цикл sleep жертвы проверил p->killed, но до того, как она вызвала sleep, приводил к тому, что жертва прерывала текущий системный вызов.

4. Разработайте план, чтобы каждый цикл sleep проверял p->killed, так что, например, процесс в драйвере virtio мог бы быстро выйти из цикла while, если он был убит другим процессом.

5. Измените xv6 так, чтобы при переключении с потока ядра одного процесса на другой использовалось только одно переключение контекста, а не переключение

через поток планировщика. Уступающему потоку нужно будет самому выбрать следующий поток и вызвать swtch. Сложности будут связаны с предотвращением случайного выполнения одного потока несколькими ядрами, корректной реализацией блокировок и недопущением взаимных блокировок.

6. Измените scheduler в xv6 так, чтобы использовать инструкцию RISC-V WFI (ожидание прерывания) при отсутствии готовых к выполнению процессов. Постарайтесь гарантировать, что в любое время, когда есть готовые к выполнению процессы, ни одно ядро не приостанавливается в WFI.

Глава 8

Файловая система

Назначение файловой системы — организация и хранение данных. Файловые системы обычно обеспечивают совместный доступ к данным для пользователей и приложений, а также persistence чтобы данные оставались доступными после перезагрузки.

Файловая система xv6 предоставляет Unix-подобные файлы, каталоги и пути (см. главу 1) и хранит свои данные на virtio-диске для обеспечения персистентности. Файловая система решает несколько задач:

- Файловой системе необходимы структуры данных на диске для представления дерева именованных каталогов и файлов, для записи идентификаторов блоков, содержащих данные каждого файла, а также для отслеживания свободных областей диска.

- Файловая система должна поддерживать crash recovery. То есть, если произошёл сбой (например, отключение питания), файловая система должна продолжать корректно работать после перезапуска. Риск заключается в том, что сбой может прервать последовательность обновлений и оставить несогласованные структуры данных на диске (например, блок, который одновременно используется файлом и помечен как свободный).

- Различные процессы могут одновременно работать с файловой системой, поэтому код файловой системы должен координировать доступ для поддержания инвариантов.

- Обращение к диску на несколько порядков медленнее, чем обращение к памяти, поэтому файловая система должна поддерживать кэш в оперативной памяти для часто используемых блоков.

В остальной части этой главы объясняется, как xv6 решает эти задачи.

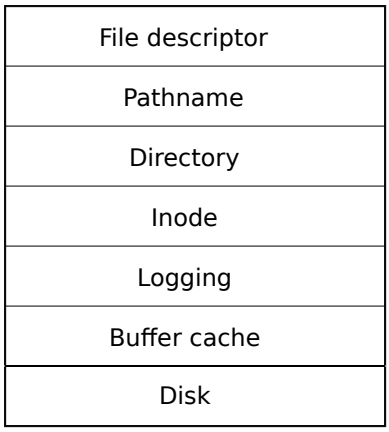

Рис. 8.1: Уровни файловой системы xv6.

8.1 Обзор

Реализация файловой системы xv6 организована в семь уровней, показанных на Figure 8.1. Уровень диска читает и записывает блоки на virtio-жёстком диске. Уровень кэша буферов кэширует дисковые блоки и синхронизирует доступ к ним, гарантируя, что только один процесс ядра одновременно может изменять данные, хранящиеся в любом конкретном блоке. Уровень журналирования позволяет вышестоящим уровням объединять обновления нескольких блоков в transaction, и гарантирует атомарное обновление блоков при сбоях (то есть обновляются либо все блоки, либо ни один). Уровень inode предоставляет отдельные файлы, каждый из которых представлен inode с уникальным i-номером и несколькими блоками, хранящими данные файла. Уровень каталогов реализует каждый каталог как особый вид inode, содержимым которого является последовательность записей каталога, каждая из которых содержит имя файла и i-номер. Уровень путей предоставляет иерархические пути вида /usr/rtm/xv6/fs.c, и разрешает их с помощью рекурсивного поиска. Уровень файловых дескрипторов абстрагирует множество Unix-ресурсов (например, каналы, устройства, файлы и т.д.) через интерфейс файловой системы, упрощая работу программистов прикладных приложений.

Дисковое оборудование традиционно представляет данные на диске как пронумерованную последовательность 512-байтных blocks (также называемых sectors): сектор 0 — это первые 512 байт, сектор 1 — следующие и так далее. Размер блока, используемый операционной системой для файловой системы, может отличаться от размера сектора, используемого диском, но, как правило, размер блока кратен размеру сектора. Xv6 хранит копии блоков, считанных в память, в объектах типа struct buf (kernel/buf.h:1). Данные, хранящиеся в этой структуре, иногда рассинхронизированы с диском: они могут ещё не быть считаны с диска (диск работает над этим, но ещё не вернул содержимое сектора), или могут быть обновлены программно, но ещё не записаны на диск.

Файловая система должна иметь план размещения inode и блоков с содержимым

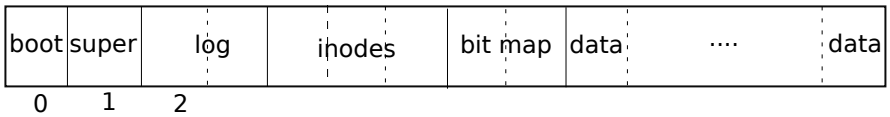

Рис. 8.2: Структура файловой системы xv6.

на диске. Для этого xv6 делит диск на несколько секций, как показано на Figure 8.2. Файловая система не использует блок 0 (в нём хранится загрузочный сектор). Блок 1 называется superblock; он содержит метаданные о файловой системе (размер файловой системы в блоках, количество блоков данных, количество inode и количество блоков в журнале). Блоки начиная с 2 хранят журнал. За журналом расположены inode, по несколько inode в блоке. За ними следуют блоки битовой карты, отслеживающие, какие блоки данных используются. Оставшиеся блоки — блоки данных; каждый из них либо помечен свободным в блоке битовой карты, либо содержит данные файла или каталога. Суперблок заполняется отдельной программой под названием mkfs, которая создаёт начальную файловую систему.

Остальная часть этой главы рассматривает каждый уровень, начиная с кэша буферов. Обращайте внимание на ситуации, когда удачно выбранные абстракции на нижних уровнях упрощают проектирование вышестоящих.

8.2 Уровень кэша буферов

Кэш буферов выполняет две задачи: (1) синхронизирует доступ к дисковым блокам, гарантируя, что в памяти присутствует только одна копия блока и что только один поток ядра одновременно использует эту копию; (2) кэширует часто используемые блоки, чтобы их не требовалось повторно считывать с медленного диска. Код находится в bio.c.

Основной интерфейс, экспортируемый кэшем буферов, состоит из bread и bwrite; первая функция получает buf содержащий копию блока, который можно читать или изменять в памяти, а вторая записывает изменённый буфер в соответствующий блок на диске. Поток ядра должен освободить буфер, вызвав brelse по завершении работы с ним. Кэш буферов использует спящую блокировку для каждого буфера, чтобы гарантировать, что только один поток одновременно использует каждый буфер (и соответственно каждый дисковый блок); bread возвращает заблокированный буфер, а brelse снимает блокировку.

Вернёмся к кэшу буферов. Кэш буферов имеет фиксированное количество буферов для хранения дисковых блоков, а это означает, что если файловая система запрашивает блок, которого ещё нет в кэше, кэш буферов должен переиспользовать буфер, в котором в данный момент хранится какой-то другой блок. Кэш буферов переиспользует наименее недавно использованный буфер для нового блока. Предполагается, что наименее недавно использованный буфер — это тот, который с наименьшей вероятностью будет использован в ближайшее время.

8.3 Код: кэш буферов

Кэш буферов — это двусвязный список буферов. Функция binit, вызываемая main (kernel/main.c:27), инициализирует список NBUF буферами из статического массива buf (kernel/bio.c:43-52). Все остальные обращения к кэшу буферов ссылаются на связный список через bcache.head, а не на массив buf.

Буфер имеет два поля состояния. Поле valid указывает, что буфер содержит копию блока. Поле disk указывает, что содержимое буфера было передано диску, который может изменить буфер (например, записать данные с диска в data).

bread (kernel/bio.c:93) вызывает bget для получения буфера для заданного сектора (kernel/bio.c:97). Если буфер нужно считать с диска, bread вызывает virtio_disk_rw для этого перед возвратом буфера.

bget (kernel/bio.c:59) сканирует список буферов в поисках буфера с заданными номером устройства и номером сектора (kernel/bio.c:65-73). Если такой буфер найден, bget захватывает спящую блокировку для этого буфера. bget затем возвращает заблокированный буфер.

Если в кэше нет буфера для заданного сектора, bget должна создать его, возможно переиспользовав буфер, который хранил другой сектор. Она повторно сканирует список буферов в поисках буфера, который не используется (b->refcnt = 0); любой такой буфер может быть использован. bget редактирует метаданные буфера для записи нового номера устройства и сектора и захватывает его спящую блокировку. Обратите внимание, что присвоение b->valid = 0 гарантирует, что bread считает данные блока с диска, а не ошибочно использует прежнее содержимое буфера.

Важно, чтобы для каждого дискового сектора существовал не более одного кэшированного буфера, чтобы читатели видели записи, и потому что файловая система использует блокировки на буферах для синхронизации. bget обеспечивает этот инвариант, удерживая bache.lock непрерывно от проверки наличия блока в кэше в первом цикле до объявления блока кэшированным во втором цикле (путём установки dev, blockno, и refcnt). Это делает проверку наличия блока и (при его отсутствии) выделение буфера для блока атомарной операцией.

Безопасно, что bget захватывает спящую блокировку буфера за пределами критической секции bcache.lock, поскольку ненулевое значение b->refcnt предотвращает переиспользование буфера для другого дискового блока. Спящая блокировка защищает чтение и запись кэшированного содержимого блока, тогда как bcache.lock защищает информацию о том, какие блоки кэшированы.

Если все буферы заняты, то слишком много процессов одновременно выполняют вызовы файловой системы; bget завершается паникой. Более корректным решением было бы переключиться в ожидание до освобождения буфера, хотя в этом случае возможна взаимная блокировка.

Как только bread считает данные с диска (если необходимо) и вернёт буфер вызывающему коду, вызывающий получает исключительный доступ к буферу и может читать или записывать байты данных. Если вызывающий изменяет буфер, он должен вызвать bwrite для записи изменённых данных на диск перед освобождением буфера.

bwrite (kernel/bio.c:107) вызывает virtio_disk_rw для взаимодействия с дисковым оборудованием.

Когда вызывающий завершает работу с буфером, он должен вызвать brelse для его освобождения. (Название brelse, являющееся сокращением от b-release, выглядит непонятно, но его стоит запомнить: оно берёт начало в Unix и используется также в BSD, Linux и Solaris.) brelse (kernel/bio.c:117) снимает спящую блокировку и перемещает буфер в начало связного списка (kernel/bio.c:128-133). Перемещение буфера обеспечивает упорядочение списка по давности использования буферов (то есть по времени освобождения): первый буфер в списке — наиболее недавно использованный, а последний — наименее недавно использованный. Два цикла в bget используют это преимущество: поиск существующего буфера в худшем случае должен обработать весь список, но проверка наиболее недавно использованных буферов в первую очередь (начиная с bcache.head и следуя по указателям next) сократит время сканирования при хорошей локальности обращений. Сканирование для выбора буфера для переиспользования выбирает наименее недавно используемый буфер путём обратного сканирования (следуя по указателям prev).

8.4 Уровень журналирования

Одна из наиболее интересных проблем при проектировании файловой системы — восстановление после сбоев. Проблема возникает потому, что многие операции файловой системы включают несколько операций записи на диск, и сбой после части из них может оставить файловую систему на диске в несогласованном состоянии. Например, предположим, что сбой происходит во время усечения файла (установки длины файла в ноль и освобождения его блоков содержимого). В зависимости от порядка операций записи на диск, сбой может оставить inode со ссылкой на блок содержимого, помеченный как свободный, или оставить выделенный, но не имеющий ссылок блок содержимого.

Последнее относительно безвредно, но inode, ссылающийся на освобождённый блок, вероятно, вызовет серьёзные проблемы после перезагрузки. После перезагрузки ядро может выделить этот блок другому файлу, и теперь два разных файла будут непреднамеренно указывать на один блок. Если бы xv6 поддерживал несколько пользователей, эта ситуация могла бы стать проблемой безопасности, поскольку владелец старого файла смог бы читать и записывать блоки в новый файл, принадлежащий другому пользователю.

Xv6 решает проблему сбоев во время операций файловой системы с помощью простой формы журналирования. Системный вызов xv6 не записывает структуры данных файловой системы на диск напрямую. Вместо этого он помещает описание всех операций записи на диск, которые он хочет произвести, в log на диске. Как только системный вызов записал в журнал все свои операции, он записывает специальную commit запись на диск, указывающую, что журнал содержит завершённую операцию. После этого системный вызов копирует записи в структуры данных файловой системы на диске. После завершения этих записей системный вызов стирает журнал на диске.

Если система должна завершить работу со сбоем и перезагрузиться, код файловой системы восстанавливается после сбоя следующим образом, перед запуском любых процессов. Если журнал помечен как содержащий завершённую операцию, код восстановления копирует записи в соответствующие места файловой системы на диске. Если журнал не помечен как содержащий завершённую операцию, код восстановления игнорирует журнал. Код восстановления завершает работу, стирая журнал.

Почему журнал xv6 решает проблему сбоев во время операций файловой системы? Если сбой происходит до фиксации операции, то журнал на диске не будет помечен как завершённый, код восстановления проигнорирует его, и состояние диска будет таким, как если бы операция вовсе не начиналась. Если сбой происходит после фиксации операции, то при восстановлении будут повторно воспроизведены все записи операции, возможно повторив их, если операция уже начала их запись в структуры данных на диске. В любом случае журнал делает операции атомарными по отношению к сбоям: после восстановления либо все записи операции присутствуют на диске, либо ни одна из них не присутствует.

8.5 Проектирование журнала

Журнал располагается по известному фиксированному адресу, указанному в суперблоке. Он состоит из заголовочного блока, за которым следует последовательность копий обновлённых блоков («журналируемые блоки»). Заголовочный блок содержит массив номеров секторов, по одному для каждого из журналируемых блоков, и счётчик журналируемых блоков. Счётчик в заголовочном блоке на диске равен либо нулю, что указывает на отсутствие транзакции в журнале, либо ненулевому значению, что указывает на наличие в журнале завершённой зафиксированной транзакции с указанным количеством журналируемых блоков. Xv6 записывает заголовочный блок при фиксации транзакции, но не раньше, и устанавливает счётчик в ноль после копирования журналируемых блоков в файловую систему. Таким образом, сбой в середине транзакции даст нулевой счётчик в заголовочном блоке журнала; сбой после фиксации даст ненулевой счётчик.

Код каждого системного вызова обозначает начало и конец последовательности записей, которые должны быть атомарными по отношению к сбоям. Чтобы позволить параллельное выполнение операций файловой системы разными процессами, система журналирования может накапливать записи нескольких системных вызовов в одной транзакции. Таким образом, одна фиксация может включать записи нескольких завершённых системных вызовов. Чтобы избежать разбиения системного вызова между транзакциями, система журналирования фиксирует транзакцию только тогда, когда не выполняется ни одного системного вызова файловой системы.

Идея совместной фиксации нескольких транзакций известна как group commit. Групповая фиксация сокращает количество дисковых операций, поскольку распределяет фиксированную стоимость фиксации между несколькими операциями. Групповая фиксация также передаёт дисковой системе сразу больше параллельных операций за-

писи, возможно позволяя диску записать все их за один оборот диска. Драйвер virtio в xv6 не поддерживает такую batching, но проектирование файловой системы xv6 это допускает.

Xv6 выделяет фиксированный объём пространства на диске для хранения журнала. Суммарное количество блоков, записываемых системными вызовами в транзакции, должно вписываться в это пространство. Это имеет два следствия. Ни один системный вызов не должен записывать больше отдельных блоков, чем есть места в журнале. Для большинства системных вызовов это не проблема, но два из них потенциально могут записывать много блоков: write и unlink. Большая операция записи в файл может записать много блоков данных и много блоков битовой карты, а также блок inode; удаление большого файла может записать много блоков битовой карты и блок inode. Системный вызов записи xv6 разбивает большие записи на несколько меньших, вписывающихся в журнал, а unlink не вызывает проблем, поскольку на практике файловая система xv6 использует только один блок битовой карты. Другое следствие ограниченного пространства журнала состоит в том, что система журналирования не может позволить системному вызову начаться, если не уверена, что его записи вместятся в оставшееся пространство журнала.

8.6 Код: журналирование

Типичное использование журнала в системном вызове выглядит следующим образом:

```
begin_op();
...
bp = bread(...);
bp->data[...] = ...;
log_write(bp);
...
end_op();
```

begin_op (kernel/log.c:128) ждёт, пока система журналирования не завершит текущую фиксацию, и пока не появится достаточно незарезервированного пространства журнала для хранения записей этого вызова. log.outstanding подсчитывает количество системных вызовов, зарезервировавших пространство журнала; суммарное зарезервированное пространство равно log.outstanding умноженному на MAXOPBLOCKS. Увеличение log.outstanding одновременно резервирует пространство и предотвращает фиксацию во время этого системного вызова. Код консервативно предполагает, что каждый системный вызов может записать до MAXOPBLOCKS отдельных блоков.

log_write (kernel/log.c:216) выступает прокси для bwrite. Она записывает номер сектора блока в памяти, резервируя ему слот в журнале на диске, и закрепляет буфер в кэше блоков, чтобы кэш блоков не вытеснил его. Блок должен оставаться в кэше до фиксации: до этого момента кэшированная копия —единственная запись об изменении; она не может быть записана на своё место на диске до фиксации; и другие операции

чтения в той же транзакции должны видеть эти изменения. log_write обнаруживает, когда блок записывается несколько раз во время одной транзакции, и выделяет этому блоку один и тот же слот в журнале. Эта оптимизация часто называется absorption. Часто бывает так, что, например, дисковый блок, содержащий inode нескольких файлов, записывается несколько раз в рамках транзакции. Поглощая несколько дисковых записей в одну, файловая система может сэкономить место в журнале и добиться лучшей производительности, поскольку на диск нужно записать лишь одну копию дискового блока.

end_op (kernel/log.c:148) сначала уменьшает счётчик активных системных вызовов. Если счётчик стал равен нулю, фиксируется текущая транзакция путём вызова commit(). В этом процессе четыре этапа. write_log() (kernel/log.c:180) копирует каждый блок, изменённый в транзакции, из кэша буферов в его слот в журнале на диске. write_head() (kernel/log.c:104) записывает заголовочный блок на диск: это точка фиксации, и сбой после этой записи приведёт к тому, что при восстановлении будут воспроизведены записи транзакции из журнала. install_trans (kernel/log.c:67) считывает каждый блок из журнала и записывает его в нужное место в файловой системе. Наконец, end_op записывает заголовок журнала со счётчиком, равным нулю; это должно произойти до того, как следующая транзакция начнёт записывать журналируемые блоки, чтобы сбой не привёл к использованию при восстановлении заголовка одной транзакции с журналируемыми блоками следующей.

recover_from_log (kernel/log.c:118) вызывается из initlog (kernel/log.c:54), которая вызывается из fsinit (kernel/fs.c:42) при загрузке, до запуска первого пользовательского процесса (kernel/proc.c:519). Она считывает заголовок журнала и имитирует действия end_op, если заголовок указывает, что журнал содержит зафиксированную транзакцию.

Пример использования журнала — функция filewrite (kernel/file.c:135). Транзакция выглядит следующим образом:

```
begin_op();
ilock(f->ip);
r = writei(f->ip, ...);
iunlock(f->ip);
end_op();
```

Этот код обёрнут в цикл, разбивающий большие записи на отдельные транзакции по несколько секторов, чтобы не переполнить журнал. Вызов writei записывает много блоков в рамках этой транзакции: inode файла, один или несколько блоков битовой карты и некоторые блоки данных.

8.7 Код: аллокатор блоков

Содержимое файлов и каталогов хранится в дисковых блоках, которые должны быть выделены из пула свободных блоков. Аллокатор блоков xv6 поддерживает битовую карту свободных блоков на диске, по одному биту на блок. Нулевой бит означает,

что соответствующий блок свободен; единичный бит означает, что он используется. Программа mkfs устанавливает биты, соответствующие загрузочному сектору, суперблоку, блокам журнала, блокам inode и блокам битовой карты.

Аллокатор блоков предоставляет две функции: balloc выделяет новый дисковый блок, а bfree освобождает блок. balloc Цикл в balloc в строке (kernel/fs.c:73) рассматривает каждый блок, начиная с блока 0 до sb.size, количества блоков в файловой системе. Он ищет блок, чей бит в битовой карте равен нулю, что означает, что блок свободен. Если balloc находит такой блок, он обновляет битовую карту и возвращает блок. Для эффективности цикл разбит на две части. Внешний цикл читает каждый блок битов битовой карты. Внутренний цикл проверяет все Bits-Per-Block (BPB) бит в одном блоке битовой карты. Гонка, которая может возникнуть, если два процесса попытаются выделить блок одновременно, предотвращается тем, что кэш буферов позволяет только одному процессу одновременно использовать любой блок битовой карты.

bfree (kernel/fs.c:93) находит нужный блок битовой карты и сбрасывает соответствующий бит. Здесь также монопольное использование, обеспечиваемое bread и brelse, позволяет избежать явной блокировки.

Как и большая часть кода, описанного в остальной части этой главы, balloc и bfree должны вызываться внутри транзакции.

8.8 Уровень inode

Термин inode может иметь одно из двух связанных значений. Он может относиться к структуре данных на диске, содержащей размер файла и список номеров блоков данных. Или «inode» может относиться к inode в памяти, которая содержит копию inode на диске, а также дополнительную информацию, необходимую в ядре.

Inode на диске упакованы в непрерывную область диска, называемую блоками inode. Каждый inode имеет одинаковый размер, поэтому легко, зная число n, найти n-й inode на диске. Собственно, это число n, называемое номером inode или i-номером, является способом идентификации inode в реализации.

Inode на диске определяется структурой struct dinode (kernel/fs.h:32). Поле type различает файлы, каталоги и специальные файлы (устройства). Нулевой тип указывает, что inode на диске свободен. Поле nlink подсчитывает количество записей каталога, ссылающихся на этот inode, чтобы определить момент, когда inode на диске и его блоки данных должны быть освобождены. Поле size записывает количество байт содержимого в файле. Массив addrs записывает номера блоков дисковых блоков, хранящих содержимое файла.

Ядро хранит набор активных inode в памяти в таблице под названием itable; struct inode (kernel/file.h:17) — это копия в памяти struct dinode на диске. Ядро хранит inode в памяти только при наличии C-указателей, ссылающихся на этот inode. Поле ref подсчитывает количество C-указателей, ссылающихся на inode в памяти, и ядро удаляет inode из памяти, если счётчик ссылок падает до нуля. Функции iget и iput захватывают и освобождают указатели на inode, изменяя счётчик ссылок. Указатели

на inode могут поступать из файловых дескрипторов, текущих рабочих каталогов и временного кода ядра, например exec.

В коде inode xv6 имеется четыре механизма блокировки или подобных им. itable.lock защищает инвариант, что inode присутствует в таблице inode не более одного раза, и инвариант, что поле ref inode в памяти подсчитывает количество указателей на inode в памяти. Каждый inode в памяти имеет поле lock, содержащее спящую блокировку, которая обеспечивает исключительный доступ к полям inode (таким как длина файла), а также к блокам содержимого файла или каталога inode. Поле inode ref, если оно больше нуля, заставляет систему поддерживать inode в таблице и не переиспользовать запись таблицы для другого inode. Наконец, каждый inode содержит поле nlink (на диске и скопированное в памяти, если inode находится в памяти), которое подсчитывает количество записей каталога, ссылающихся на файл; xv6 не освобождает inode, если его счётчик ссылок больше нуля.

Указатель на struct inode, возвращённый iget(), гарантированно остаётся действительным до соответствующего вызова iput(); inode не будет удалён, и память, на которую указывает указатель, не будет переиспользована для другого inode. iget() предоставляет неисключительный доступ к inode, так что может существовать много указателей на один и тот же inode. Многие части кода файловой системы зависят от этого поведения iget(), как для долгосрочных ссылок на inode (как открытые файлы и текущие каталоги), так и для предотвращения гонок при избежании взаимоблокировок в коде, работающем с несколькими inode (например, при поиске по имени пути).

Структура struct inode, возвращённая iget, может не содержать никакого полезного содержимого. Чтобы убедиться, что она содержит копию inode на диске, код должен вызвать ilock. Это блокирует inode (чтобы ни один другой процесс не мог ilock его) и считывает inode с диска, если это ещё не было сделано. iunlock снимает блокировку с inode. Разделение захвата указателей на inode и блокировки помогает избежать взаимоблокировок в некоторых ситуациях, например при поиске в каталоге. Несколько процессов могут держать C-указатель на inode, возвращённый iget, но только один процесс может блокировать inode одновременно.

Таблица inode хранит только те inode, на которые содержат C-указатели код ядра или структуры данных ядра. Её основная задача — синхронизация доступа нескольких процессов. Таблица inode также кэширует часто используемые inode, но кэширование вторично; если inode используется часто, кэш буферов, вероятно, будет держать его в памяти. Код, изменяющий inode в памяти, записывает его на диск с помощью iupdate.

8.9 Код: inode

Для выделения нового inode (например, при создании файла) xv6 вызывает ialloc (kernel/fs.c:200). ialloc аналогична balloc: она перебирает структуры inode на диске, по одному блоку за раз, в поисках той, которая помечена как свободная. Обнаружив её, она захватывает её, записывая новый type на диск, а затем возвращает запись из таблицы inode с хвостовым вызовом iget (kernel/fs.c:214). Корректная работа ialloc зависит

от того факта, что только один процесс одновременно может держать ссылку на bp: ialloc может быть уверена, что никакой другой процесс не увидит inode как доступный одновременно и не попытается его захватить.

iget (kernel/fs.c:248) просматривает таблицу inode в поисках активной записи (ip->ref > 0) с нужным устройством и номером inode. Если она находит её, возвращается новая ссылка на этот inode (kernel/fs.c:257-261). При сканировании iget записывает позицию первого пустого слота (kernel/fs.c:262-263), который используется при необходимости выделить запись в таблице.

Код должен блокировать inode с помощью ilock перед чтением или записью его метаданных или содержимого. ilock (kernel/fs.c:294) использует для этой цели спящую блокировку. Как только ilock получает исключительный доступ к inode, он считывает inode с диска (скорее всего, из кэша буферов), если необходимо. Функция iunlock (kernel/fs.c:322) снимает спящую блокировку, что может разбудить все ожидающие процессы.

iput (kernel/fs.c:338) освобождает C-указатель на inode, уменьшая счётчик ссылок (kernel/fs.c:361). Если это последняя ссылка, слот inode в таблице inode теперь свободен и может быть переиспользован для другого inode.

Если iput обнаруживает, что нет C-указателей на inode и что inode не имеет ссылок на него (не присутствует ни в одном каталоге), то inode и его блоки данных должны быть освобождены. iput вызывает itrunc для усечения файла до нуля байт, освобождая блоки данных; устанавливает тип inode в 0 (нераспределённый); и записывает inode на диск (kernel/fs.c:343).

Протокол блокировки в iput в случае освобождения inode заслуживает более пристального внимания. Одна из опасностей состоит в том, что параллельный поток может ждать в ilock для использования этого inode (например, для чтения файла или просмотра каталога) и не будет готов обнаружить, что inode больше не выделен. Этого не может произойти, потому что нет способа для системного вызова получить указатель на inode в памяти, если у него нет ссылок на него и ip->ref равен одному. Эта единственная ссылка принадлежит потоку, вызывающему iput. Верно, что iput проверяет, что счётчик ссылок равен одному, вне критической секции itable.lock, но в этот момент счётчик ссылок каталога известно равен нулю, поэтому ни один поток не попытается захватить новую ссылку. Другая основная опасность состоит в том, что параллельный вызов ialloc может выбрать тот же inode, который iput освобождает. Это может произойти только после того, как iupdate записывает на диск, чтобы inode имел нулевой тип. Эта гонка безвредна; поток, выделяющий inode, вежливо подождёт захвата спящей блокировки inode перед чтением или записью inode, к тому моменту iput с ним завершит работу.

iput() может записывать на диск. Это означает, что любой системный вызов, использующий файловую систему, может записывать на диск, поскольку системный вызов может быть последним, имеющим ссылку на файл. Даже вызовы вроде read(), которые кажутся операциями только чтения, могут в конечном итоге вызвать iput(). Это, в свою очередь, означает, что даже системные вызовы только чтения должны быть обёрнуты в транзакции, если они используют файловую систему.

Существует сложное взаимодействие между iput() и сбоями. iput() не усекает файл немедленно, когда счётчик ссылок файла падает до нуля, потому что какой-то процесс ещё может держать ссылку на inode в памяти: процесс всё ещё может читать и записывать файл, поскольку успешно его открыл. Но если сбой произойдёт до того, как последний процесс закроет файловый дескриптор для этого файла, то файл будет помечен выделённым на диске, но ни одна запись каталога не будет указывать на него.

Файловые системы справляются с этим случаем одним из двух способов. Простое решение состоит в том, что при восстановлении после перезагрузки файловая система сканирует всю файловую систему на предмет файлов, которые помечены выделёнными, но на которые не указывает ни одна запись каталога. Если такие файлы существуют, их можно освободить.

Второе решение не требует сканирования файловой системы. В этом решении файловая система записывает на диск (например, в суперблок) i-номер inode файла, счётчик ссылок которого упал до нуля, но счётчик ссылок в памяти не равен нулю. Если файловая система удаляет файл, когда его счётчик ссылок в памяти достигает нуля, то она обновляет список на диске, удаляя этот inode из списка. При восстановлении файловая система освобождает все файлы из списка.

Xv6 не реализует ни одно из этих решений, что означает, что inode могут быть помечены выделёнными на диске, даже если они больше не используются. Это означает, что со временем xv6 рискует исчерпать дисковое пространство.

8.10 Код: содержимое inode

Структура inode на диске, struct dinode, содержит размер и массив номеров блоков (см. Figure 8.3). Данные inode находятся в блоках, перечисленных в массиве addrs dinode. Первые NDIRECT блоков данных перечислены в первых NDIRECT записях массива; эти блоки называются direct blocks. Следующие NINDIRECT блоков данных перечислены не в inode, а в блоке данных, называемом indirect block. Последняя запись в массиве addrs даёт адрес косвенного блока. Таким образом, первые 12 кБ (NDIRECT x BSIZE) байт файла можно загрузить из блоков, перечисленных в inode, тогда как следующие 256 кБ (NINDIRECT x BSIZE) байт можно загрузить только после обращения к косвенному блоку. Это хорошее, но непростое для клиентов представление на диске. Функция bmap управляет этим представлением, чтобы подпрограммы более высокого уровня, такие как readi и writei, которые мы скоро рассмотрим, не нуждались в управлении этой сложностью. bmap возвращает номер дискового блока для bn-го блока данных inode ip. Если у ip ещё нет такого блока, bmap выделяет его.

Функция bmap (kernel/fs.c:406) начинает с простого случая: первые NDIRECT блоков перечислены непосредственно в inode (kernel/fs.c:411-419). Следующие NINDIRECT блоков перечислены в косвенном блоке по адресу ip->addrs[NDIRECT]. bmap читает косвенный блок (kernel/fs.c:430) и затем читает номер блока из нужной позиции внутри блока (kernel/fs.c:431). Если номер блока превышает NDIRECT+NINDIRECT, bmap завершается паникой; writei содержит проверку, предотвращающую это (kernel/fs.c:536).

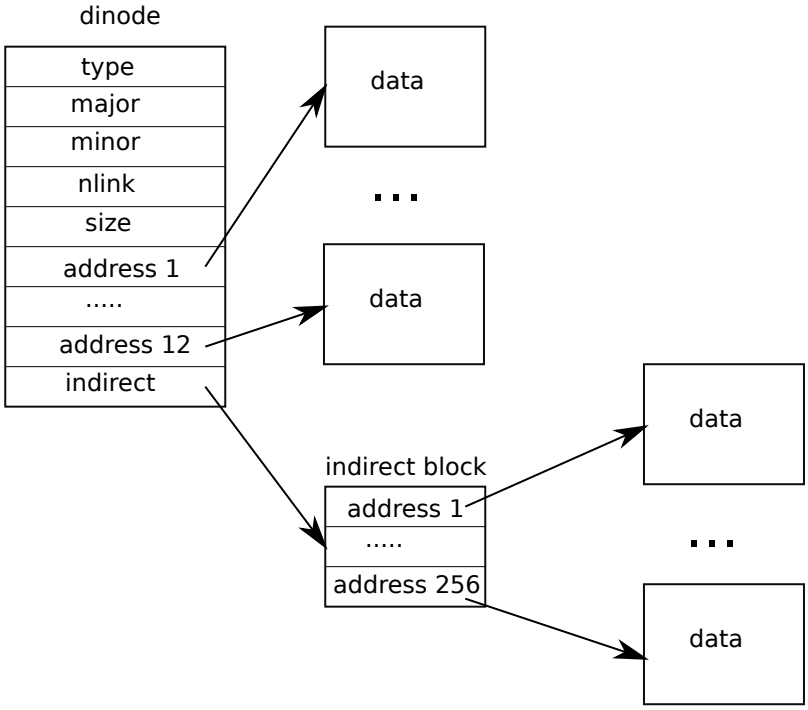

Рис. 8.3: Представление файла на диске.

bmap выделяет блоки по мере необходимости. Нулевое значение ip->addrs[] или косвенной записи указывает, что блок не выделен. Когда bmap обнаруживает нули, она заменяет их номерами свежих блоков, выделяемых по требованию (kernel/fs.c:412-413) (kernel/fs.c:424-425).

itrunc освобождает блоки файла, сбрасывая размер inode до нуля. itrunc (kernel/fs.c:449) начинает с освобождения прямых блоков (kernel/fs.c:455-460), затем перечисленных в косвенном блоке (kernel/fs.c:465-468), и наконец самого косвенного блока (kernel/fs.c:470-471).

bmap упрощает работу readi и writei с данными inode. readi (kernel/fs.c:495) начинает с проверки, что смещение и счётчик не выходят за конец файла. Чтения, начинающиеся за концом файла, возвращают ошибку (kernel/fs.c:500-501), тогда как чтения, начинающиеся в конце или пересекающие конец файла, возвращают меньше байт, чем запрошено (kernel/fs.c:502-503). Основной цикл обрабатывает каждый блок файла, копируя данные из буфера в dst (kernel/fs.c:505-517). writei (kernel/fs.c:529) идентична readi, за тремя исключениями: записи, начинающиеся в конце или пересекающие конец файла, увеличивают файл до максимального размера (kernel/fs.c:536-537); цикл копирует данные в буферы, а не из них (kernel/fs.c:36); и если запись расширила файл, writei должна обновить его размер

Функция stati (kernel/fs.c:481) копирует метаданные inode в структуру stat, которая

доступна пользовательским программам через системный вызов stat.

8.11 Код: уровень каталогов

Каталог реализован внутренне почти так же, как и файл. Его inode имеет тип T_DIR, а его данные — последовательность записей каталога. Каждая запись — это struct dirent (kernel/fs.h:58), содержащая имя и номер inode. Имя имеет длину не более DIRSIZ (14) символов; если оно короче, оно заканчивается нулевым байтом (NULL, 0). Записи каталога с номером inode равным нулю свободны.

Функция dirlookup (kernel/fs.c:575) ищет в каталоге запись с заданным именем. Если она её находит, возвращается указатель на соответствующий inode, незаблокированный, и *poff устанавливается в байтовое смещение записи внутри каталога, на случай, если вызывающий захочет её отредактировать. Если dirlookup находит запись с правильным именем, она обновляет *poff и возвращает незаблокированный inode, полученный через iget. dirlookup —это причина, по которой iget возвращает незаблокированные inode. Вызывающий заблокировал dp, поэтому если поиск был для ., псевдонима текущего каталога, попытка заблокировать inode перед возвратом привела бы к повторной блокировке dp и взаимоблокировке. (Существуют более сложные сценарии взаимоблокировки, включающие несколько процессов и .., псевдоним родительского каталога; . — не единственная проблема.) Вызывающий может разблокировать dp и затем заблокировать ip, обеспечивая, что в каждый момент удерживается только одна блокировка.

Функция dirlink (kernel/fs.c:603) записывает новую запись каталога с заданными именем и номером inode в каталог dp. Если имя уже существует, dirlink возвращает ошибку (kernel/fs.c:609-613). Основной цикл читает записи каталога в поисках нераспределённой записи. Когда она находится, цикл прерывается досрочно (kernel/fs.c:586-587), со значением off, установленным в смещение доступной записи. В противном случае цикл завершается со значением off, установленным в dp->size. В любом случае dirlink добавляет новую запись в каталог, записывая по смещению off

8.12 Код: пути

Поиск по имени пути включает последовательность вызовов dirlookup, по одному для каждого компонента пути. namei (kernel/fs.c:710) вычисляет path и возвращает соответствующий inode. Функция nameiparent — это вариант: она останавливается перед последним элементом, возвращая inode родительского каталога и копируя последний элемент в name. Обе вызывают обобщённую функцию namex для выполнения основной работы.

namex (kernel/fs.c:675) начинает с определения того, откуда начинается разбор пути. Если путь начинается с косой черты, разбор начинается с корня; в противном случае — с текущего каталога (kernel/fs.c:679-682). Затем используется skipelem для последовательного рассмотрения каждого элемента пути (kernel/fs.c:684). Каждая итерация цик-

ла должна найти name в текущем inode ip. Итерация начинается с блокировки ip и проверки, что это каталог. Если нет, поиск завершается с ошибкой (kernel/fs.c:685-689). (Блокировка ip необходима не потому, что ip->type может измениться внезапно — он не может, — а потому что до выполнения ilock не гарантируется, что ip->type загружен с диска.) Если вызов — nameiparent и это последний элемент пути, цикл останавливается досрочно, согласно определению nameiparent; последний элемент пути уже скопирован в name, поэтому namex должна лишь вернуть незаблокированный ip (kernel/fs.c:690-694). Наконец, цикл ищет элемент пути с помощью dirlookup и готовится к следующей итерации, устанавливая ip = next (kernel/fs.c:695-700). Когда в пути не остаётся элементов, возвращается ip.

Процедура namex может выполняться долго: она может включать несколько дисковых операций для чтения inode и блоков каталогов для каталогов, пройденных по пути (если они не находятся в кэше буферов). Xv6 тщательно спроектирован так, что если вызов namex одним потоком ядра заблокирован на дисковом вводе-выводе, другой поток ядра, ищущий другой путь, может продолжать работу параллельно. namex блокирует каждый каталог в пути отдельно, чтобы поиски в разных каталогах могли выполняться параллельно.

Такой параллелизм порождает определённые сложности. Например, пока один поток ядра ищет путь, другой поток ядра может изменять дерево каталогов, удаляя каталог. Потенциальный риск состоит в том, что поиск может происходить в каталоге, который был удалён другим потоком ядра, и его блоки были переиспользованы для другого каталога или файла.

Xv6 избегает таких гонок. Например, при выполнении dirlookup в namex ищущий поток удерживает блокировку каталога и dirlookup возвращает inode, полученный с помощью iget. iget увеличивает счётчик ссылок inode. Только после получения inode от dirlookup namex освобождает блокировку каталога. Теперь другой поток может удалить inode из каталога, но xv6 ещё не удалит inode, поскольку счётчик ссылок inode всё ещё больше нуля.

Другой риск — взаимоблокировка. Например, next указывает на тот же inode, что и ip при поиске «.». Блокировка next до освобождения блокировки на ip привела бы к взаимоблокировке. Чтобы избежать этой взаимоблокировки, namex разблокирует каталог перед получением блокировки на next. Здесь мы снова видим, почему разделение между iget и ilock важно.

8.13 Уровень файловых дескрипторов

Интересная особенность Unix-интерфейса состоит в том, что большинство ресурсов в Unix представлены как файлы, включая такие устройства, как консоль, каналы и, конечно, сами файлы. Уровень файловых дескрипторов — это уровень, который обеспечивает это единообразие.

Xv6 даёт каждому процессу собственную таблицу открытых файлов, или файловых дескрипторов, как мы видели в Chapter 1. Каждый открытый файл представлен

struct file (kernel/file.h:1), которая является обёрткой над inode или каналом, а также смещением ввода-вывода. Каждый вызов open создаёт новый открытый файл (новый struct file): если несколько процессов открывают один и тот же файл независимо, различные экземпляры будут иметь разные смещения ввода-вывода. С другой стороны, один открытый файл (один и тот же struct file) может встречаться несколько раз в таблице файлов одного процесса и также в таблицах файлов нескольких процессов. Это произошло бы, если бы один процесс открыл файл с помощью open, а затем создал псевдонимы с помощью dup или передал его дочернему процессу с помощью fork. Счётчик ссылок отслеживает количество ссылок на конкретный открытый файл. Файл может быть открыт для чтения, записи или того и другого. Поля readable и writable отслеживают это.

Все открытые файлы в системе хранятся в глобальной таблице файлов, ftable. Таблица файлов имеет функции для выделения файла (filealloc), создания дублированной ссылки (filedup), освобождения ссылки (fileclose), и чтения и записи данных (fileread и filewrite).

Первые три следуют уже знакомой форме. filealloc (kernel/file.c:30) сканирует таблицу файлов в поисках файла без ссылок (f->ref == 0) и возвращает новую ссылку; filedup (kernel/file.c:48) увеличивает счётчик ссылок; а fileclose (kernel/file.c:60) уменьшает его. Когда счётчик ссылок файла достигает нуля, fileclose освобождает соответствующий канал или inode, в зависимости от типа.

Функции filestat , fileread , и filewrite реализуют операции stat, read, и write над файлами. filestat (kernel/file.c:88) разрешена только для inode и вызывает stati. fileread и filewrite проверяют, что операция разрешена режимом открытия, а затем передают вызов либо реализации канала, либо inode. Если файл представляет собой inode, fileread и filewrite используют смещение ввода-вывода как смещение для операции и затем продвигают его (kernel/file.c:122-123) (kernel/file.c:162-163). Каналы не имеют понятия смещения. Вспомним, что функции inode требуют от вызывающего обработки блокировки (kernel/file.c:94-96) (kernel/file.c:121-124) (kernel/file.c:161-164). Блокировка inode имеет удобный побочный эффект: смещения чтения и записи обновляются атомарно, так что несколько одновременных записей в один файл не могут перезаписать данные друг друга, хотя их записи могут чередоваться.

8.14 Код: системные вызовы

Благодаря функциям, предоставляемым нижними уровнями, реализация большинства системных вызовов тривиальна (см. (kernel/sysfile.c)). Есть несколько вызовов, заслуживающих более пристального рассмотрения.

Функции sys_link и sys_unlink редактируют каталоги, создавая или удаляя ссылки на inode. Они — ещё один хороший пример мощи использования транзакций. sys_link (kernel/sysfile.c:124) начинает с получения своих аргументов — двух строк: old и new (kernel/sysfile.c:129). Предполагая, что old существует и не является каталогом (kernel/sysfile.c:133-136), sys_link увеличивает счётчик ip->nlink. Затем sys_link вызыва-

ет nameiparent для нахождения родительского каталога и последнего элемента пути new (kernel/sysfile.c:149) и создаёт новую запись каталога, указывающую на inode old 's (kernel/sysfile.c:152). Новый родительский каталог должен существовать и находиться на том же устройстве, что и существующий inode: номера inode имеют уникальное значение только в пределах одного диска. Если возникает такая ошибка, sys_link должна вернуться и уменьшить ip->nlink.

Транзакции упрощают реализацию, поскольку требуют обновления нескольких дисковых блоков, но нет необходимости беспокоиться о порядке их выполнения. Они либо все успешно завершатся, либо ни одна. Например, без транзакций обновление ip->nlink перед созданием ссылки временно поставило бы файловую систему в небезопасное состояние, и сбой в промежутке мог бы вызвать хаос. С транзакциями об этом беспокоиться не нужно.

sys_link создаёт новое имя для существующего inode. Функция create (kernel/sysfile.c:246) создаёт новое имя для нового inode. Это обобщение трёх системных вызовов создания файлов: open с флагом O_CREATE создаёт новый обычный файл, mkdir создаёт новый каталог, а mkdev создаёт новый файл устройства. Как и sys_link, create начинает с вызова nameiparent для получения inode родительского каталога. Затем вызывает dirlookup для проверки, существует ли уже имя (kernel/sysfile.c:256). Если имя уже существует, поведение create's зависит от того, для какого системного вызова он используется: open имеет другую семантику, чем mkdir и mkdev. Если create используется от имени open (type == T_FILE) и существующее имя — это обычный файл, тогда open считает это успехом, поэтому create тоже (kernel/sysfile.c:260). В противном случае это ошибка (kernel/sysfile.c:261-262). Если имя ещё не существует, create выделяет новый inode с помощью ialloc (kernel/sysfile.c:265). Если новый inode — каталог, create инициализирует его записями . и .. Наконец, когда данные правильно инициализированы, create может связать его с родительским каталогом (kernel/sysfile.c:278). create, как и sys_link, удерживает две блокировки inode одновременно: ip и dp. Взаимоблокировка невозможна, потому что inode ip только что выделен: ни один другой процесс в системе не будет удерживать блокировку ip 's и затем пытаться заблокировать dp.

Используя create, легко реализовать sys_open, sys_mkdir, и sys_mknod. sys_open (kernel/sysfile.c:305) наиболее сложен, поскольку создание нового файла — лишь небольшая часть того, что он может делать. Если open передаётся флаг O_CREATE, он вызывает create (kernel/sysfile.c:320). В противном случае он вызывает namei (kernel/sysfile.c:326). create возвращает заблокированный inode, а namei нет, поэтому sys_open должен заблокировать inode самостоятельно. Это даёт удобное место для проверки, что каталоги открываются только для чтения, но не для записи. Предполагая, что inode был получен тем или иным способом, sys_open выделяет файл и файловый дескриптор (kernel/sysfile.c:344) и затем заполняет файл (kernel/sysfile.c:356-361). Обратите внимание, что ни один другой процесс не может получить доступ к частично инициализированному файлу, поскольку он находится только в таблице текущего процесса.

В главе 7 была рассмотрена реализация каналов ещё до того, как у нас появилась

файловая система. Функция sys_pipe связывает эту реализацию с файловой системой, предоставляя способ создать пару каналов. Её аргумент — указатель на пространство для двух целых чисел, куда будут записаны два новых файловых дескриптора. Затем она выделяет канал и устанавливает файловые дескрипторы.

8.15 Реальный мир

Кэш буферов в реальной операционной системе значительно сложнее, чем в xv6, но выполняет те же две задачи: кэширование и синхронизацию доступа к диску. Кэш буферов xv6, как и в V6, использует простую политику вытеснения по принципу LRU (наименее недавно использованный); существует множество более сложных политик, каждая из которых хороша для одних нагрузок и менее подходит для других. Более эффективный LRU-кэш избавился бы от связного списка, используя вместо него хеш-таблицу для поиска и кучу для вытеснения по LRU. Современные кэши буферов обычно интегрированы с системой виртуальной памяти для поддержки файлов, отображаемых в память.

Система журналирования xv6 неэффективна. Фиксация не может происходить параллельно с системными вызовами файловой системы. Система записывает в журнал целые блоки, даже если изменено лишь несколько байт в блоке. Выполняются синхронные записи в журнал, по одному блоку за раз, каждая из которых, вероятно, требует полного времени оборота диска. Реальные системы журналирования решают все эти проблемы.

Журналирование — не единственный способ обеспечить восстановление после сбоев. Ранние файловые системы использовали при перезагрузке специальную утилиту (например, программу UNIX fsck) для проверки каждого файла и каталога, а также списков свободных блоков и inode, в поисках несогласованностей и их устранения. Такая проверка может занимать часы для больших файловых систем, и бывают ситуации, когда невозможно устранить несогласованности так, чтобы исходные системные вызовы оставались атомарными. Восстановление из журнала гораздо быстрее и обеспечивает атомарность системных вызовов при сбоях.

Xv6 использует то же базовое размещение inode и каталогов на диске, что и ранний UNIX; эта схема оказалась на удивление устойчивой на протяжении многих лет. BSD's UFS/FFS и Linux's ext2/ext3 используют по существу те же структуры данных. Наиболее неэффективная часть разметки файловой системы — каталог, который требует линейного сканирования всех дисковых блоков при каждом поиске. Это разумно, когда каталоги занимают лишь несколько дисковых блоков, но обходится дорого для каталогов, содержащих много файлов. Microsoft Windows's NTFS, macOS's HFS и Solaris's ZFS, если назвать лишь несколько, реализуют каталог как сбалансированное дерево блоков на диске. Это сложно, но гарантирует логарифмическое время поиска в каталоге.

Xv6 наивно обращается к сбоям диска: если операция ввода-вывода диска завершается неудачей, xv6 завершается паникой. Насколько это оправданно, зависит от оборудования: если операционная система работает поверх специального оборудования, исполь-

зующего резервирование для маскировки сбоев диска, возможно, операционная система видит сбои настолько редко, что паника приемлема. С другой стороны, операционные системы, использующие обычные диски, должны ожидать сбоев и обрабатывать их более корректно, чтобы потеря одного блока в одном файле не влияла на использование остальной части файловой системы.

Xv6 требует, чтобы файловая система помещалась на одном дисковом устройстве и не изменяла размер. Поскольку большие базы данных и мультимедийные файлы постоянно повышают требования к хранилищу, операционные системы разрабатывают способы устранения узкого места «один диск на файловую систему». Базовый подход —объединение многих дисков в один логический диск. Аппаратные решения, такие как RAID, по-прежнему наиболее популярны, но нынешняя тенденция направлена на реализацию как можно большей части этой логики в программном обеспечении. Эти программные реализации, как правило, допускают широкую функциональность, например расширение или сжатие логического устройства путём добавления или удаления дисков на ходу. Конечно, уровень хранилища, который может динамически расширяться или сжиматься, требует файловой системы, способной делать то же самое: массив фиксированного размера блоков inode, используемый xv6, плохо работал бы в таких условиях. Разделение управления дисками и файловой системы может быть наиболее чистым дизайном, но сложный интерфейс между ними привёл к тому, что некоторые системы, такие как Sun's ZFS, объединили их.

Файловой системе xv6 не хватает многих других возможностей современных файловых систем; например, в ней отсутствует поддержка снимков и инкрементного резервного копирования.

Современные Unix-системы позволяют получать доступ ко многим видам ресурсов с помощью тех же системных вызовов, что и к дисковому хранилищу: именованные каналы, сетевые соединения, удалённо доступные сетевые файловые системы, а также интерфейсы мониторинга и управления, такие как /proc. Вместо операторов if в xv6 в fileread и filewrite, эти системы обычно дают каждому открытому файлу таблицу указателей на функции, по одному на операцию, и вызывают указатель на функцию для вызова реализации этого inode. Сетевые файловые системы и файловые системы на уровне пользователя предоставляют функции, которые преобразуют эти вызовы в сетевые RPC и ждут ответа перед возвратом.

8.16 Упражнения

1. Почему в balloc происходит паника? Может ли xv6 восстановиться?

2. Почему в ialloc происходит паника? Может ли xv6 восстановиться?

3. Почему filealloc не завершается паникой при исчерпании файлов? Почему это встречается чаще и потому стоит обработки?

4. Предположим, что файл, соответствующий ip, был удалён другим процессом между вызовами iunlock(ip) и dirlink внутри sys_link. Будет ли ссылка создана корректно? Почему или почему нет?

5. create выполняет четыре вызова функций (один к ialloc и три к dirlink), которые должны завершиться успешно. Если хотя бы один не завершится успешно, create вызывает panic. Почему это приемлемо? Почему ни один из этих четырёх вызовов не может завершиться неудачей?

6. sys_chdir вызывает iunlock(ip) перед iput(cp->cwd), который может попытаться заблокировать cp->cwd, однако откладывание iunlock(ip) до после iput не привело бы к взаимоблокировкам. Почему нет?

7. Реализуйте системный вызов lseek. Поддержка lseek потребует также изменения filewrite, чтобы заполнить пробелы в файле нулями, если lseek устанавливает off за пределы f->ip->size.

8. Добавьте O_TRUNC и O_APPEND в open, чтобы операторы > и >> работали в оболочке.

9. Измените файловую систему для поддержки символических ссылок.

10. Измените файловую систему для поддержки именованных каналов.

11. Измените файловую и виртуальную системы памяти для поддержки файлов, отображаемых в память.

Глава 9

Параллелизм: дополнительные аспекты

Одновременно обеспечить высокую параллельную производительность, корректность при конкурентном выполнении и понятный код —серьёзная задача при проектировании ядра. Прямолинейное использование блокировок —наилучший путь к корректности, однако он не всегда применим. В этой главе рассмотрены примеры, в которых xv6 вынужден использовать блокировки нетривиальным образом, а также случаи, где xv6 применяет техники, похожие на блокировки, но не являющиеся ими.

9.1 Паттерны блокировок

Кэшированные элементы нередко создают трудности при блокировании. Например, кэш блоков файловой системы (kernel/bio.c:26) хранит копии до NBUF дисковых блоков. Крайне важно, чтобы каждый дисковый блок имел не более одной копии в кэше; иначе разные процессы могут вносить конфликтующие изменения в разные копии того, что должно быть одним и тем же блоком. Каждый кэшированный блок хранится в struct buf (kernel/buf.h:1). В struct buf есть поле блокировки, которое помогает гарантировать, что только один процесс одновременно использует данный дисковый блок. Однако одной блокировки недостаточно: что если блок отсутствует в кэше, а два процесса хотят использовать его одновременно? Структуры struct buf для него нет (поскольку блок ещё не кэширован), и блокировать нечего. Xv6 решает эту задачу, связывая дополнительную блокировку (bcache.lock) с набором идентификаторов кэшированных блоков. Код, которому нужно проверить наличие блока в кэше (например, bget (kernel/bio.c:59)), или изменить набор кэшированных блоков, должен удерживать bcache.lock; после того как код нашёл нужный блок и struct buf, он может освободить bcache.lock и заблокировать только конкретный блок. Это распространённый паттерн: одна блокировка для набора элементов плюс по одной блокировке на каждый элемент.

Как правило, та же функция, которая захватывает блокировку, её и освобождает. Но более точный способ смотреть на это: блокировка захватывается в начале последовательности операций, которая должна выглядеть атомарной, и освобождается по её окончании. Если последовательность начинается и заканчивается в разных функциях,

разных потоках или на разных процессорах, то захват и освобождение блокировки тоже происходят в разных местах. Назначение блокировки — заставить другие использования ждать, а не привязать данные к конкретному исполнителю. Один из примеров — acquire в yield (kernel/proc.c:494), который освобождается в потоке планировщика, а не в захватившем процессе. Другой пример — acquiresleep в ilock (kernel/fs.c:294); этот код нередко засыпает при чтении диска; он может проснуться на другом процессоре, что означает, что блокировка может быть захвачена и освобождена на разных процессорах.

Освобождение объекта, защищённого блокировкой, встроенной в сам объект, — тонкое дело, поскольку владения блокировкой недостаточно для гарантии корректности освобождения. Проблема возникает, когда другой поток ожидает в acquire для использования объекта; освобождение объекта неявно освобождает встроенную блокировку, что приведёт к некорректной работе ожидающего потока. Одно из решений — отслеживать, сколько ссылок на объект существует, чтобы освобождать его только тогда, когда исчезает последняя ссылка. Пример — pipeclose (kernel/pipe.c:59); pi->readopen и pi->writeopen отслеживают, имеются ли файловые дескрипторы, ссылающиеся на канал.

Обычно блокировки обрамляют последовательности чтения и записи наборов связанных элементов; блокировки гарантируют, что другие потоки видят только завершённые последовательности обновлений (при условии, что они тоже используют блокировки). А как быть в ситуациях, когда обновление — это простая запись в одну общую переменную? Например, setkilled и killed (kernel/proc.c:614) используют блокировку вокруг простых операций с p->killed. Если бы блокировки не было, один поток мог бы записывать p->killed в то же время, когда другой поток читает её. Это гонка, и спецификация языка C гласит, что гонка порождает неопределённое поведение, то есть программа может аварийно завершиться или выдать некорректные результаты[1]. Блокировки предотвращают гонку и избавляют от неопределённого поведения.

Одна из причин, по которым гонки могут ломать программы, состоит в том, что при отсутствии блокировок или эквивалентных конструкций компилятор может генерировать машинный код, который читает и записывает память совершенно иначе, чем исходный код на C. Например, машинный код потока, вызывающего killed, мог бы скопировать p->killed в регистр и читать только это кэшированное значение; это означало бы, что поток может никогда не увидеть никаких записей в p->killed. Блокировки предотвращают подобное кэширование.

9.2 Паттерны, похожие на блокировки

Во многих местах xv6 использует счётчик ссылок или флаг подобно блокировке, чтобы обозначить, что объект выделен и не должен быть освобождён или повторно использован. p->state процесса действует именно так, как и счётчики ссылок в структурах file, inode и buf. В каждом из этих случаев блокировка защищает флаг или счётчик ссылок, но именно последний предотвращает преждевременное освобождение объекта.

[1] «Threads and data races» в https://en.cppreference.com/w/c/language/memory_model

Файловая система использует счётчики ссылок struct inode как своеобразную разделяемую блокировку, которую может удерживать несколько процессов, чтобы избежать взаимоблокировок, которые возникли бы при использовании обычных блокировок. Например, цикл в namex (kernel/fs.c:675) последовательно блокирует каталог, задаваемый каждым компонентом пути. Однако namex должен освобождать каждую блокировку в конце итерации цикла, поскольку при удержании нескольких блокировок одновременно мог бы возникнуть дедлок с самим собой, если путь содержит точку (например, a/./b). Дедлок мог бы также возникнуть при параллельном поиске, затрагивающем данный каталог и ... Как объясняет Глава 8, решение состоит в том, что цикл передаёт inode каталога на следующую итерацию с увеличенным счётчиком ссылок, но без блокировки.

Некоторые элементы данных в разное время защищаются разными механизмами и порой защищены от конкурентного доступа неявно — через структуру кода xv6, а не явными блокировками. Например, когда физическая страница свободна, она защищена блокировкой kmem.lock (kernel/kalloc.c:24). Если страница затем выделяется как канал (kernel/pipe.c:23), она защищается другой блокировкой (встроенной pi->lock). Если страница перераспределяется для пользовательской памяти нового процесса, она не защищается никакой блокировкой. Вместо этого её защищает тот факт, что аллокатор не отдаст эту страницу другому процессу (пока она не будет освобождена). Владение памятью нового процесса устроено непросто: сначала родитель выделяет её и манипулирует ею в fork, затем её использует дочерний процесс, а после его завершения — родитель снова владеет памятью и передаёт её в kfree. Здесь можно выделить два урока: объект данных может быть защищён от конкурентного доступа разными способами в разные моменты своего жизненного цикла, а защита может иметь форму неявной структуры, а не явных блокировок.

Последний пример, похожий на блокировку, — необходимость отключать прерывания вокруг вызовов mycpu() (kernel/proc.c:83). Отключение прерываний делает вызывающий код атомарным по отношению к таймерным прерываниям, которые могут вызвать переключение контекста и тем самым переместить процесс на другой процессор.

9.3 Полное отсутствие блокировок

Существует несколько мест, где xv6 разделяет изменяемые данные без каких-либо блокировок. Одно из них — реализация спин-блокировок, хотя атомарные инструкции RISC-V можно рассматривать как опирающиеся на блокировки, реализованные в аппаратуре. Другое — переменная started в main.c (kernel/main.c:7), которая используется для предотвращения работы других процессоров до завершения инициализации xv6 нулевым процессором; volatile гарантирует, что компилятор действительно генерирует инструкции загрузки и сохранения.

В xv6 есть случаи, когда один процессор или поток записывает данные, а другой процессор или поток читает их, при этом нет специальной блокировки, выделенной для защиты этих данных. Например, в fork родитель записывает страницы пользова-

тельской памяти дочернего процесса, а дочерний процесс (другой поток, возможно, на другом процессоре) читает их; никакая блокировка явно не защищает эти страницы. Строго говоря, это не проблема блокировок, поскольку дочерний процесс не начинает выполнение, пока родитель не завершит запись. Это потенциальная проблема порядка обращений к памяти (см. Главу 6), поскольку без барьера памяти нет оснований ожидать, что один процессор увидит записи другого. Тем не менее, поскольку родитель освобождает блокировки, а дочерний процесс захватывает блокировки при запуске, барьеры памяти в acquire и release гарантируют, что процессор дочернего процесса увидит записи родителя.

9.4 Параллелизм

Блокировки прежде всего предназначены для подавления параллелизма в интересах корректности. Поскольку производительность тоже важна, разработчики ядра часто задумываются о том, как использовать блокировки так, чтобы добиться как корректности, так и параллелизма. Хотя xv6 не разрабатывался с прицелом на высокую производительность, всё же стоит подумать, какие операции xv6 могут выполняться параллельно, а какие могут конкурировать за блокировки.

Каналы в xv6 — пример достаточно хорошего параллелизма. Каждый канал имеет свою блокировку, поэтому разные процессы могут читать и писать в разные каналы параллельно на разных процессорах. Однако для конкретного канала писатель и читатель должны ждать, пока другой не освободит блокировку; они не могут читать/писать в один и тот же канал одновременно. Чтение из пустого канала (или запись в полный канал) также блокируется, но это не связано со схемой блокировок.

Переключение контекста — более сложный пример. Два потока ядра, каждый выполняющийся на своём процессоре, могут одновременно вызвать yield, sched и swtch, и эти вызовы будут выполняться параллельно. Каждый поток удерживает блокировку, но это разные блокировки, поэтому им не нужно ждать друг друга. Однако оказавшись в scheduler, два процессора могут конкурировать за блокировки при поиске в таблице процессов процесса в состоянии RUNNABLE. То есть xv6, вероятно, выигрывает в производительности от нескольких процессоров во время переключения контекста, но не настолько, насколько могло бы.

Ещё один пример — параллельные вызовы fork из разных процессов на разных процессорах. Вызовы могут вынужденно ждать друг друга из-за pid_lock и kmem.lock, а также из-за блокировок отдельных процессов, нужных для поиска процесса в состоянии UNUSED в таблице процессов. С другой стороны, оба форкающихся процесса могут копировать страницы пользовательской памяти и формировать страницы таблицы страниц в полностью параллельном режиме.

Схема блокировок в каждом из приведённых примеров жертвует параллельной производительностью в определённых случаях. В каждом случае можно добиться большего параллелизма с помощью более сложного проектирования. Целесообразность этого зависит от деталей: как часто вызываются соответствующие операции, сколько времени

код проводит, удерживая оспариваемую блокировку, сколько процессоров могут одновременно выполнять конфликтующие операции, являются ли другие части кода более узкими местами. Сложно угадать, создаёт ли данная схема блокировок проблемы с производительностью и насколько новое решение лучше, поэтому нередко требуются измерения на реалистичных рабочих нагрузках.

9.5 Упражнения

1. Измените реализацию каналов в xv6 так, чтобы разрешить чтению и записи в один и тот же канал выполняться параллельно на разных ядрах.

2. Измените scheduler() в xv6, чтобы уменьшить конкуренцию за блокировки при одновременном поиске процессов, готовых к выполнению, разными ядрами.

3. Устраните часть последовательного выполнения в fork() xv6.

Глава 10

Заключение

В этом тексте основные идеи операционных систем были представлены путём построчного изучения одной операционной системы — xv6. Одни строки кода воплощают суть ключевых идей (например, переключение контекста, граница пользователь/ядро, блокировки и т. д.), и каждая такая строка важна; другие строки кода иллюстрируют, как можно реализовать ту или иную идею операционной системы, и могут быть легко сделаны иначе (например, лучший алгоритм планирования, лучшие дисковые структуры данных для представления файлов, более продвинутое журналирование для поддержки параллельных транзакций и т. д.). Все идеи были рассмотрены в контексте одного конкретного, весьма успешного интерфейса системных вызовов — интерфейса Unix, однако эти идеи применимы и к проектированию других операционных систем.

Литература

[1] Linux common vulnerabilities and exposures (CVEs). https://cve.mitre.org/cgi-bin/cvekey.cgi?keyword=linux.

[2] The RISC-V instruction set manual Volume I: unprivileged ISA. https://github.com/riscv/riscv-isa-manual/releases/download/Ratified-IMAFDQC/riscv-spec-20191213.pdf, 2019.

[3] The RISC-V instruction set manual Volume II: privileged architecture. https://github.com/riscv/riscv-isa-manual/releases/download/Priv-v1.12/riscv-privileged-20211203.pdf, 2021.

[4] Hans-J Boehm. Threads cannot be implemented as a library. ACM PLDI Conference, 2005.

[5] Edsger Dijkstra. Cooperating sequential processes. https://www.cs.utexas.edu/users/EWD/transcriptions/EWD01xx/EWD123.html, 1965.

[6] Maurice Herlihy and Nir Shavit. The Art of Multiprocessor Programming, Revised Reprint. 2012.

[7] Brian W. Kernighan. The C Programming Language. Prentice Hall Professional Technical Reference, 2nd edition, 1988.

[8] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. Sel4: Formal verification of an OS kernel. In Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles, page 207–220, 2009.

[9] Donald Knuth. Fundamental Algorithms. The Art of Computer Programming. (Second ed.), volume 1. 1997.

[10] L Lamport. A new solution of dijkstra's concurrent programming problem. Communications of the ACM, 1974.

[11] John Lions. Commentary on UNIX 6th Edition. Peer to Peer Communications, 2000.

[12] Paul E. Mckenney, Silas Boyd-wickizer, and Jonathan Walpole. RCU usage in the linux kernel: One decade later, 2013.

[13] Martin Michael and Daniel Durich. The NS16550A: UART design and application considerations. http://bitsavers.trailing-edge.com/components/national/_appNotes/AN-0491.pdf, 1987.

[14] Aleph One. Smashing the stack for fun and profit. http://phrack.org/issues/49/14.html#article.

[15] David Patterson and Andrew Waterman. The RISC-V Reader: an open architecture Atlas. Strawberry Canyon, 2017.

[16] Dave Presotto, Rob Pike, Ken Thompson, and Howard Trickey. Plan 9, a distributed system. In In Proceedings of the Spring 1991 EurOpen Conference, pages 43–50, 1991.

[17] Dennis M. Ritchie and Ken Thompson. The UNIX time-sharing system. Commun. ACM, 17(7):365–375, July 1974.

Предметный указатель

., 114, 117
.., 114, 117
/init, 32, 45
_entry, 31

absorption, 108
acquire, 74, 79
argc, 45
argv, 45

balloc, 109, 110
batching, 107
bcache.head, 104
begin_op, 107
bfree, 109
bget, 104
binit, 104
block, 102
bmap, 112
bread, 103, 104
brelse, 103, 105
BSIZE, 112
buf, 103
bwrite, 103, 104, 107

chan, 89, 92
commit, 105
copyinstr, 56
copyout, 45
CPU, 10
cpu->context, 85
crash recovery, 101
create, 117

direct blocks, 112
dirlink, 114
dirlookup, 114, 115, 117
DIRSIZ, 114
disk, 104
dup, 116

ecall, 26, 30
ELF_MAGIC, 44
end_op, 108
exec, 12, 13, 15, 32, 45, 55
exit, 12, 93

filealloc, 116
fileclose, 116
filedup, 116
fileread, 116, 119
filestat, 116
filewrite, 108, 116, 119
fork, 11, 13, 15, 116
forkret, 86
freerange, 42
fsck, 118
fsinit, 108
ftable, 116

getcmd, 13
group commit, 106

hartid, 87

ialloc, 110, 117
iget, 109–111, 114
ilock, 110, 111, 115
indirect block, 112
initcode.S, 32, 55

initlog, 108
inode, 102, 109
install_trans, 108
interface design, 9
iput, 109, 111
itable, 109
itrunc, 111, 113
iunlock, 111

kalloc, 42
kfree, 42
kinit, 42
kvminit, 40
kvminithart, 41
kvmmake, 40
kvmmap, 40

loadseg, 45
log, 105
log_write, 107

main, 40–42, 104
malloc, 13
mappages, 40
mkdev, 117
mkdir, 117
mkfs, 103
mycpu, 87
myproc, 88

namei, 44, 117
nameiparent, 114, 115, 117
namex, 114, 115
NBUF, 104
NDIRECT, 112
NINDIRECT, 112

O_CREATE, 117
open, 116, 117

p->context, 87
p->killed, 95
p->kstack, 30
p->lock, 85–87, 91
p->pagetable, 30
p->state, 30
p->xxx, 30
persistence, 101
PGROUNDUP, 42
PHYSTOP, 40, 41
PID, 11
piperead, 93
pipewrite, 93
pop_off, 79
printf, 12
proc_mapstacks, 40
proc_pagetable, 45
PTE_R, 37
PTE_U, 38
PTE_V, 37
PTE_W, 38
PTE_X, 38
push_off, 79

read, 116
readi, 45, 112, 113
recover_from_log, 108
release, 74, 79
RUNNABLE, 86, 91–93

satp, 38
sbrk, 13
scause, 52
sched, 85, 86, 91
scheduler, 85, 86
sector, 102
sepc, 52
sfence.vma, 41
skipelem, 114
sleep, 89–91
SLEEPING, 91, 92
sret, 30
sscratch, 52
sstatus, 52
stat, 114, 116
stati, 113, 116
struct context, 85
struct cpu, 87

struct dinode, 109, 112
struct dirent, 114
struct elfhdr, 44
struct file, 116
struct inode, 109
struct pipe, 93
struct proc, 30
struct run, 42
struct spinlock, 73
stval, 58
stvec, 52
superblock, 103
swtch, 84–86
SYS_exec, 56
sys_link, 116, 117
sys_mkdir, 117
sys_mknod, 117
sys_open, 117
sys_pipe, 118
sys_sleep, 78
sys_unlink, 116
syscall, 56

T_DIR, 114
T_FILE, 117
ticks, 78
tickslock, 78
TRAMPOLINE, 54
transaction, 102
type cast, 42

UART, 63
unlink, 107
usertrap, 85
ustack, 45
uvmalloc, 45

valid, 104
virtio_disk_rw, 104, 105

wait, 12, 93
wakeup, 76, 89, 91
walk, 40
walkaddr, 45
write, 107, 116
writei, 108, 112, 113

yield, 85, 86

ZOMBIE, 94

Драйвер, 63
Канал (pipe), 17
Путь, 18
Файловый дескриптор, 13
адресное пространство, 29
активное ожидание, 89
атомарное, 74
барьером памяти, 80
блокировка, 69
блокировку условия, 90
блокировок сна, 80
буфере ассоциативной трансляции (TLB), 37, 41
вводом-выводом, 14
вектором, 52
верхняя половина, 63
взаимное исключение, 72
взаимной блокировки, 76
виртуальный адрес, 29
гонка, 122
гонки, 71
дочерним, 11
записей таблицы страниц (PTE), 35
изоляцию, 23
инверсия приоритетов, 96
иноде (inode), 19
исключение, 51
исключение при обращении к странице, 58
исключение страничного отказа, 36
каналом ожидания, 89
конвои, 96
конкурентность, 69
конкурентным вводом-выводом, 65
конкуренцию, 73
контекстами, 84
конфликтуют, 73

координации последовательности, 88
корневым, 18
критической секцией, 72
круговой очерёдностью, 96
ленивое выделение памяти, 59
ловушка, 51
машинный режим, 25
метаданные, 19
микроядром, 26
многоядерном, 23
моделью памяти, 80
монолитного ядра, 23
монолитным ядром, 26
мультиплексирования, 83
мультипроцессор, 23
неопределённое поведение, 122
нижняя половина, 63
области подкачки, 60
оболочка, 10
обработчиком, 52
опроса, 89
опросом (polling), 67
отображённых в память, 39
память пользователя, 29
перенаправление ввода-вывода, 15
подкачка страниц по требованию, 59
подкачку на диск, 60
пользовательский режим, 25
потерянного пробуждения, 90
поток, 30
прерывание, 51
прерывания о завершении передачи, 64
привилегированные инструкции, 25
программируемым вводом-выводом, 67
пространстве пользователя, 10, 25
пространстве ядра, 10, 25
процесс, 29
процессом, 9
прямой доступ к памяти (DMA), 67

разделяет время, 11
разделять по времени, 23
регистров управления, отображённых на память, 64
реентерабельные блокировки, 77
режим супервизора, 25
рекурсивными блокировками, 77
родительским, 11
семафора, 88
сигнал, 97
системный вызов, 10
сопрограммами, 86
ссылками (links), 19
стадом громов, 97
страницей, 36
страницу-ограничитель, 39
текущего каталога, 18
трамплина, 30, 54
упорядочивания, 73
управления конкурентностью, 69
условной синхронизации, 88
физический адрес, 29
форка с копированием при записи (COW), 58
формате ELF, 44
фрейма ловушки, 30
ядра, 9
ядром, 26